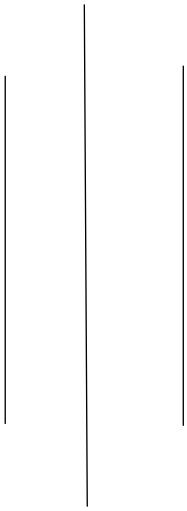


# Tribhuvan University

## Faculty of Humanities and Social Science

### Saraswati Multiple Campus

Lekhnath marga, Kathmandu



## Lab Sheet

**Submitted By:**

Name: Bikram Gyawali

Roll No: ….

Level: BCA (5th Semester)

Subject: Computer Graphics

Subject Code: CACS 305

**Submitted To:**

Name: Er. Manoj Giri

……

Signature

# Table of Contents


**I. 2 Dimensional Transformation** ..... 1

1. **Package for input, frame and calculation for custom and library function.** ..... 1
2. **2d translation using Custom function.** ..... 6
3. **2d translation using Library function.** ..... 8
4. **2d Scaling through origin point using Custom function.** ..... 10
5. **2d Scaling through arbitrary point using Custom function** ..... 12
6. **2d Scaling about Origin using Library function.** ..... 14
7. **2d Scaling through Arbitrary Point using Library function.** ..... 16
8. **2d AntiClock-Wise Rotation using Custom function.** ..... 18
9. **2d ClockWise Rotation using Custom function.** ..... 20
10. **2d Rotation through Arbitrary Point using Custom function.** ..... 22
11. **2d Anticlockwise Rotation using Library function** ..... 24
12. **2d Clockwise Rotation using Library function** ..... 26
13. **2d Rotation through arbitrary point using Library function.** ..... 28
14. **2d Reflection along X-Axis using Custom function.** ..... 30
15. **2d Reflection along X-Axis using Custom function.** ..... 32
16. **2d Reflection along X-Axis using Custom function.** ..... 34
17. **2d Reflection along Y=X using Custom function.** ..... 36
18. **2d Reflection along Y=mx+c using Custom function.** ..... 38
19. **2d Reflection along X Axis using Library function** ..... 40
20. **2d Reflection along Y Axis using Library function** ..... 42
21. **2d Reflection along Origin using Library function.** ..... 44
22. **2d Reflection along Y=X using Library function.** ..... 46
23. **2d Reflection along Y=mx+c using Library function.** ..... 48
24. **2d Shearing along X-Axis using Custom function.** ..... 50
25. **2d Shearing along Y-Axis using Custom function.** ..... 52
26. **2d Shearing along XY-Axis using Custom function.** ..... 54
27. **2d Shearing along X-Axis using Library function.** ..... 56
28. **2d Shearing along Y-Axis using Library function.** ..... 58
29. **2d Shearing along Xy-Axis using Library function.** ..... 60

**II. 3-Dimensional Transformation** ..... 62

1. **3D package for input coordinate, JFrame used by all the program.** ..... 62
2. **3D package for Library Function Programs.** ..... 67
3. **3d Translation using Custom function.** ..... 72
4. **3D Translation using Library Function.** ..... 74
5. **3D Scaling through origin using Custom Function.** ..... 76

6. 3D Scaling through selected Fixed-point using Custom Function. ..... 78
7. 3D Scaling through origin using Library Function. ..... 81
8. 3D Scaling though fixed-point using Library Function. ..... 83
9. 3D Rotation through X-Axis using Custom Function ..... 85
10. 3D Rotation through Y-Axis through Custom Function. ..... 87
11. 3D Rotation through Z-Axis using Custom Function. ..... 89
12. 3D Rotation through X-Axis using Library Function. ..... 91
13. 3D Rotation through Y-Axis using Library Function. ..... 93
14. 3D Rotation through Z-Axis using Library Function. ..... 95
15. 3D Reflection through ZX plane using Custom Function. ..... 97
16. 3D Reflection using YZ plane using Custom Function. ..... 99
17. 3D Reflection through XY plane using Custom Function. ..... 101
18. 3D Reflection through ZX plane using Library Function. ..... 103
19. 3D Reflection through YZ plane using Library Function. ..... 105
20. 3D Reflection through XY plane using Library Function. ..... 107
21. 3D Shearing along X-Axis using Custom Function. ..... 109
22. 3D Shearing through Y-Axis using Custom Function. ..... 111
23. 3D Shearing along Z-Axis using Custom Function. ..... 113
24. 3D Shearing along X-Axis using Library Function. ..... 115
25. 3D Shearing along Y-Axis using Library Function. ..... 117
26. 3D Shearing along Z-Axis using Library Function. ..... 119

III. Line Drawing Algorithm ..... 121
1. Program to create Package for input and swing. ..... 121
2. DDA Algorithm using Custom Function ..... 125
3. DDA Algorithm using Library Function ..... 127
4. Bresenham Line Drawing Algorithm using Custom Function. ..... 128
5. Bresenham Line Drawing using Library Funcition. ..... 130

IV. Circle Drawing Algorithm ..... 131
1. Program to create package use for input and swing. ..... 131
2. General Circle Drawing using Custom Function. ..... 133
3. General Circle Drawing using Library Function ..... 135
4. Mid-Point Circle Drawing using Custom Function. ..... 137
5. Mid-Point Circle Drawing using Library Function. ..... 139

V. Mid-Point Ellipse Drawing Algorithm ..... 140
1. Program to create package for input and swing. ..... 140
2. Ellipse Drawing using Custom Function. ..... 143
3. Ellipse Drawing using Library Function. ..... 146

**VI. Clipping Algorithm** ..... 147
**1. Program to create package for input and swing.** ..... 147
**2. Point Clipping using Custom Function** ..... 155
**3. Point Clipping using Library Function** ..... 157
**4. Cohen Sutherland Line Clipping using Custom Function** ..... 159
**5. Cohen Sutherland Line Clipping Algorithm** ..... 162
**6. Sutherland Hodgman Polygon Clipping Algorithm** ..... 164
**7. Sutherland Hodgeman Polygon Clipping Algorithm** ..... 167
**VII. Area Filling Algorithm** ..... 170
**1. Program to create a Package for input and swing** ..... 170
**2. Boundary-Fill Algorithm using Custom Function** ..... 172
**3. Boundary-Fill Algorithm using Library Function.** ..... 175
**4. Flood-Fill Algorithm using Custom Function** ..... 177
**5. Flood-Fill Algorithm using Library Function** ..... 180

# I. 2 Dimensional Transformation

## 1. Package for input, frame and calculation for custom and library function.

**Code:**

```
package package2D;
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;
public class BGhelperPackage {
   public int Xa, Ya;
   public int Xb, Yb;
   public int Xc, Yc;
   public void inputCoordinate(Scanner sa) {
      System.out.println("\nEnter Coordinate of Point A");
      System.out.print("First X Coordinate dinu hos : ");
      Xa = sa.nextInt();
      System.out.print("First Y Coordinate dinu hos : ");
      Ya = sa.nextInt();
      System.out.println("\nEnter Coordinate of Point B");
      System.out.print("Second X Coordinate dinu hos : ");
      Xb = sa.nextInt();
      System.out.print("Second Y Coordinate dinu hos : ");
      Yb = sa.nextInt();
      System.out.println("\nEnter Coordinate of Point C");
      System.out.print("Third X Coordinate dinu hos : ");
      Xc = sa.nextInt();
      System.out.print("Third Y Coordinate dinu hos : ");
      Yc = sa.nextInt(); }
```

```
public static void createFrame(String title, JPanel Panel) {
    JFrame BGF = new JFrame(title);
    BGF.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    BGF.setSize(900, 700);
    BGF.setLocationRelativeTo(null);
    BGF.add(Panel);
    BGF.setVisible(true);
}

public static int screenX(int xCoordinateBG, JPanel AshliPanel) {
    return AshliPanel.getWidth() / 2 + xCoordinateBG;
}

public static int screenY(int yCoordinateBG, JPanel AshliPanel) {
    return AshliPanel.getHeight() / 2 - yCoordinateBG;
}

public static void drawAxis(Graphics g, JPanel AshliPanel) {
    int centerXBG = AshliPanel.getWidth() / 2;
    int centerYBG = AshliPanel.getHeight() / 2;

    g.drawLine(0, centerYBG, AshliPanel.getWidth(), centerYBG);
    g.drawLine(centerXBG, 0, centerXBG, AshliPanel.getHeight());

    g.drawString("+X", AshliPanel.getWidth() - 35, centerYBG - 5);
    g.drawString("-X", 10, centerYBG - 5);
    g.drawString("+Y", centerXBG + 5, 20);
    g.drawString("-Y", centerXBG + 5, AshliPanel.getHeight() - 10);
    g.drawString("O(0,0)", centerXBG + 5, centerYBG + 15);
```

```
    }
    public static void BGDrawLine(Graphics g, JPanel AshliPanel, int Xa, int Ya, int Xb,
int Yb) {
        int screenXaBG = screenX(Xa, AshliPanel);
        int screenYaBG = screenY(Ya, AshliPanel);
        int screenXbBG = screenX(Xb, AshliPanel);
        int screenYbBG = screenY(Yb, AshliPanel);
        int dxBG = screenXbBG - screenXaBG;
        int dyBG = screenYbBG - screenYaBG;
        int stepsBG = Math.max(Math.abs(dxBG), Math.abs(dyBG));

        if (stepsBG == 0) {
            g.fillRect(screenXaBG, screenYaBG, 3, 3);
            return;
        }

        float xIncrementBG = (float) dxBG / stepsBG;
        float yIncrementBG = (float) dyBG / stepsBG;
        float currentXBG = screenXaBG;
        float currentYBG = screenYaBG;
        for (int Bikram = 0; Bikram <= stepsBG; Bikram++) {
            g.fillRect(Math.round(currentXBG), Math.round(currentYBG), 2, 2);
            currentXBG = currentXBG + xIncrementBG;
            currentYBG = currentYBG + yIncrementBG;
        }
    }

    public static void drawPointName(Graphics g, JPanel AshliPanel,
            String pointNameBG, int xCoordinateBG, int yCoordinateBG) {
```

```
        int screenXBG = screenX(xCoordinateBG, AshliPanel);
        int screenYBG = screenY(yCoordinateBG, AshliPanel);

        g.drawString( pointNameBG + "(" + xCoordinateBG + "," + yCoordinateBG + ")",
screenXBG + 5, screenYBG - 5);
    }

    public static class CustomPanel extends JPanel {
        private int Xa, Ya;
        private int Xb, Yb;
        private int Xc, Yc;
        private int Xa_Naya, Ya_Naya;
        private int Xb_Naya, Yb_Naya;
        private int Xc_Naya, Yc_Naya;

        public CustomPanel(
                int Xa, int Ya,
                int Xb, int Yb,
                int Xc, int Yc,
                int Xa_Naya, int Ya_Naya,
                int Xb_Naya, int Yb_Naya,
                int Xc_Naya, int Yc_Naya) {

            this.Xa = Xa;
            this.Ya = Ya;
            this.Xb = Xb;
            this.Yb = Yb;
            this.Xc = Xc;
            this.Yc = Yc;
```

```
            this.Xa_Naya = Xa_Naya;
            this.Ya_Naya = Ya_Naya;
            this.Xb_Naya = Xb_Naya;
            this.Yb_Naya = Yb_Naya;
            this.Xc_Naya = Xc_Naya;
            this.Yc_Naya = Yc_Naya;
        }

        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            drawAxis(g, this);
            BGDrawLine(g, this, Xa, Ya, Xb, Yb);
            BGDrawLine(g, this, Xb, Yb, Xc, Yc);
            BGDrawLine(g, this, Xc, Yc, Xa, Ya);
            drawPointName(g, this, "A", Xa, Ya);
            drawPointName(g, this, "B", Xb, Yb);
            drawPointName(g, this, "C", Xc, Yc);

            BGDrawLine(g, this, Xa_Naya, Ya_Naya, Xb_Naya, Yb_Naya);
            BGDrawLine(g, this, Xb_Naya, Yb_Naya, Xc_Naya, Yc_Naya);
            BGDrawLine(g, this, Xc_Naya, Yc_Naya, Xa_Naya, Ya_Naya);
            drawPointName(g, this, "A'", Xa_Naya, Ya_Naya);
            drawPointName(g, this, "B'", Xb_Naya, Yb_Naya);
            drawPointName(g, this, "C'", Xc_Naya, Yc_Naya);
        }
    }
}
```

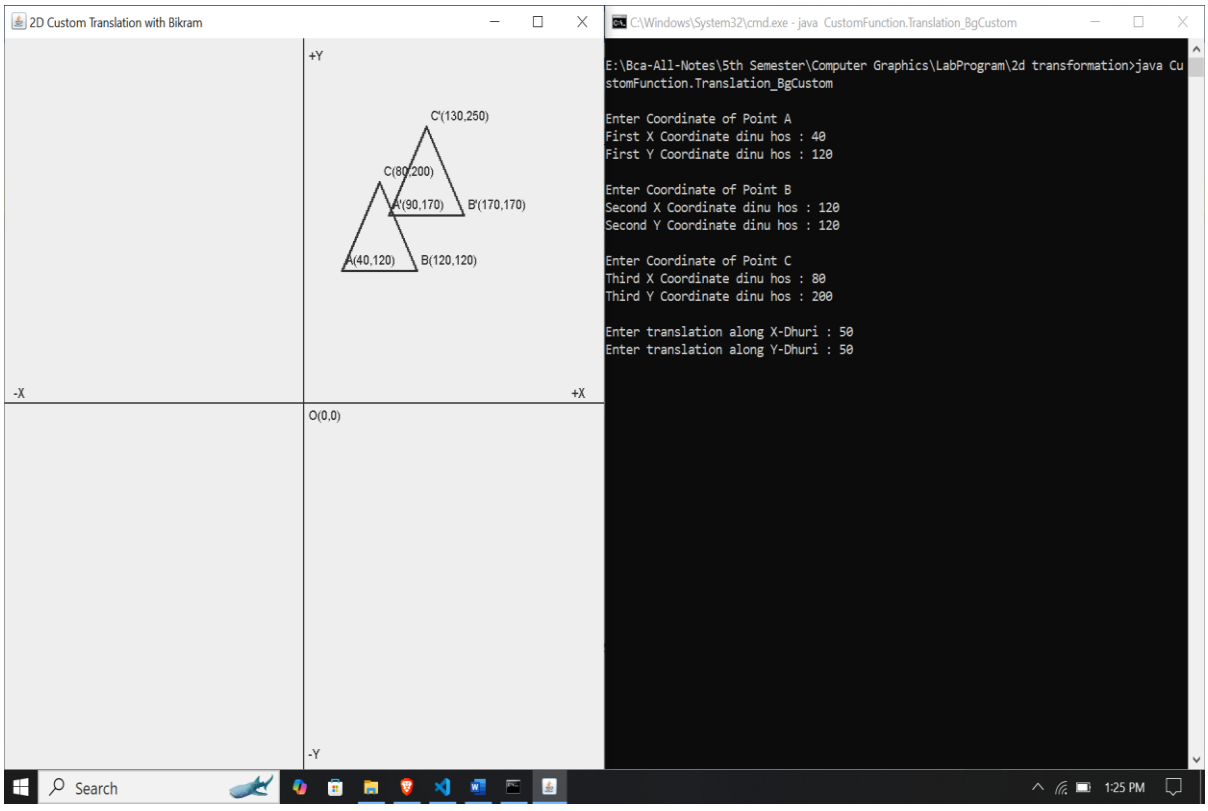
## 2. 2d translation using Custom function.

**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;
public class Translation_BgCustom {
   private int Xa, Ya;
   private int Xb, Yb;
   private int Xc, Yc;
   private int Tx, Ty;
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      Xa = bg.Xa;
      Ya = bg.Ya;
      Xb = bg.Xb;
      Yb = bg.Yb;
      Xc = bg.Xc;
      Yc = bg.Yc;
      System.out.print("\nEnter translation along X-Dhuri : ");
      Tx = sa.nextInt();
      System.out.print("Enter translation along Y-Dhuri : ");
      Ty = sa.nextInt();
      int Xa_Naya = BGTranslationX(Xa, Tx);
      int Ya_Naya = BGTranslationY(Ya, Ty);
      int Xb_Naya = BGTranslationX(Xb, Tx);
      int Yb_Naya = BGTranslationY(Yb, Ty);
      int Xc_Naya = BGTranslationX(Xc, Tx);
      int Yc_Naya = BGTranslationY(Yc, Ty);
```

```
        BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
Xa, Ya, Xb, Yb, Xc, Yc, Xa_Naya, Ya_Naya, Xb_Naya, Yb_Naya, Xc_Naya, Yc_Naya);
        BGhelperPackage.createFrame("2D Custom Translation with Bikram", AshliPanel);
    }
    public int BGTranslationX(int Xa, int Tx) {
        return Xa + Tx;
    }
    public int BGTranslationY(int Ya, int Ty) {
        return Ya + Ty;
    }
    public static void main(String[] args) {
        new Translation_BgCustom().BGHehe();
    }
}
```

**Output:**



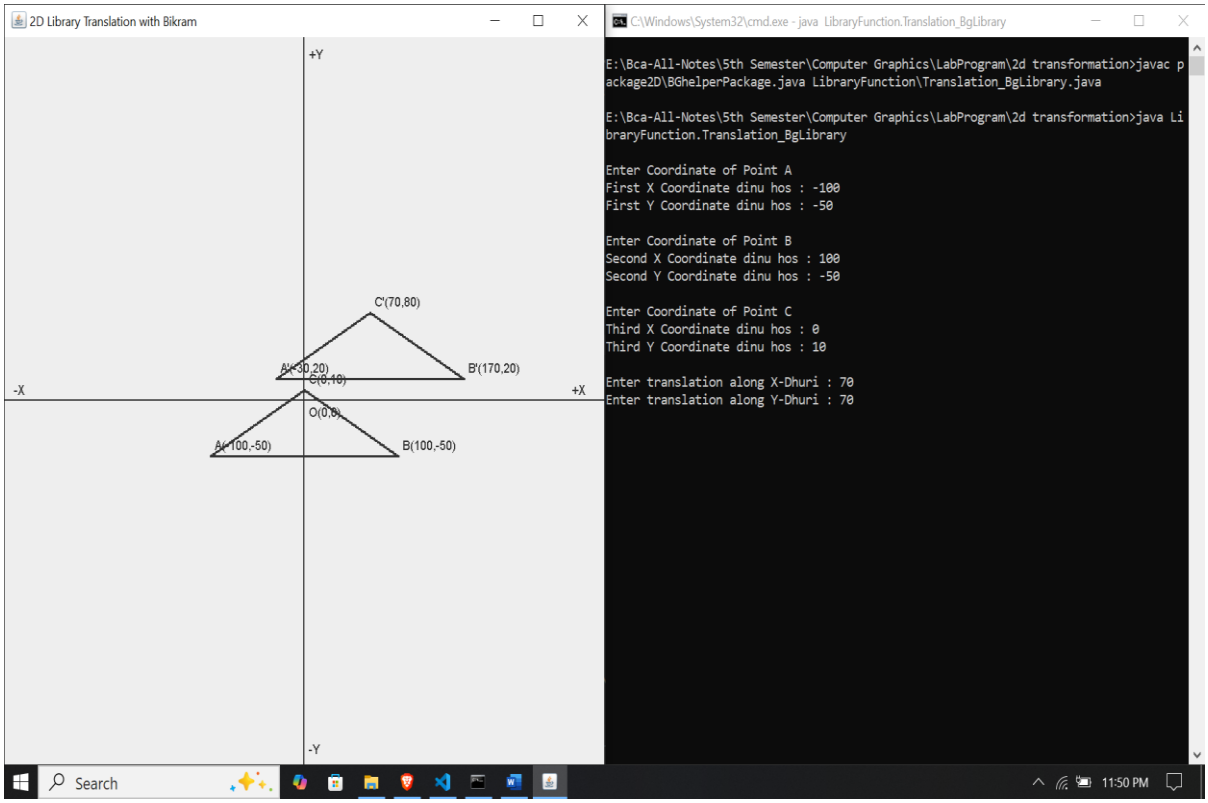
## 3. 2d translation using Library function.

**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;
public class Translation_BgLibrary {
    private int Xa, Ya;
    private int Xb, Yb;
    private int Xc, Yc;
    private int Tx, Ty;
    public void BGHehe() {
        Scanner sa = new Scanner(System.in);
        BGhelperPackage bg = new BGhelperPackage();
        bg.inputCoordinate(sa);
        Xa = bg.Xa;
        Ya = bg.Ya;
        Xb = bg.Xb;
        Yb = bg.Yb;
        Xc = bg.Xc;
        Yc = bg.Yc;
        System.out.print("\nEnter translation along X-Dhuri : ");
        Tx = sa.nextInt();
        System.out.print("Enter translation along Y-Dhuri : ");
        Ty = sa.nextInt();
        AffineTransform BGTransform = new AffineTransform();
        BGTransform.translate(Tx, Ty);
        Point2D A_Naya = BGTransform.transform(new Point2D.Double(Xa, Ya), null);
        Point2D B_Naya = BGTransform.transform(new Point2D.Double(Xb, Yb), null);
```

```
        Point2D C_Naya = BGTransform.transform(new Point2D.Double(Xc, Yc), null);
        BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
                Xa, Ya, Xb, Yb, Xc, Yc,
                (int) Math.round(A_Naya.getX()),
                (int) Math.round(A_Naya.getY()),
                (int) Math.round(B_Naya.getX()),
                (int) Math.round(B_Naya.getY()),
                (int) Math.round(C_Naya.getX()),
                (int) Math.round(C_Naya.getY()));
        BGhelperPackage.createFrame("2D Library Translation with Bikram", AshliPanel);
    }
    public static void main(String[] args) {
        new Translation_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 4. 2d Scaling through origin point using Custom function.

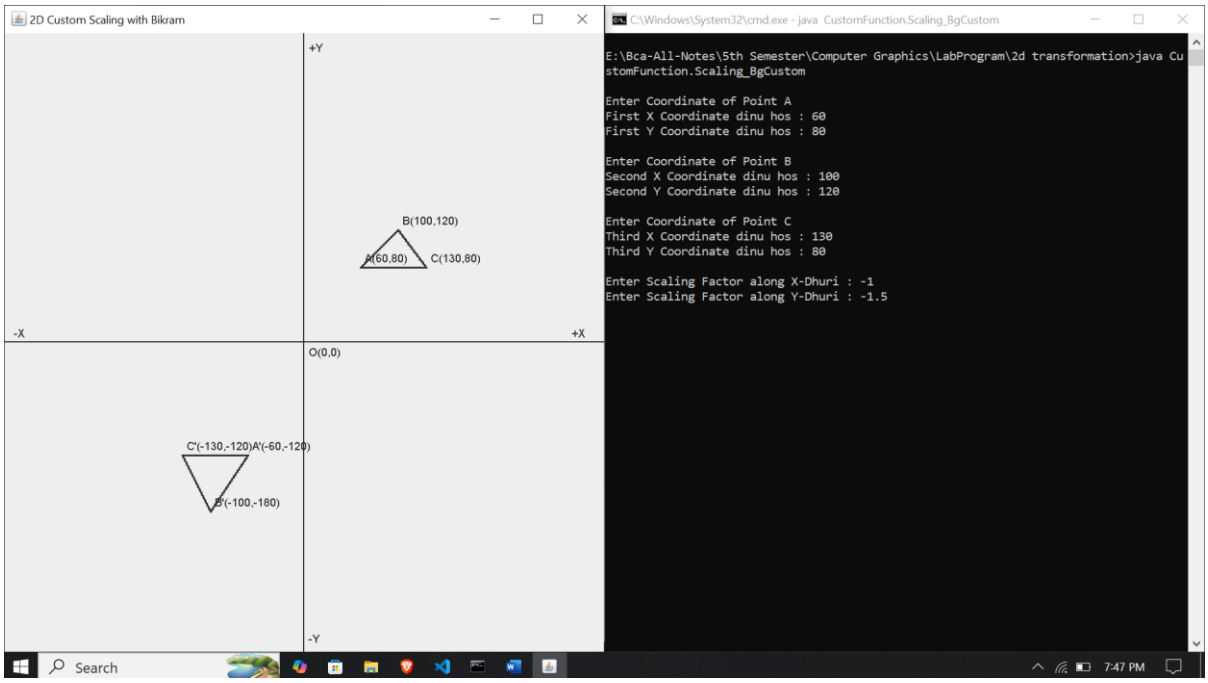
**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;
public class Scaling_BgCustom {
   private int Xa, Ya;
   private int Xb, Yb;
   private int Xc, Yc;
   private double Sx, Sy
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      Xa = bg.Xa;
      Ya = bg.Ya;
      Xb = bg.Xb;
      Yb = bg.Yb;
      Xc = bg.Xc;
      Yc = bg.Yc;
      System.out.print("\nEnter Scaling Factor along X-Dhuri : ");
      Sx = sa.nextDouble();
      System.out.print("Enter Scaling Factor along Y-Dhuri : ");
      Sy = sa.nextDouble();
      int Xa_Naya = BGScalingX(Xa, Sx);
      int Ya_Naya = BGScalingY(Ya, Sy);
      int Xb_Naya = BGScalingX(Xb, Sx);
      int Yb_Naya = BGScalingY(Yb, Sy);
      int Xc_Naya = BGScalingX(Xc, Sx);
      int Yc_Naya = BGScalingY(Yc, Sy);
      BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
```

```
            Xa, Ya, Xb, Yb, Xc, Yc,
            Xa_Naya, Ya_Naya, Xb_Naya, Yb_Naya, Xc_Naya, Yc_Naya);
        BGhelperPackage.createFrame("2D Custom Scaling with Bikram", AshliPanel);
    }
    public int BGScalingX(int Xa, double Sx) {
        return (int) Math.round(Xa * Sx);
    }

    public int BGScalingY(int Ya, double Sy) {
        return (int) Math.round(Ya * Sy);
    }
    public static void main(String[] args) {
        new Scaling_BgCustom().BGHehe();
    }
}
```

**Output:**



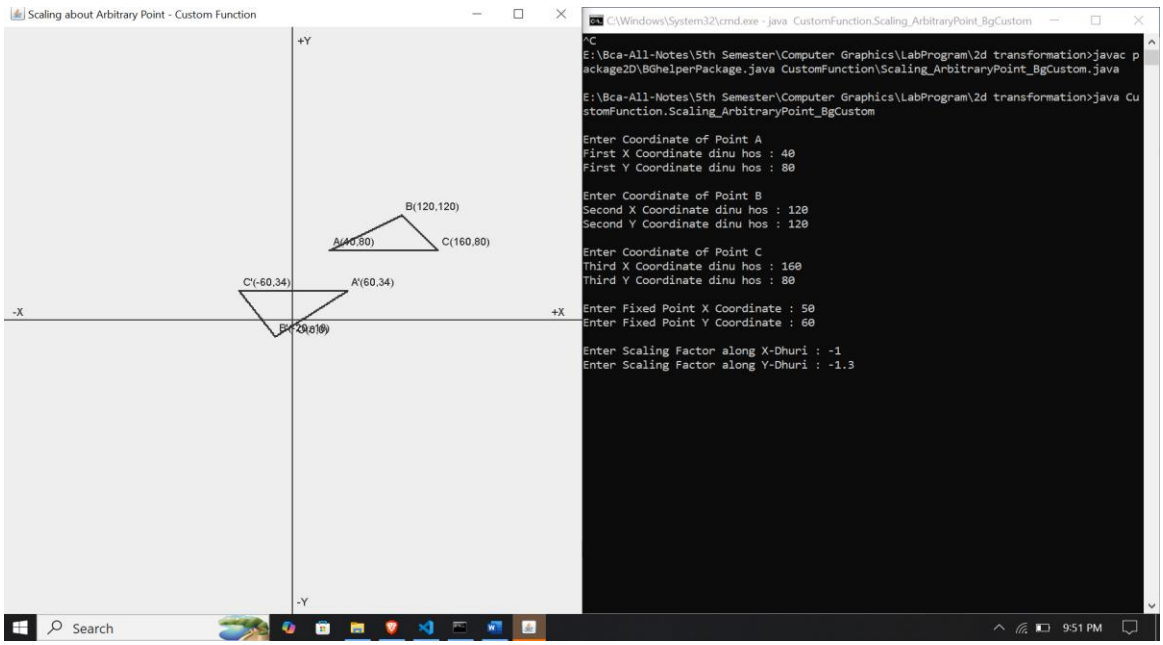
## 5. 2d Scaling through arbitrary point using Custom function

**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;
public class Scaling_ArbitraryPoint_BgCustom {
   private int Xa, Ya, Xb, Yb, Xc, Yc;
   private int Xf, Yf;
   private double Sx, Sy;
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa)
      Xa = bg.Xa;
      Ya = bg.Ya;
      Xb = bg.Xb;
      Yb = bg.Yb;
      Xc = bg.Xc;
      Yc = bg.Yc;
      System.out.print("\nEnter Fixed Point X Coordinate : ");
      Xf = sa.nextInt();
      System.out.print("Enter Fixed Point Y Coordinate : ");
      Yf = sa.nextInt();
      System.out.print("\nEnter Scaling Factor along X-Dhuri : ");
      Sx = sa.nextDouble();
      System.out.print("Enter Scaling Factor along Y-Dhuri : ");
      Sy = sa.nextDouble();
      int Xa_Naya = BGScalingX(Xa, Xf, Sx);
      int Ya_Naya = BGScalingY(Ya, Yf, Sy);
      int Xb_Naya = BGScalingX(Xb, Xf, Sx);
      int Yb_Naya = BGScalingY(Yb, Yf, Sy);
```

```
        int Xc_Naya = BGScalingX(Xc, Xf, Sx);
        int Yc_Naya = BGScalingY(Yc, Yf, Sy);
        BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
                Xa, Ya,Xb, Yb,Xc, Yc,Xa_Naya, Ya_Naya,Xb_Naya, Yb_Naya,Xc_Naya,
Yc_Naya);
        BGhelperPackage.createFrame("Scaling about Arbitrary Point - Custom Function",
AshliPanel);
    }
    public int BGScalingX(int Xa, int Xf, double Sx) {
        return (int) Math.round(Xf + (Xa - Xf) * Sx);
    }
    public int BGScalingY(int Ya, int Yf, double Sy) {
        return (int) Math.round(Yf + (Ya - Yf) * Sy);
    }
    public static void main(String[] args) {
        new Scaling_ArbitraryPoint_BgCustom().BGHehe();
    }
}
```

**Output:**



## 6. 2d Scaling about Origin using Library function.

**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;

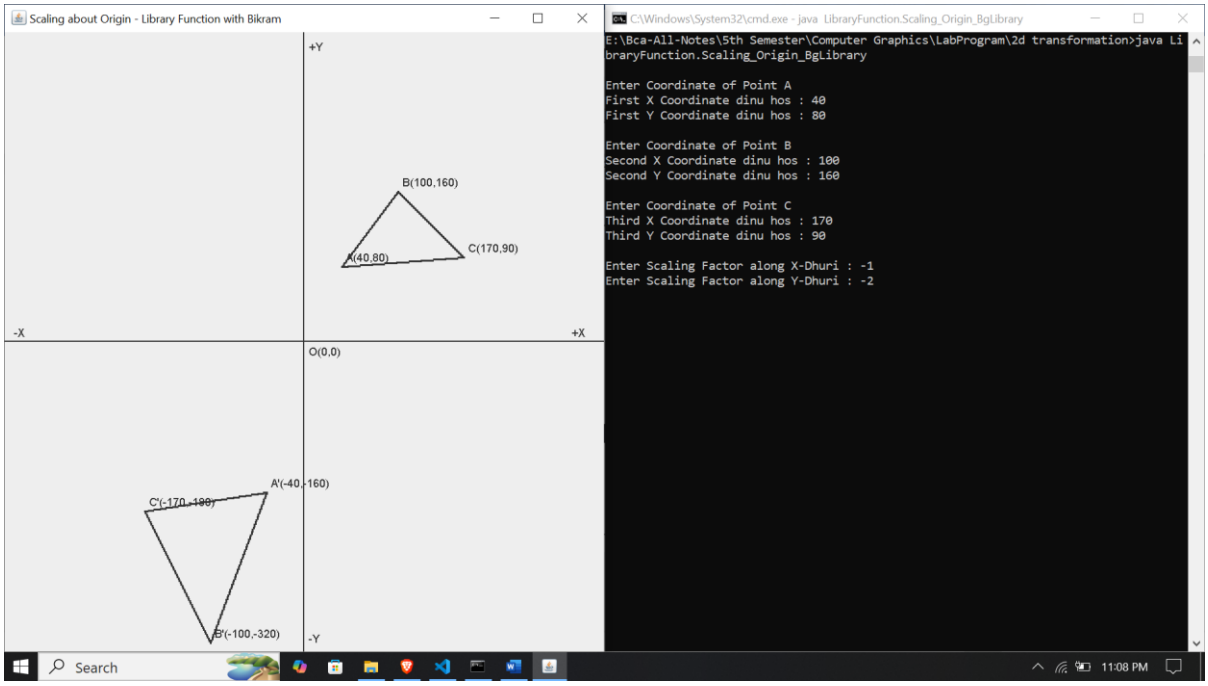
public class Scaling_Origin_BgLibrary {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      System.out.print("\nEnter Scaling Factor along X-Dhuri : ");
      double Sx = sa.nextDouble();

      System.out.print("Enter Scaling Factor along Y-Dhuri : ");
      double Sy = sa.nextDouble();

      AffineTransform BGTransform = AffineTransform.getScaleInstance(Sx, Sy);
      Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya),
null);
      Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);
      Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc),
null);
      BGhelperPackage.CustomPanel AshliPanel = new
BGhelperPackage.CustomPanel(bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            (int) Math.round(A_Naya.getX()),
            (int) Math.round(A_Naya.getY()),
            (int) Math.round(B_Naya.getX()),
```

```
                (int) Math.round(B_Naya.getY()),
                (int) Math.round(C_Naya.getX()),
                (int) Math.round(C_Naya.getY()));
        BGhelperPackage.createFrame("Scaling about Origin - Library Function with Bikram", AshliPanel);
    }
    public static void main(String[] args) {
        new Scaling_Origin_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 7. 2d Scaling through Arbitrary Point using Library function.

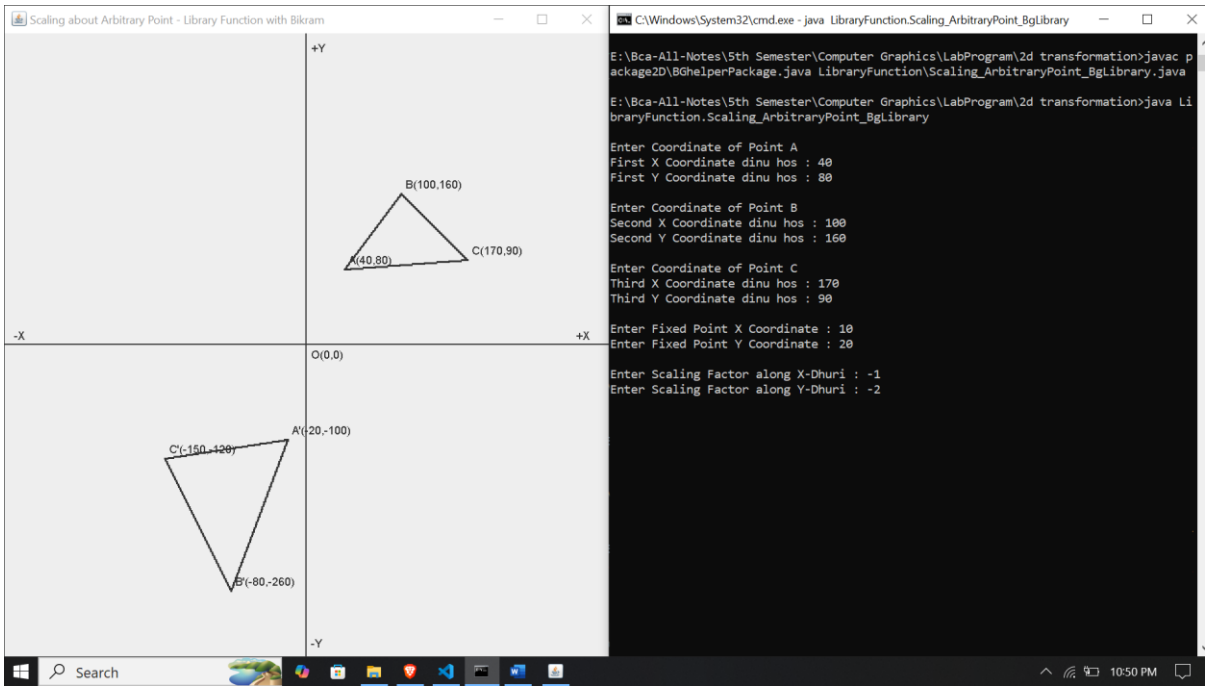
**Code:**

```
 package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;
public class Scaling_ArbitraryPoint_BgLibrary {
    public void BGHehe() {
        Scanner sa = new Scanner(System.in);
        BGhelperPackage bg = new BGhelperPackage();
        bg.inputCoordinate(sa);
        System.out.print("\nEnter Fixed Point X Coordinate : ");
        int Xf = sa.nextInt();
        System.out.print("Enter Fixed Point Y Coordinate : ");
        int Yf = sa.nextInt();
        System.out.print("\nEnter Scaling Factor along X-Dhuri : ");
        double Sx = sa.nextDouble();
        System.out.print("Enter Scaling Factor along Y-Dhuri : ");
        double Sy = sa.nextDouble();
        AffineTransform BGTransform = new AffineTransform();
        BGTransform.translate(Xf, Yf);
        BGTransform.scale(Sx, Sy);
        BGTransform.translate(-Xf, -Yf);
        Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya),
null);
        Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);
        Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc),
null);
        BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
```

```
                bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
                (int) Math.round(A_Naya.getX()),
                (int) Math.round(A_Naya.getY()),
                (int) Math.round(B_Naya.getX()),
                (int) Math.round(B_Naya.getY()),
                (int) Math.round(C_Naya.getX()),
                (int) Math.round(C_Naya.getY()));
        BGhelperPackage.createFrame("Scaling about Arbitrary Point - Library Function
with Bikram", AshliPanel);
    }

    public static void main(String[] args) {
        new Scaling_ArbitraryPoint_BgLibrary().BGHehe();
    }
}
```

**Output:**



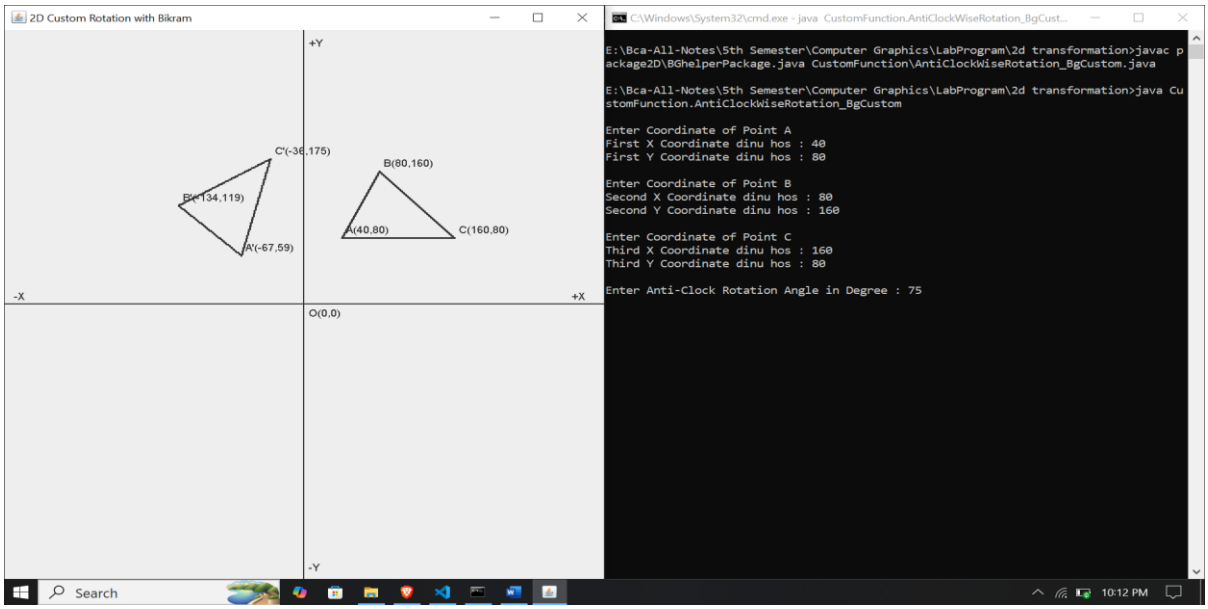
## 8. 2d AntiClock-Wise Rotation using Custom function.

**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;
public class AntiClockWiseRotation_BgCustom {
   private int Xa, Ya;
   private int Xb, Yb;
   private int Xc, Yc;
   private double Angle;
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      Xa = bg.Xa;
      Ya = bg.Ya;
      Xb = bg.Xb;
      Yb = bg.Yb;
      Xc = bg.Xc;
      Yc = bg.Yc;
      System.out.print("\nEnter Anti-Clock Rotation Angle in Degree : ");
      Angle = sa.nextDouble();
      double Radian = Math.toRadians(Angle);
      int Xa_Naya = BGRotationX(Xa, Ya, Radian);
      int Ya_Naya = BGRotationY(Xa, Ya, Radian);
      int Xb_Naya = BGRotationX(Xb, Yb, Radian);
      int Yb_Naya = BGRotationY(Xb, Yb, Radian);
      int Xc_Naya = BGRotationX(Xc, Yc, Radian);
      int Yc_Naya = BGRotationY(Xc, Yc, Radian);
      BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
```

```
                Xa, Ya,
                Xb, Yb,
                Xc, Yc,
                Xa_Naya, Ya_Naya,
                Xb_Naya, Yb_Naya,
                Xc_Naya, Yc_Naya);
        BGhelperPackage.createFrame("2D Custom Rotation with Bikram", AshliPanel);
    }
    public int BGRotationX(int Xa, int Ya, double Radian) {
        return (int) Math.round(Xa * Math.cos(Radian) - Ya * Math.sin(Radian));
    }
    public int BGRotationY(int Xa, int Ya, double Radian) {
        return (int) Math.round(Xa * Math.sin(Radian) + Ya * Math.cos(Radian));
    }
    public static void main(String[] args) {
        new AntiClockWiseRotation_BgCustom().BGHehe();
    }
}
```

**Output:**



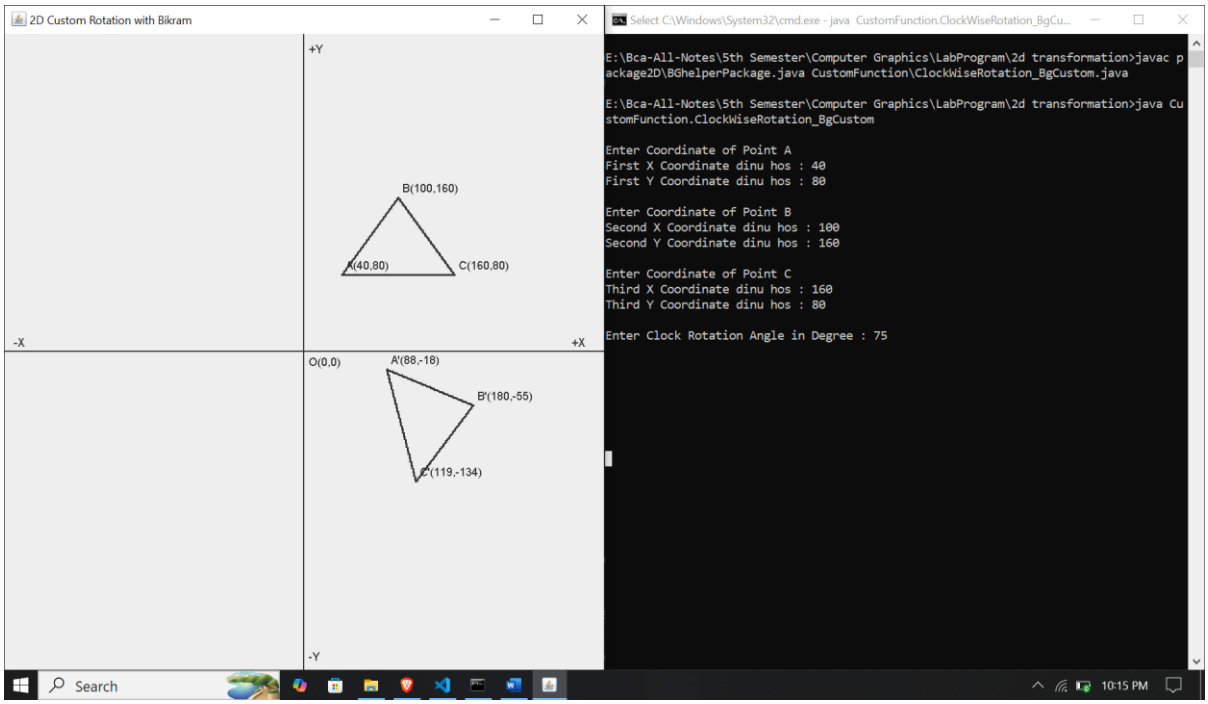
## 9. 2d ClockWise Rotation using Custom function.

**Code:**

```
Package CustomFunction
import package2D.BGhelperPackage;
import java.util.Scanner;
public class ClockWiseRotation_BgCustom {
   private int Xa, Ya;
   private int Xb, Yb;
   private int Xc, Yc;
   private double Angle;
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      Xa = bg.Xa;
      Ya = bg.Ya;
      Xb = bg.Xb;
      Yb = bg.Yb;
      Xc = bg.Xc;
      Yc = bg.Yc;
      System.out.print("\nEnter Clock Rotation Angle in Degree : ");
      Angle = sa.nextDouble();
      double Radian = Math.toRadians(Angle);
      int Xa_Naya = BGRotationX(Xa, Ya, Radian);
      int Ya_Naya = BGRotationY(Xa, Ya, Radian);
      int Xb_Naya = BGRotationX(Xb, Yb, Radian);
      int Yb_Naya = BGRotationY(Xb, Yb, Radian);
      int Xc_Naya = BGRotationX(Xc, Yc, Radian);
      int Yc_Naya = BGRotationY(Xc, Yc, Radian);
      BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
```

```
            Xa, Ya,  Xb, Yb,Xc, Yc,
            Xa_Naya, Ya_Naya,
            Xb_Naya, Yb_Naya,
            Xc_Naya, Yc_Naya);
        BGhelperPackage.createFrame("2D Custom Rotation with Bikram", AshliPanel);
    }
    public int BGRotationX(int Xa, int Ya, double Radian) {
        return (int) Math.round(Xa * Math.cos(Radian) + Ya * Math.sin(Radian));
    }
    public int BGRotationY(int Xa, int Ya, double Radian) {
        return (int) Math.round(-Xa * Math.sin(Radian) + Ya * Math.cos(Radian));
    }
    public static void main(String[] args) {
        new ClockWiseRotation_BgCustom().BGHehe();
    }
}
```

**Output:**



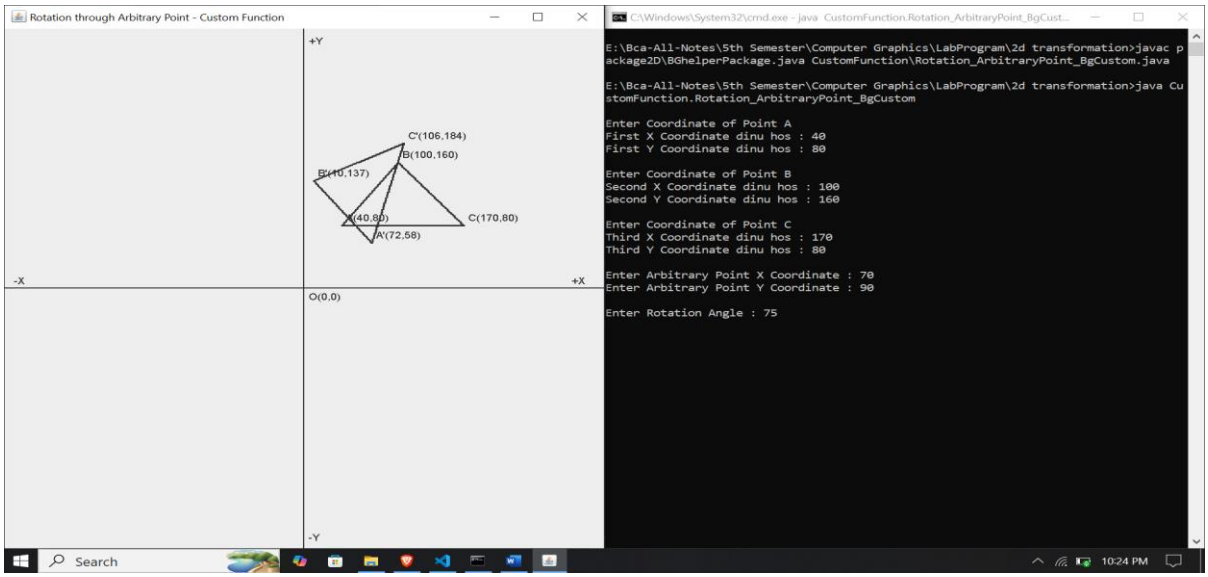
## 10.2d Rotation through Arbitrary Point using Custom function.

**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;
public class Rotation_ArbitraryPoint_BgCustom {
   private int Xa, Ya, Xb, Yb, Xc, Yc;
   private int Xp, Yp;
   private double Angle;
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      Xa = bg.Xa;
      Ya = bg.Ya;
      Xb = bg.Xb;
      Yb = bg.Yb;
      Xc = bg.Xc;
      Yc = bg.Yc;
      System.out.print("\nEnter Arbitrary Point X Coordinate : ");
      Xp = sa.nextInt();
      System.out.print("Enter Arbitrary Point Y Coordinate : ");
      Yp = sa.nextInt();
      System.out.print("\nEnter Rotation Angle : ");
      Angle = sa.nextDouble();
      double Radian = Math.toRadians(Angle);
      int Xa_Naya = BGRotationX(Xa, Ya, Xp, Yp, Radian);
      int Ya_Naya = BGRotationY(Xa, Ya, Xp, Yp, Radian);
      int Xb_Naya = BGRotationX(Xb, Yb, Xp, Yp, Radian);
      int Yb_Naya = BGRotationY(Xb, Yb, Xp, Yp, Radian);
```

```
        int Xc_Naya = BGRotationX(Xc, Yc, Xp, Yp, Radian);
        int Yc_Naya = BGRotationY(Xc, Yc, Xp, Yp, Radian);
        BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
  Xa, Ya,Xb, Yb,Xc, Yc,Xa_Naya, Ya_Naya,Xb_Naya, Yb_Naya,  Xc_Naya, Yc_Naya);
        BGhelperPackage.createFrame("Rotation through Arbitrary Point - Custom
Function", AshliPanel);
    }
    public int BGRotationX(int Xa, int Ya, int Xp, int Yp, double Radian) {
        return (int) Math.round(Xp + (Xa - Xp) * Math.cos(Radian) - (Ya - Yp) *
Math.sin(Radian));
    }
    public int BGRotationY(int Xa, int Ya, int Xp, int Yp, double Radian) {
        return (int) Math.round(Yp + (Xa - Xp) * Math.sin(Radian) + (Ya - Yp) *
Math.cos(Radian));
    }
    public static void main(String[] args) {
        new Rotation_ArbitraryPoint_BgCustom().BGHehe();
    }
}
```

**Output:**



## 11.2d Anticlockwise Rotation using Library function

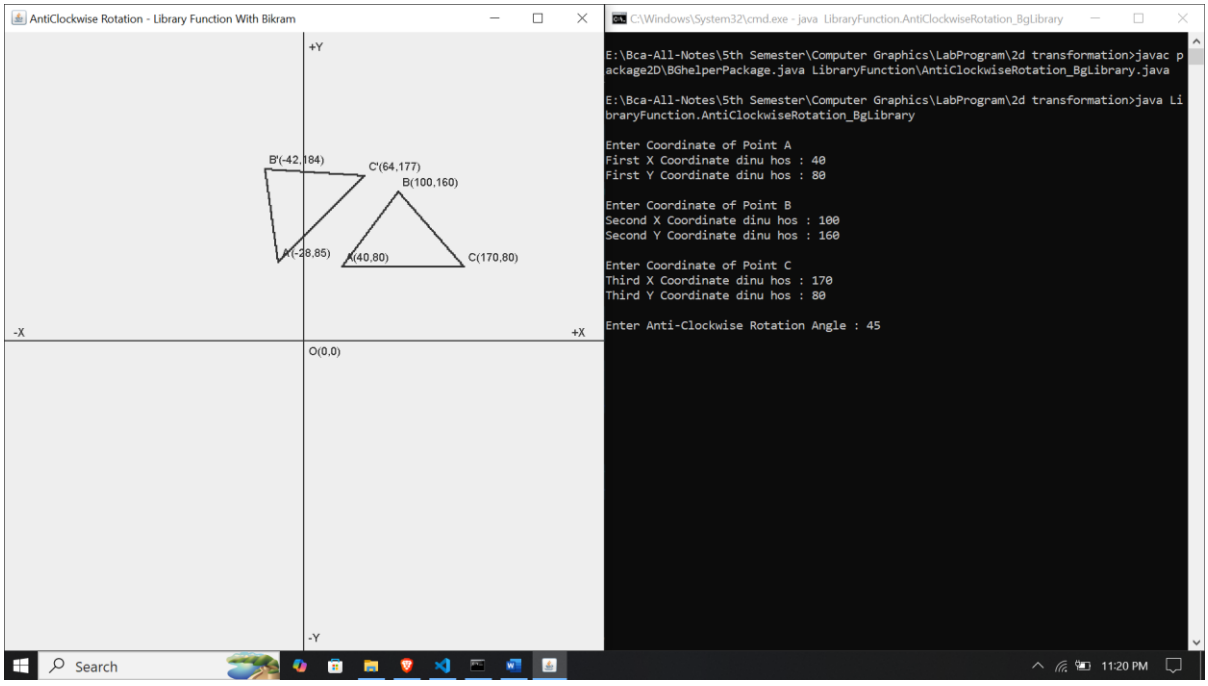
**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;
public class AntiClockwiseRotation_BgLibrary {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      System.out.print("\nEnter Anti-Clockwise Rotation Angle : ");
      double Angle = sa.nextDouble();

      double Radian = Math.toRadians(Angle);
      AffineTransform BGTransform = AffineTransform.getRotateInstance(Radian);
      Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya),
null);
      Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);
      Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc),
null);
      BGhelperPackage.CustomPanel AshliPanel = new
BGhelperPackage.CustomPanel(bg.Xa, bg.Ya,bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            (int) Math.round(A_Naya.getX()),
            (int) Math.round(A_Naya.getY()),
            (int) Math.round(B_Naya.getX()),
            (int) Math.round(B_Naya.getY()),
            (int) Math.round(C_Naya.getX()),
```

```
            (int) Math.round(C_Naya.getY()));
        BGhelperPackage.createFrame("AntiClockwise Rotation - Library Function With
Bikram ", AshliPanel);
    }
    public static void main(String[] args) {
        new AntiClockwiseRotation_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 12. 2d Clockwise Rotation using Library function

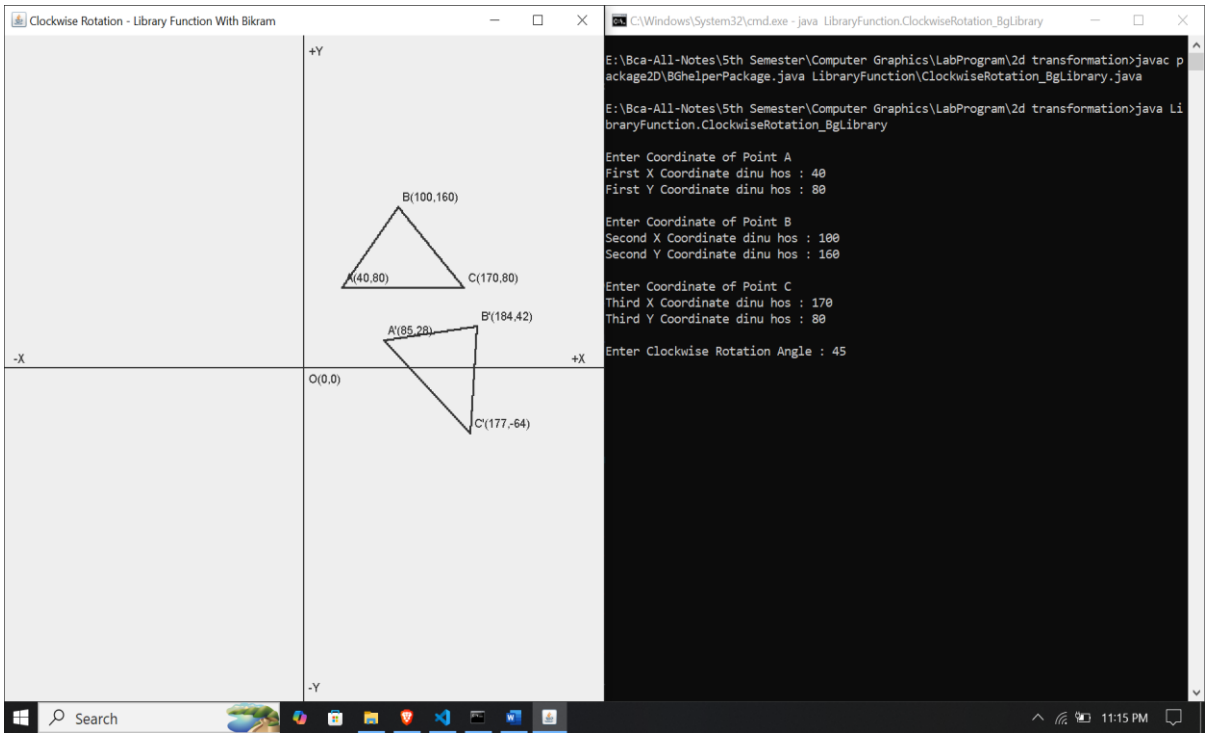
**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;
public class ClockwiseRotation_BgLibrary {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      System.out.print("\nEnter Clockwise Rotation Angle : ");
      double Angle = sa.nextDouble();
      double Radian = Math.toRadians(-Angle);
      AffineTransform BGTransform = AffineTransform.getRotateInstance(Radian);
      Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya),
null);
      Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);
      Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc),
null);
      BGhelperPackage.CustomPanel AshliPanel = new
BGhelperPackage.CustomPanel(bg.Xa, bg.Ya,bg.Xb, bg.Yb,bg.Xc, bg.Yc,
            (int) Math.round(A_Naya.getX()),
            (int) Math.round(A_Naya.getY()),

            (int) Math.round(B_Naya.getX()),
            (int) Math.round(B_Naya.getY()),
            (int) Math.round(C_Naya.getX()),
```

```
            (int) Math.round(C_Naya.getY()));
        BGhelperPackage.createFrame("Clockwise Rotation - Library Function With
Bikram ", AshliPanel);
    }
    public static void main(String[] args) {
        new ClockwiseRotation_BgLibrary().BGHehe();
    }
}
```

**Output:**



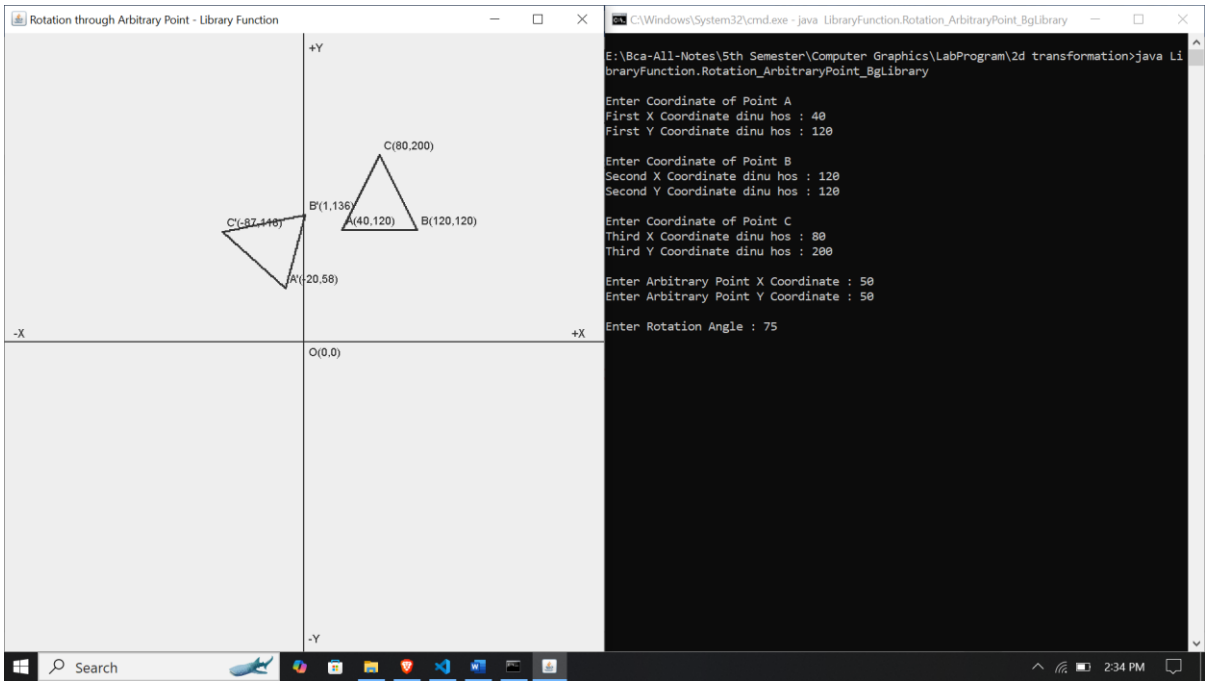
## 13.2d Rotation through arbitrary point using Library function.

**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;
public class Rotation_ArbitraryPoint_BgLibrary {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      System.out.print("\nEnter Arbitrary Point X Coordinate : ");
      int Xp = sa.nextInt();
      System.out.print("Enter Arbitrary Point Y Coordinate : ");
      int Yp = sa.nextInt();
      System.out.print("\nEnter Rotation Angle : ");
      double Angle = sa.nextDouble();
      double Radian = Math.toRadians(Angle);
      AffineTransform BGTransform = AffineTransform.getRotateInstance(Radian, Xp,
Yp);
      Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya),
null);
      Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);
      Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc),
null);
      BGhelperPackage.CustomPanel AshliPanel = new
BGhelperPackage.CustomPanel(bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            (int) Math.round(A_Naya.getX()),
            (int) Math.round(A_Naya.getY()),
```

```
                (int) Math.round(B_Naya.getX()),
                (int) Math.round(B_Naya.getY()),
                (int) Math.round(C_Naya.getX()),
                (int) Math.round(C_Naya.getY()));
        BGhelperPackage.createFrame("Rotation through Arbitrary Point - Library
Function", AshliPanel);
    }
    public static void main(String[] args) {
        new Rotation_ArbitraryPoint_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 14.2d Reflection along X-Axis using Custom function.

**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;
public class Reflection_XAxis_BgCustom {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      int Xa_Naya = BGReflectionX(bg.Xa);
      int Ya_Naya = BGReflectionY(bg.Ya);
      int Xb_Naya = BGReflectionX(bg.Xb);
      int Yb_Naya = BGReflectionY(bg.Yb);

      int Xc_Naya = BGReflectionX(bg.Xc);
      int Yc_Naya = BGReflectionY(bg.Yc);

      BGhelperPackage.CustomPanel AshliPanel = new
BGhelperPackage.CustomPanel(bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc,
            bg.Yc,
            Xa_Naya, Ya_Naya,
            Xb_Naya, Yb_Naya,
            Xc_Naya, Yc_Naya);
      BGhelperPackage.createFrame("Reflection Along X-Axis - Custom Function",
AshliPanel);
   }

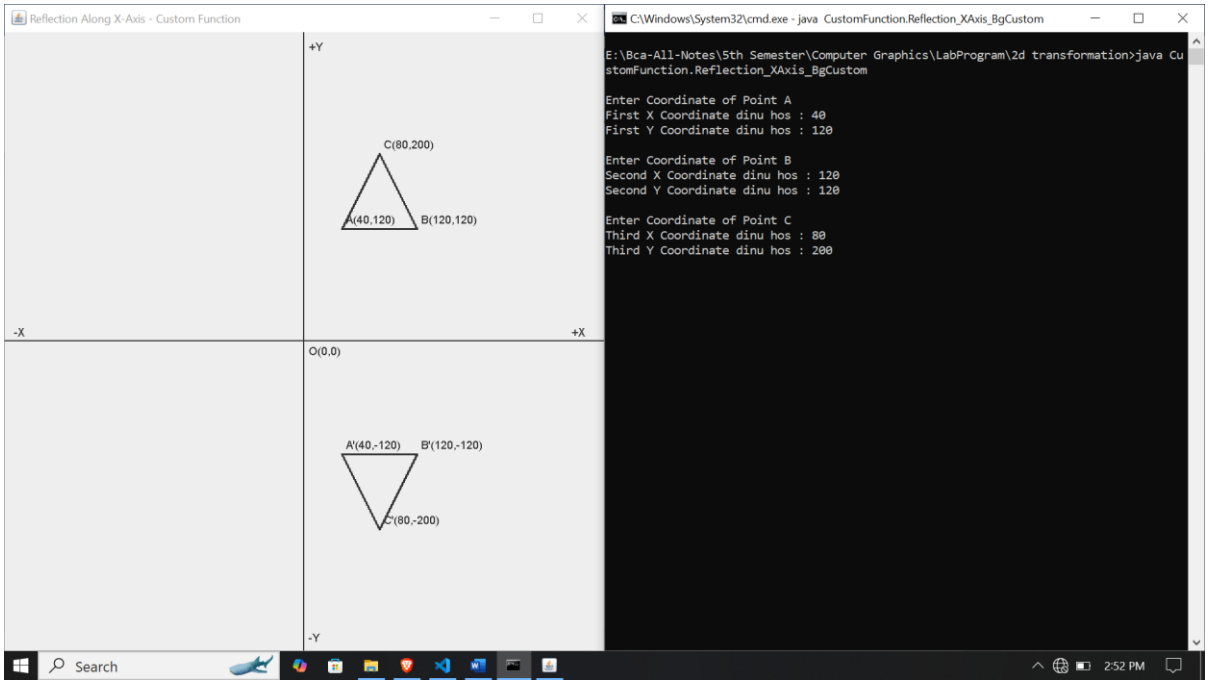
   public int BGReflectionX(int Xa) {
```

```
        return Xa;
    }

    public int BGReflectionY(int Ya) {
        return -Ya;
    }

    public static void main(String[] args) {
        new Reflection_XAxis_BgCustom().BGHehe();
    }
}
```

**Output:**



## 15. 2d Reflection along X-Axis using Custom function.

**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;

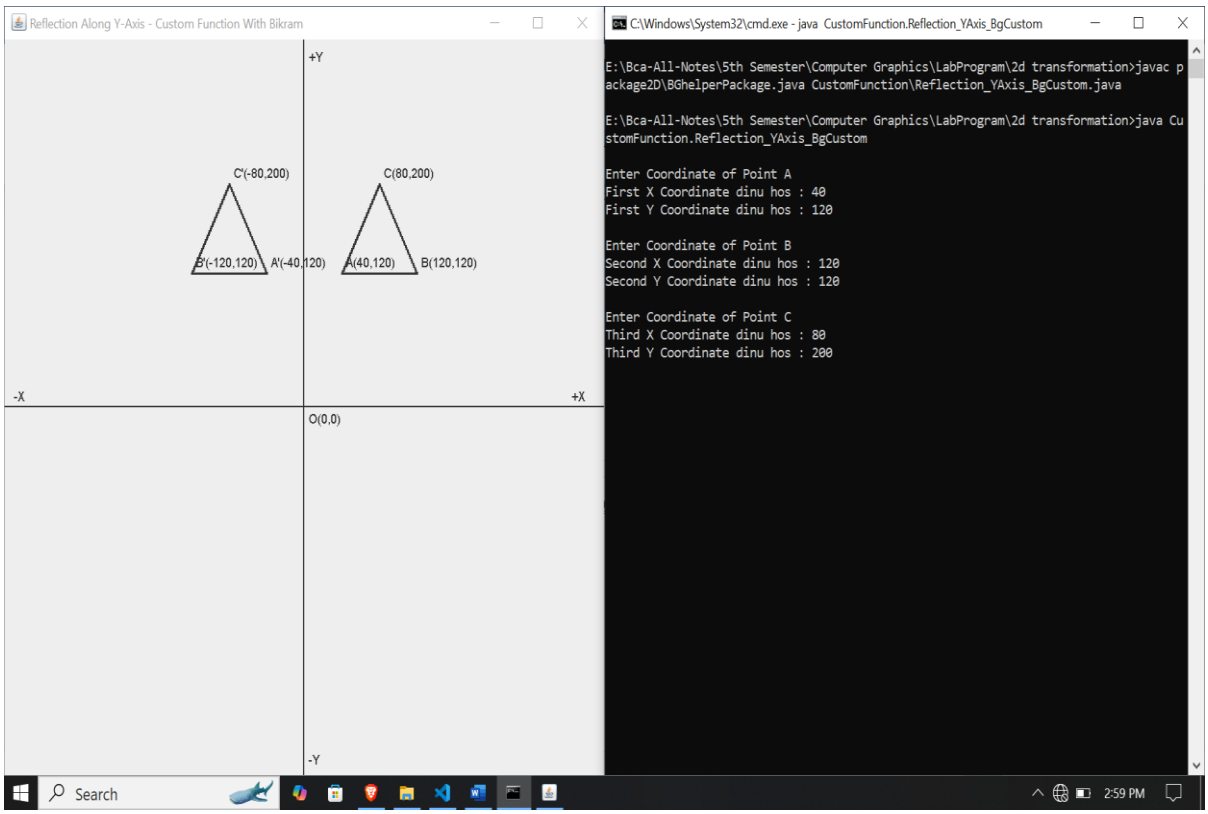
public class Reflection_YAxis_BgCustom {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);

      int Xa_Naya = BGReflectionX(bg.Xa);
      int Ya_Naya = BGReflectionY(bg.Ya);
      int Xb_Naya = BGReflectionX(bg.Xb);
      int Yb_Naya = BGReflectionY(bg.Yb);
      int Xc_Naya = BGReflectionX(bg.Xc);
      int Yc_Naya = BGReflectionY(bg.Yc);

      BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
            bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            Xa_Naya, Ya_Naya, Xb_Naya, Yb_Naya, Xc_Naya, Yc_Naya);
      BGhelperPackage.createFrame("Reflection Along Y-Axis - Custom Function With
Bikram", AshliPanel);
   }
   public int BGReflectionX(int Xa) {
      return -Xa;
   }
   public int BGReflectionY(int Ya) {
```

```
        return Ya;
    }
    public static void main(String[] args) {
        new Reflection_YAxis_BgCustom().BGHehe();
    }
}
```

**Output:**



## 16. 2d Reflection along X-Axis using Custom function.

**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;

public class Reflection_Origin_BgCustom {
    public void BGHehe() {
        Scanner sa = new Scanner(System.in);
        BGhelperPackage bg = new BGhelperPackage();
        bg.inputCoordinate(sa);

        int Xa_Naya = BGReflectionX(bg.Xa);
        int Ya_Naya = BGReflectionY(bg.Ya);
        int Xb_Naya = BGReflectionX(bg.Xb);
        int Yb_Naya = BGReflectionY(bg.Yb);
        int Xc_Naya = BGReflectionX(bg.Xc);
        int Yc_Naya = BGReflectionY(bg.Yc);

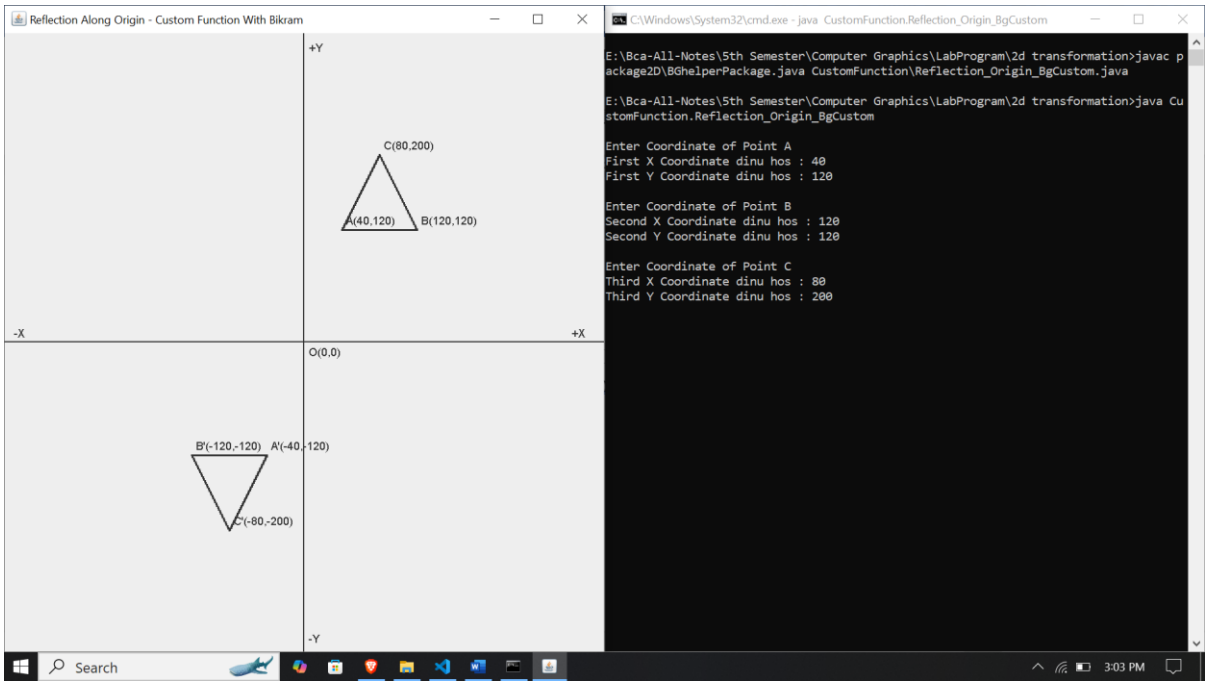
        BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
                bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
                Xa_Naya, Ya_Naya, Xb_Naya, Yb_Naya, Xc_Naya, Yc_Naya);
        BGhelperPackage.createFrame("Reflection Along Origin - Custom Function With
Bikram", AshliPanel);
    }

    public int BGReflectionX(int Xa) {
        return -Xa;
    }
```

```
    public int BGReflectionY(int Ya) {
        return -Ya;
    }

    public static void main(String[] args) {
        new Reflection_Origin_BgCustom().BGHehe();
    }
}
```

**Output:**



## 17. 2d Reflection along Y=X using Custom function.

**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;
public class Reflection_YequalsX_BgCustom {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);

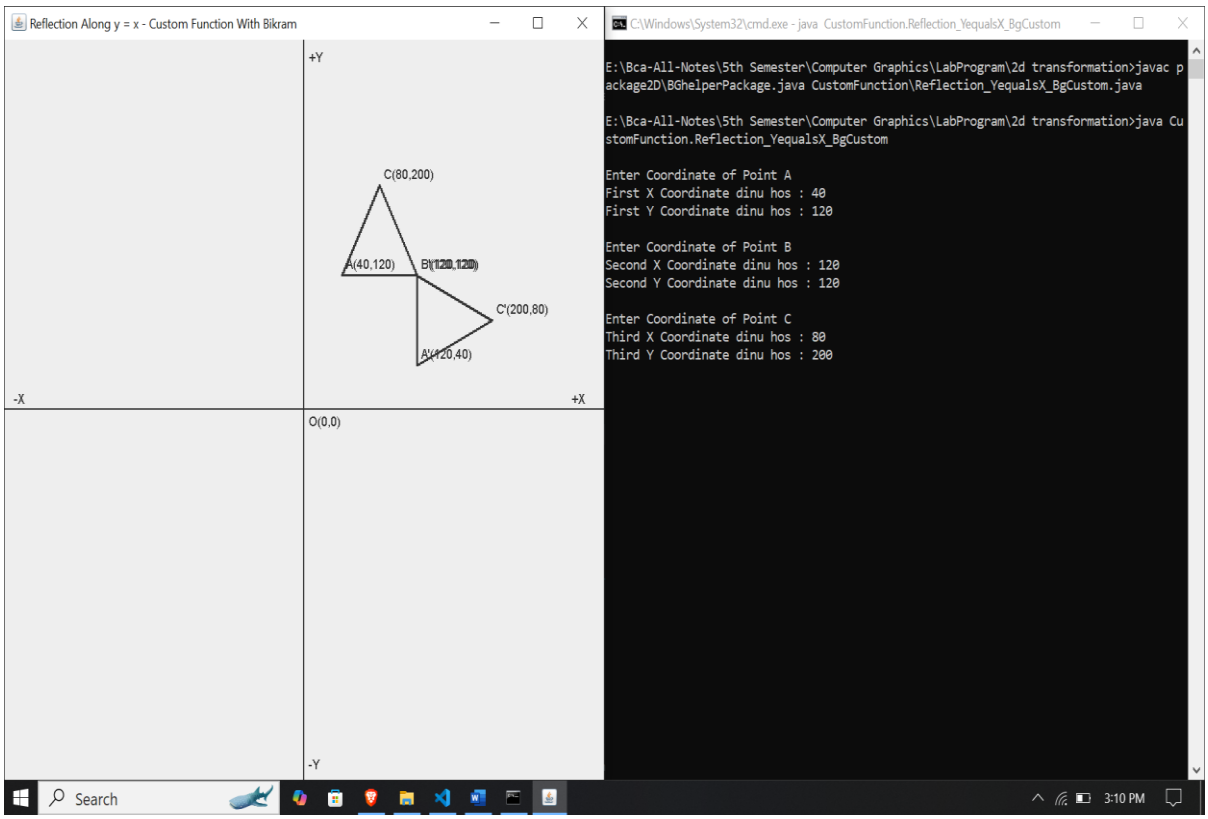
      int Xa_Naya = BGReflectionX(bg.Ya);
      int Ya_Naya = BGReflectionY(bg.Xa);
      int Xb_Naya = BGReflectionX(bg.Yb);
      int Yb_Naya = BGReflectionY(bg.Xb);
      int Xc_Naya = BGReflectionX(bg.Yc);
      int Yc_Naya = BGReflectionY(bg.Xc);

      BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
            bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            Xa_Naya, Ya_Naya, Xb_Naya, Yb_Naya, Xc_Naya, Yc_Naya);
      BGhelperPackage.createFrame("Reflection Along y = x - Custom Function With
Bikram", AshliPanel);
   }

   public int BGReflectionX(int Ya) {
      return Ya;
   }
   public int BGReflectionY(int Xa) {
```

```
        return Xa;
    }
    public static void main(String[] args) {
        new Reflection_YequalsX_BgCustom().BGHehe();
    }
}
```

**Output:**



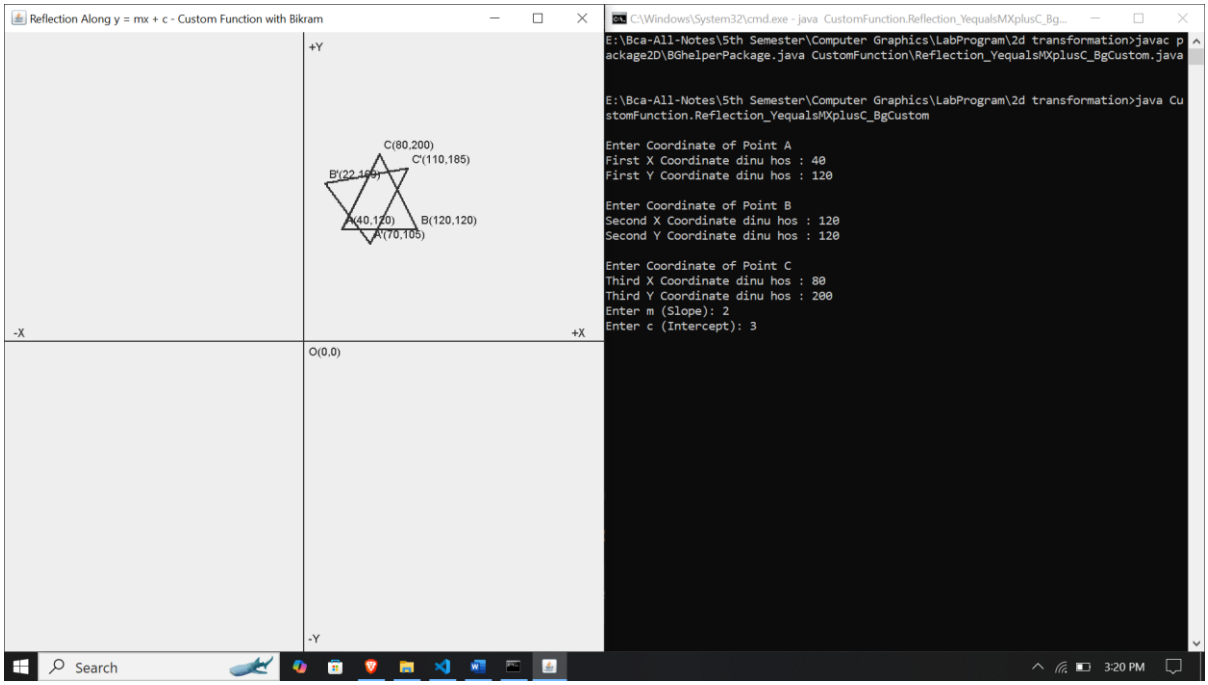
## 18.2d Reflection along Y=mx+c using Custom function.

**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;
public class Reflection_YequalsMXplusC_BgCustom {
    private double m;
    private double c;
    public void BGHehe() {
        Scanner sa = new Scanner(System.in);
        BGhelperPackage bg = new BGhelperPackage();
        bg.inputCoordinate(sa);
        System.out.print("Enter m (Slope): ");
        m = sa.nextDouble();
        System.out.print("Enter c (Intercept): ");
        c = sa.nextDouble();
        int Xa_Naya = BGReflectionX(bg.Xa, bg.Ya);
        int Ya_Naya = BGReflectionY(bg.Xa, bg.Ya);
        int Xb_Naya = BGReflectionX(bg.Xb, bg.Yb);
        int Yb_Naya = BGReflectionY(bg.Xb, bg.Yb);
        int Xc_Naya = BGReflectionX(bg.Xc, bg.Yc);
        int Yc_Naya = BGReflectionY(bg.Xc, bg.Yc);
        BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
                bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
                Xa_Naya, Ya_Naya, Xb_Naya, Yb_Naya, Xc_Naya, Yc_Naya);
        BGhelperPackage.createFrame("Reflection Along y = mx + c - Custom Function
with Bikram", AshliPanel);
    }
    public int BGReflectionX(int Xa, int Ya) {
```

```
        double BGDistance = (m * Xa - Ya + c) / (m * m + 1);
        return (int) Math.round(Xa - 2 * m * BGDistance);
    }
    public int BGReflectionY(int Xa, int Ya) {
        double BGDistance = (m * Xa - Ya + c) / (m * m + 1);
        return (int) Math.round(Ya + 2 * BGDistance);
    }
    public static void main(String[] args) {
        new Reflection_YequalsMXplusC_BgCustom().BGHehe();
    }
}
```

**Output:**



## 19.2d Reflection along X Axis using Library function

**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;

public class Reflection_XAxis_BgLibrary {
public void BGHehe() {
Scanner sa = new Scanner(System.in);
BGhelperPackage bg = new BGhelperPackage();
bg.inputCoordinate(sa);
   AffineTransform BGTransform = AffineTransform.getScaleInstance(1, -1);

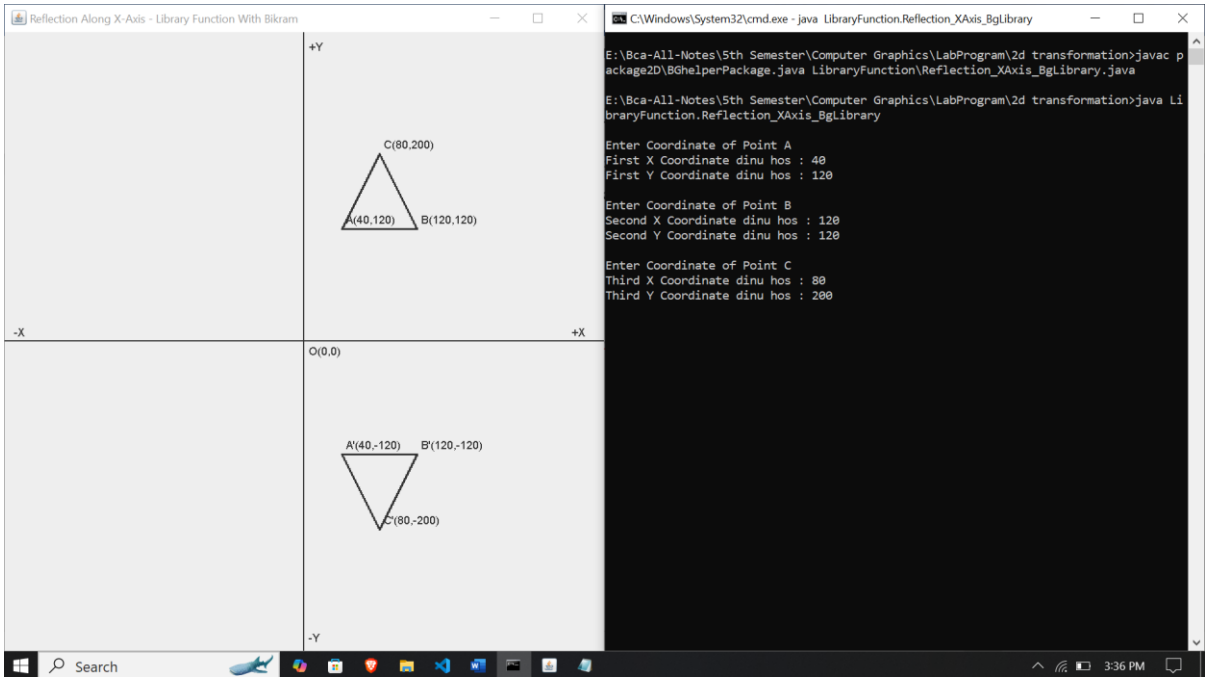
   Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya), null);
   Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);
   Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc), null);

   BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
      bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
      (int)Math.round(A_Naya.getX()), (int)Math.round(A_Naya.getY()),
      (int)Math.round(B_Naya.getX()), (int)Math.round(B_Naya.getY()),
      (int)Math.round(C_Naya.getX()), (int)Math.round(C_Naya.getY())
   );
```

```
    BGhelperPackage.createFrame("Reflection Along X-Axis - Library Function With
Bikram", AshliPanel);
}

public static void main(String[] args) {
    new Reflection_XAxis_BgLibrary().BGHehe();
}
}
```

**Output:**



## 20.2d Reflection along Y Axis using Library function

**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;

public class Reflection_YAxis_BgLibrary {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);

      AffineTransform BGTransform = AffineTransform.getScaleInstance(-1, 1);

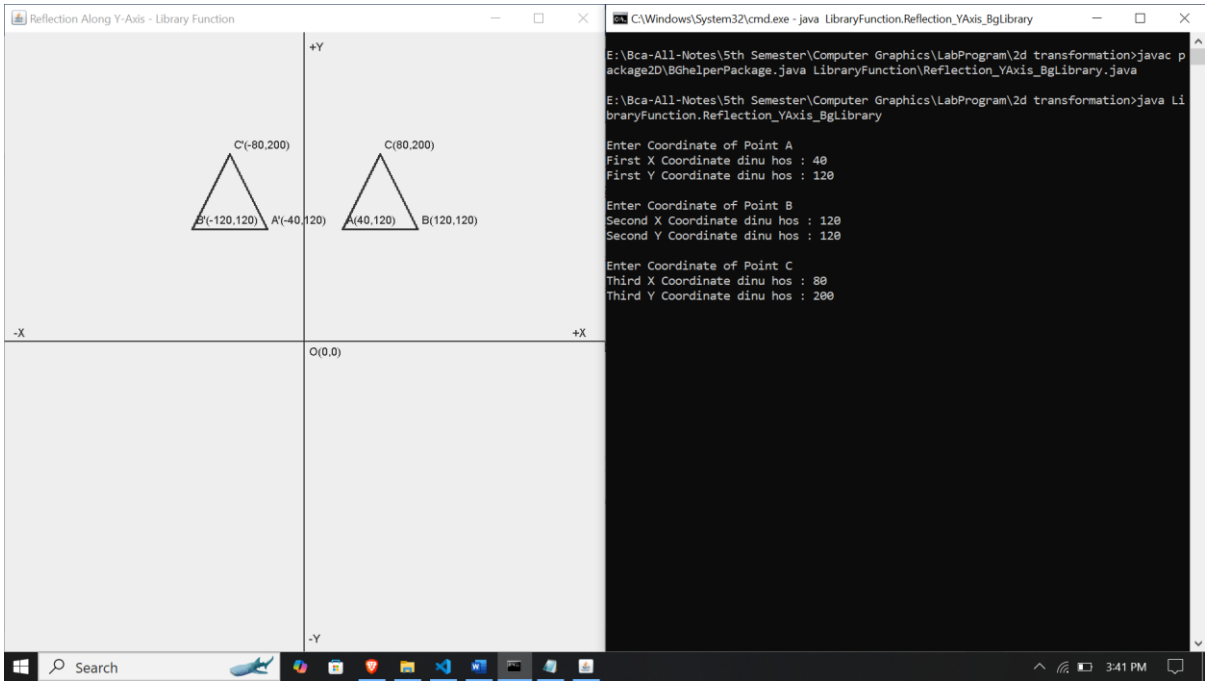
      Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya),
null);
      Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);
      Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc),
null);

      BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
            bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            (int) Math.round(A_Naya.getX()), (int) Math.round(A_Naya.getY()),
            (int) Math.round(B_Naya.getX()), (int) Math.round(B_Naya.getY()),
            (int) Math.round(C_Naya.getX()), (int) Math.round(C_Naya.getY()));
```

```
        BGhelperPackage.createFrame("Reflection Along Y-Axis - Library Function",
AshliPanel);
    }

    public static void main(String[] args) {
        new Reflection_YAxis_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 21.2d Reflection along Origin using Library function.

**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;

public class Reflection_Origin_BgLibrary {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);

      AffineTransform BGTransform = AffineTransform.getScaleInstance(-1, -1);

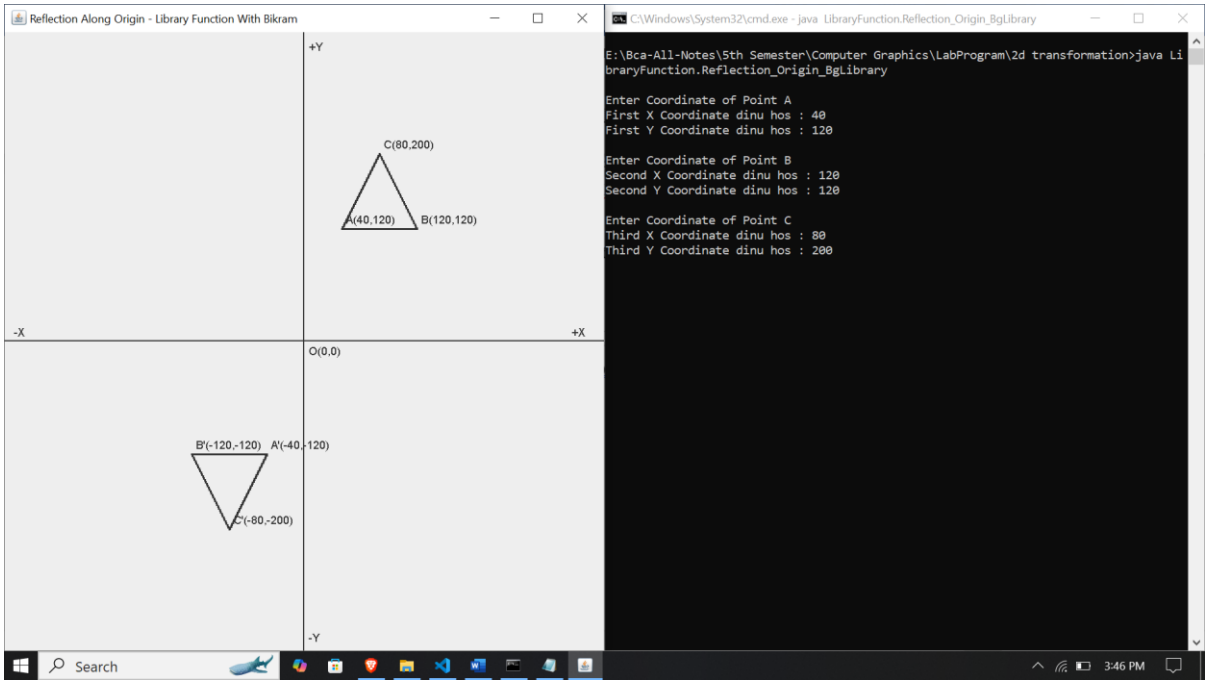
      Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya),
null);
      Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);
      Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc),
null);

      BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
            bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            (int) Math.round(A_Naya.getX()), (int) Math.round(A_Naya.getY()),
            (int) Math.round(B_Naya.getX()), (int) Math.round(B_Naya.getY()),
            (int) Math.round(C_Naya.getX()), (int) Math.round(C_Naya.getY()));
```

```
        BGhelperPackage.createFrame("Reflection Along Origin - Library Function With
Bikram", AshliPanel);
    }

    public static void main(String[] args) {
        new Reflection_Origin_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 22.2d Reflection along Y=X using Library function.

**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;

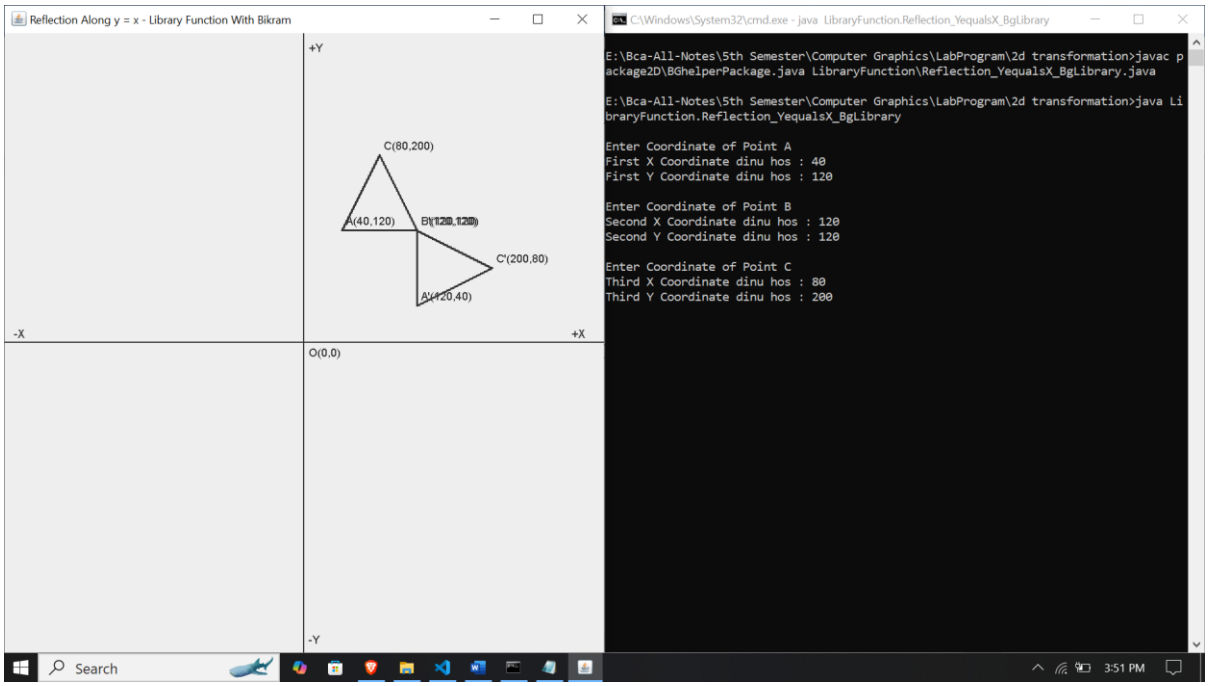
public class Reflection_YequalsX_BgLibrary {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      AffineTransform BGTransform = new AffineTransform(0, 1, 1, 0, 0, 0);

      Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya),
null);
      Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);
      Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc),
null);

      BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
            bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            (int) Math.round(A_Naya.getX()), (int) Math.round(A_Naya.getY()),
            (int) Math.round(B_Naya.getX()), (int) Math.round(B_Naya.getY()),
            (int) Math.round(C_Naya.getX()), (int) Math.round(C_Naya.getY()));
      BGhelperPackage.createFrame("Reflection Along y = x - Library Function With
Bikram", AshliPanel);
   }
```

```
    public static void main(String[] args) {
        new Reflection_YequalsX_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 23. 2d Reflection along Y=mx+c using Library function.

**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;
public class Reflection_YequalsMXplusC_BgLibrary {
   public void BGHehe() {
       Scanner sa = new Scanner(System.in);
       BGhelperPackage bg = new BGhelperPackage();
       bg.inputCoordinate(sa);
       System.out.print("Enter m (Slope): ");
       double m = sa.nextDouble();
       System.out.print("Enter c (Intercept): ");
      double c = sa.nextDouble();
       double denominator = m * m + 1;
       double matrixA = (1 - m * m) / denominator;
       double matrixB = (2 * m) / denominator;
       double matrixC = matrixB;
       double matrixD = (m * m - 1) / denominator;
       double translateX = (-2 * m * c) / denominator;
       double translateY = (2 * c) / denominator;
       AffineTransform BGTransform = new AffineTransform(
             matrixA, matrixB, matrixC, matrixD, translateX, translateY);
      Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya),
null);
```

```
        Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);

        Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc),
null);

        BGhelperPackage.CustomPanel AshliPanel = new
BGhelperPackage.CustomPanel(bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,

                (int) Math.round(A_Naya.getX()), (int) Math.round(A_Naya.getY()),

                (int) Math.round(B_Naya.getX()), (int) Math.round(B_Naya.getY()),

                (int) Math.round(C_Naya.getX()), (int) Math.round(C_Naya.getY()));

        BGhelperPackage.createFrame("Reflection Along y = mx + c - Library Function
with Bikram", AshliPanel);

}

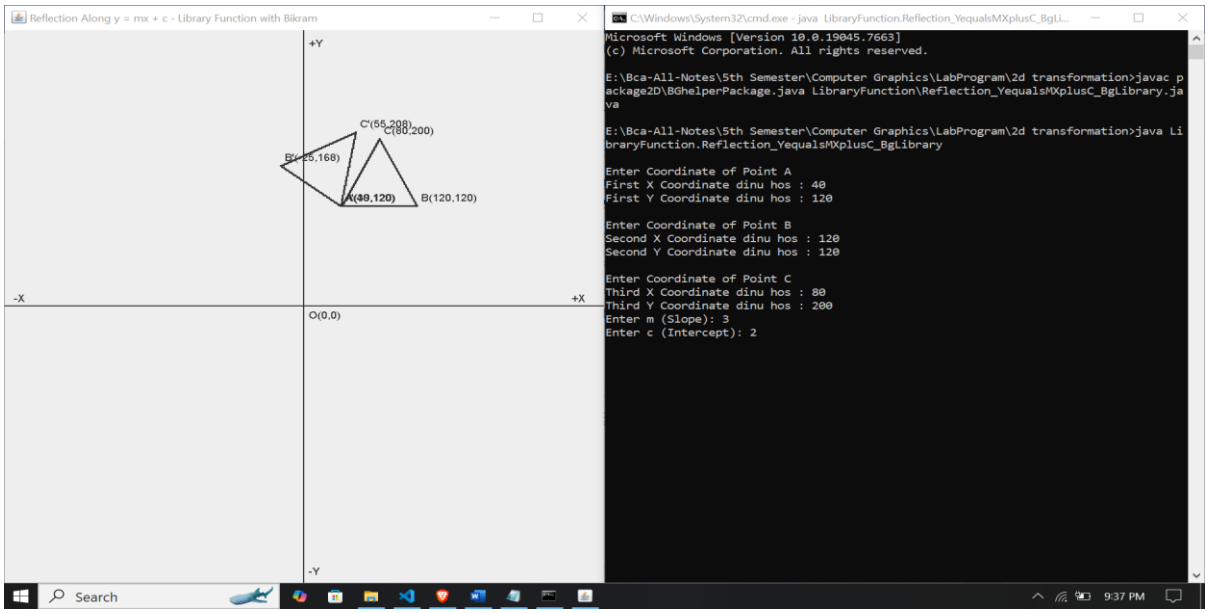

    public static void main(String[] args) {

        new Reflection_YequalsMXplusC_BgLibrary().BGHehe();

    }

}
```

**Output:**



## 24. 2d Shearing along X-Axis using Custom function.

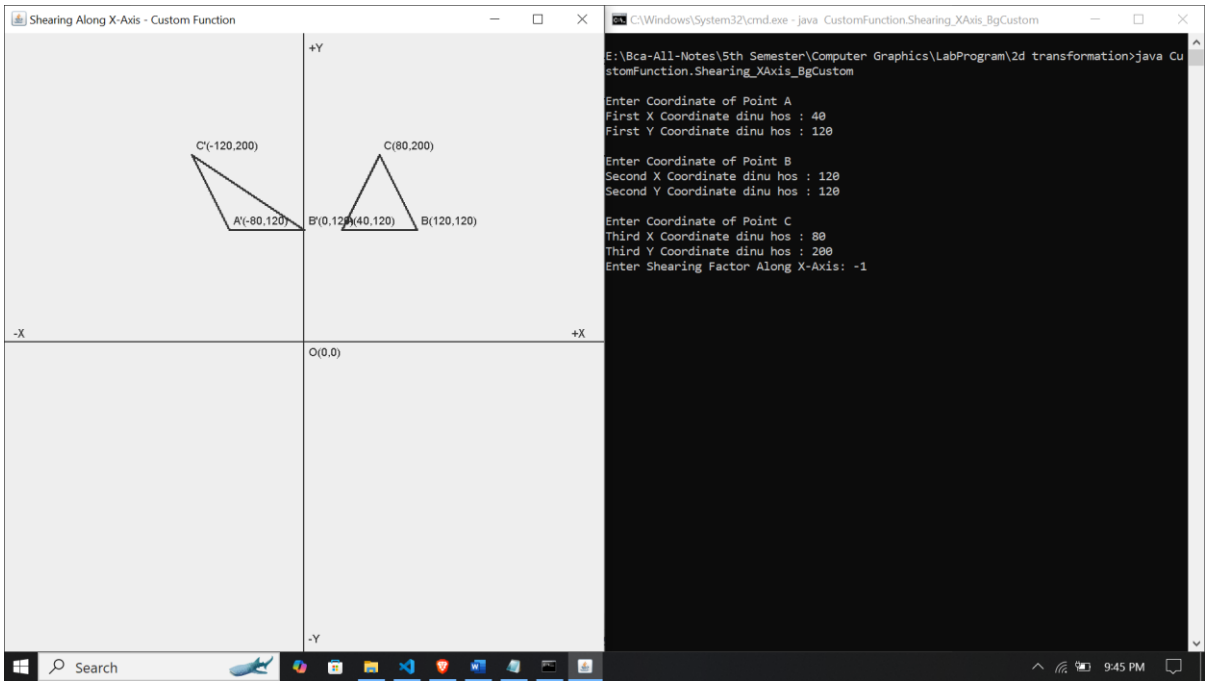
**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;

public class Shearing_XAxis_BgCustom {
   private double Shx;
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      System.out.print("Enter Shearing Factor Along X-Axis: ");
      Shx = sa.nextDouble();
      int Xa_Naya = BGShearingX(bg.Xa, bg.Ya);
      int Ya_Naya = BGShearingY(bg.Ya);
      int Xb_Naya = BGShearingX(bg.Xb, bg.Yb);
      int Yb_Naya = BGShearingY(bg.Yb);
      int Xc_Naya = BGShearingX(bg.Xc, bg.Yc);
      int Yc_Naya = BGShearingY(bg.Yc);
      BGhelperPackage.CustomPanel AshliPanel = new
BGhelperPackage.CustomPanel(bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            Xa_Naya, Ya_Naya, Xb_Naya, Yb_Naya, Xc_Naya, Yc_Naya);
      BGhelperPackage.createFrame("Shearing Along X-Axis - Custom Function",
AshliPanel);
   }
   public int BGShearingX(int Xa, int Ya) {
      return (int) Math.round(Xa + Shx * Ya);
```

```
    }
    public int BGShearingY(int Ya) {
        return Ya;
    }
    public static void main(String[] args) {
        new Shearing_XAxis_BgCustom().BGHehe();
    }
}
```

**Output:**



## 25. 2d Shearing along Y-Axis using Custom function.

**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;

public class Shearing_YAxis_BgCustom {
   private double Shy;
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);

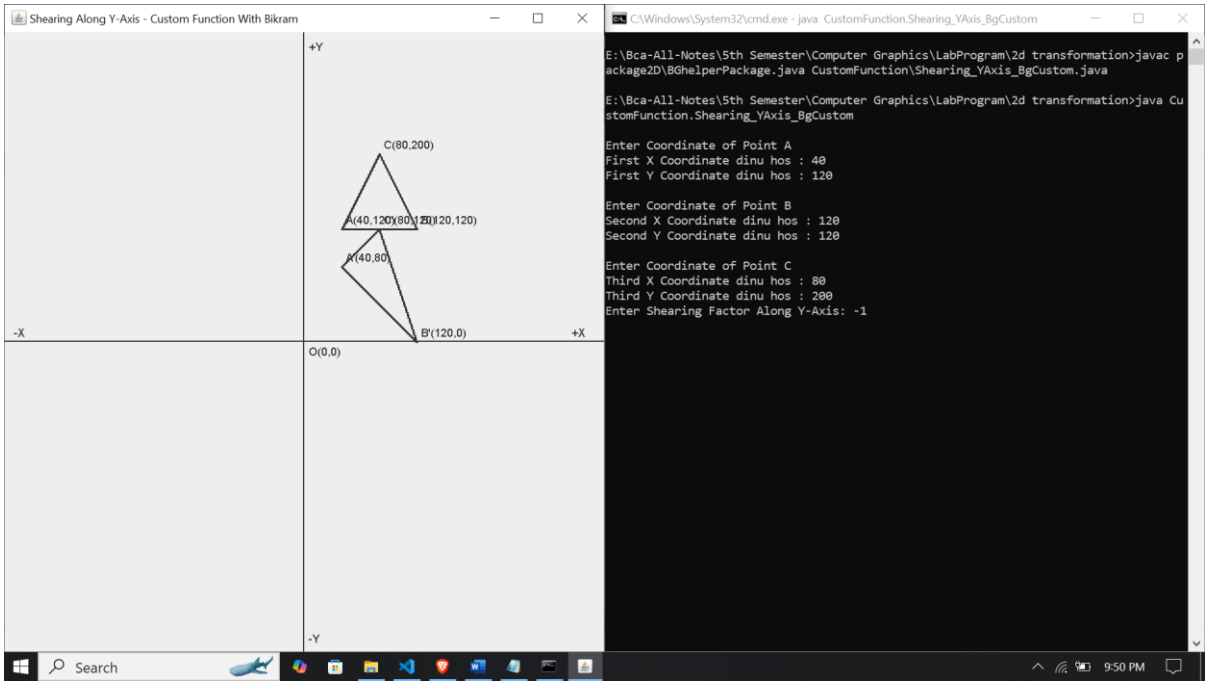
      System.out.print("Enter Shearing Factor Along Y-Axis: ");
      Shy = sa.nextDouble();
      int Xa_Naya = BGShearingX(bg.Xa);
      int Ya_Naya = BGShearingY(bg.Xa, bg.Ya);
      int Xb_Naya = BGShearingX(bg.Xb);
      int Yb_Naya = BGShearingY(bg.Xb, bg.Yb);
      int Xc_Naya = BGShearingX(bg.Xc);
      int Yc_Naya = BGShearingY(bg.Xc, bg.Yc);

      BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
            bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            Xa_Naya, Ya_Naya, Xb_Naya, Yb_Naya, Xc_Naya, Yc_Naya);

      BGhelperPackage.createFrame("Shearing Along Y-Axis - Custom Function With
Bikram ", AshliPanel);
   }
```

```
    public int BGShearingX(int Xa) {
        return Xa;
    }
    public int BGShearingY(int Xa, int Ya) {
        return (int) Math.round(Ya + Shy * Xa);
    }
    public static void main(String[] args) {
        new Shearing_YAxis_BgCustom().BGHehe();
    }
}
```

**Output:**



## 26. 2d Shearing along XY-Axis using Custom function.

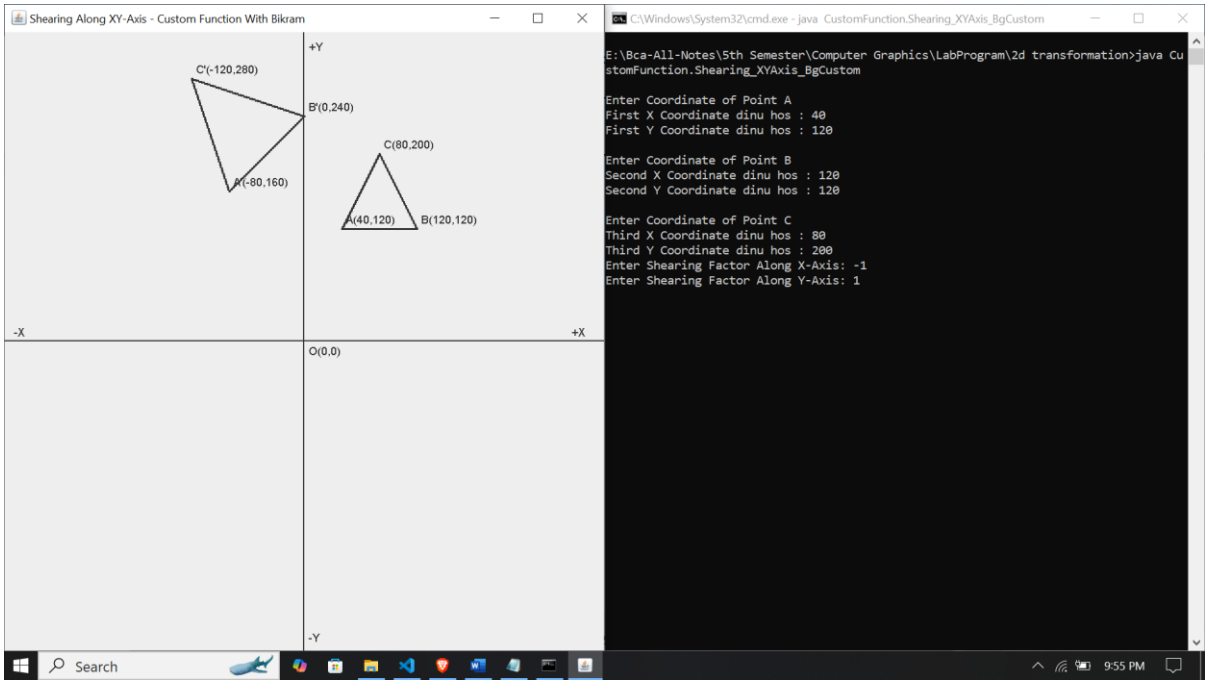
**Code:**

```
package CustomFunction;
import package2D.BGhelperPackage;
import java.util.Scanner;
public class Shearing_XYAxis_BgCustom {
   private double Shx;
   private double Shy;
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      System.out.print("Enter Shearing Factor Along X-Axis: ");
      Shx = sa.nextDouble();
      System.out.print("Enter Shearing Factor Along Y-Axis: ");
      Shy = sa.nextDouble();
      int Xa_Naya = BGShearingX(bg.Xa, bg.Ya);
      int Ya_Naya = BGShearingY(bg.Xa, bg.Ya);
      int Xb_Naya = BGShearingX(bg.Xb, bg.Yb);
      int Yb_Naya = BGShearingY(bg.Xb, bg.Yb);
      int Xc_Naya = BGShearingX(bg.Xc, bg.Yc);
      int Yc_Naya = BGShearingY(bg.Xc, bg.Yc);
      BGhelperPackage.CustomPanel AshliPanel = new
BGhelperPackage.CustomPanel(  bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            Xa_Naya, Ya_Naya, Xb_Naya, Yb_Naya, Xc_Naya, Yc_Naya);
      BGhelperPackage.createFrame("Shearing Along XY-Axis - Custom Function With
Bikram", AshliPanel);
   }
   public int BGShearingX(int Xa, int Ya) {
```

```
        return (int) Math.round(Xa + Shx * Ya);
    }
    public int BGShearingY(int Xa, int Ya) {
        return (int) Math.round(Ya + Shy * Xa);
    }

    public static void main(String[] args) {
        new Shearing_XYAxis_BgCustom().BGHehe();
    }
}
```

**Output:**



## 27. 2d Shearing along X-Axis using Library function.

**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;

public class Shearing_XAxis_BgLibrary {

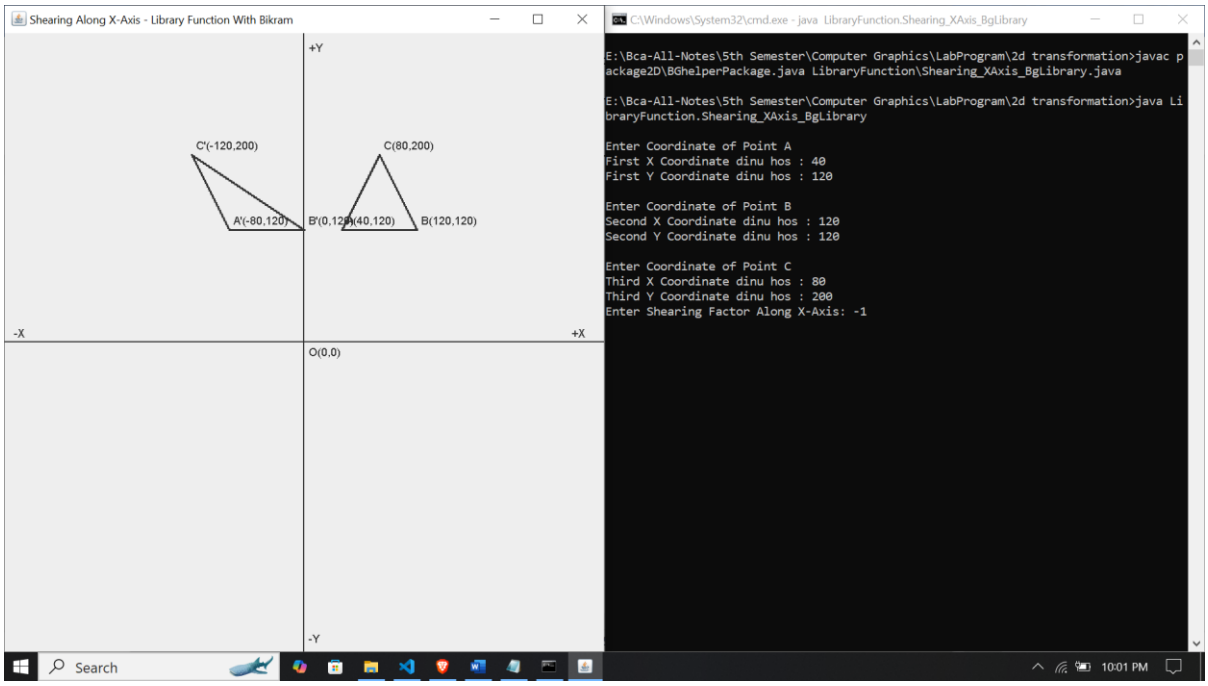
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      System.out.print("Enter Shearing Factor Along X-Axis: ");
      double Shx = sa.nextDouble();

      AffineTransform BGTransform = AffineTransform.getShearInstance(Shx, 0);
      Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya),
null);
      Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);
      Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc),
null);
      BGhelperPackage.CustomPanel AshliPanel = new
BGhelperPackage.CustomPanel(  bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            (int) Math.round(A_Naya.getX()), (int) Math.round(A_Naya.getY()),
            (int) Math.round(B_Naya.getX()), (int) Math.round(B_Naya.getY()),
            (int) Math.round(C_Naya.getX()), (int) Math.round(C_Naya.getY()));
```

```
        BGhelperPackage.createFrame("Shearing Along X-Axis - Library Function With
Bikram", AshliPanel);
    }

    public static void main(String[] args) {
        new Shearing_XAxis_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 28.2d Shearing along Y-Axis using Library function.

**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;

public class Shearing_YAxis_BgLibrary {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      System.out.print("Enter Shearing Factor Along Y-Axis: ");
      double Shy = sa.nextDouble();

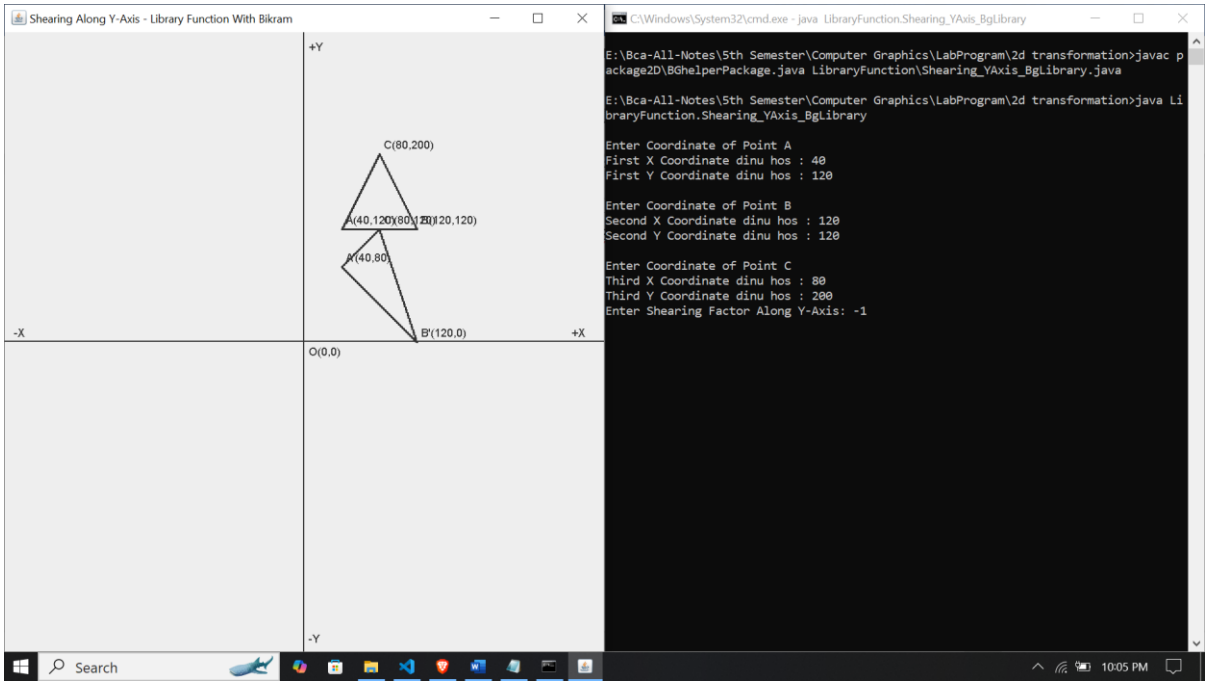
      AffineTransform BGTransform = AffineTransform.getShearInstance(0, Shy);

      Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya),
null);
      Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);
      Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc),
null);
      BGhelperPackage.CustomPanel AshliPanel = new
BGhelperPackage.CustomPanel(bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            (int) Math.round(A_Naya.getX()), (int) Math.round(A_Naya.getY()),
            (int) Math.round(B_Naya.getX()), (int) Math.round(B_Naya.getY()),
            (int) Math.round(C_Naya.getX()), (int) Math.round(C_Naya.getY()));
```

```
        BGhelperPackage.createFrame("Shearing Along Y-Axis - Library Function With
Bikram", AshliPanel);
    }

    public static void main(String[] args) {
        new Shearing_YAxis_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 29.2d Shearing along Xy-Axis using Library function.

**Code:**

```
package LibraryFunction;
import package2D.BGhelperPackage;
import java.awt.geom.AffineTransform;
import java.awt.geom.Point2D;
import java.util.Scanner;

public class Shearing_XYAxis_BgLibrary {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage bg = new BGhelperPackage();
      bg.inputCoordinate(sa);
      System.out.print("Enter Shearing Factor Along X-Axis: ");
      double Shx = sa.nextDouble();
      System.out.print("Enter Shearing Factor Along Y-Axis: ");
      double Shy = sa.nextDouble();

      AffineTransform BGTransform = AffineTransform.getShearInstance(Shx, Shy);

      Point2D A_Naya = BGTransform.transform(new Point2D.Double(bg.Xa, bg.Ya),
null);
      Point2D B_Naya = BGTransform.transform(new Point2D.Double(bg.Xb, bg.Yb),
null);
      Point2D C_Naya = BGTransform.transform(new Point2D.Double(bg.Xc, bg.Yc),
null);

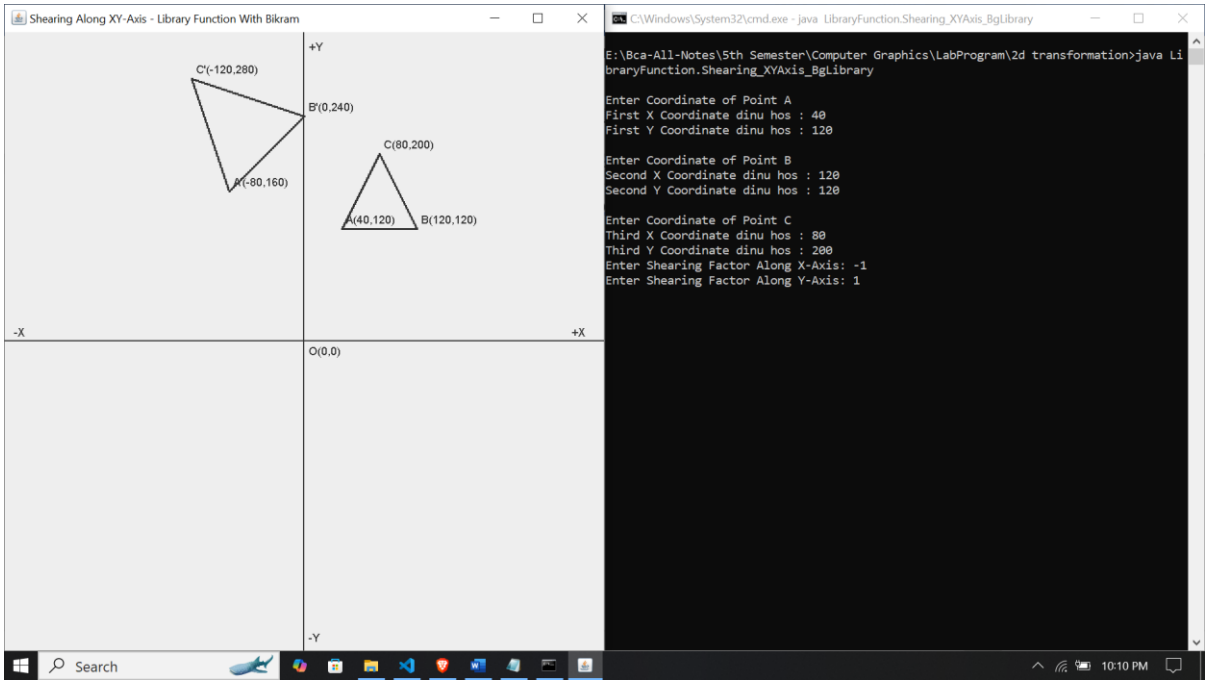
      BGhelperPackage.CustomPanel AshliPanel = new BGhelperPackage.CustomPanel(
            bg.Xa, bg.Ya, bg.Xb, bg.Yb, bg.Xc, bg.Yc,
            (int) Math.round(A_Naya.getX()), (int) Math.round(A_Naya.getY()),
```

```
            (int) Math.round(B_Naya.getX()), (int) Math.round(B_Naya.getY()),
            (int) Math.round(C_Naya.getX()), (int) Math.round(C_Naya.getY()));


        BGhelperPackage.createFrame("Shearing Along XY-Axis - Library Function With
Bikram", AshliPanel);
    }


    public static void main(String[] args) {
        new Shearing_XYAxis_BgLibrary().BGHehe();
    }
}
```

**Output:**



# II. 3-Dimensional Transformation

## 1. 3D package for input coordinate, JFrame used by all the program.

**Code:**

```
package package3D;
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;
public class BGhelperPackage3D {
   public double Xa, Ya, Za;
   public double Xb, Yb, Zb;
   public double Xc, Yc, Zc;
   public double Xd, Yd, Zd;
   public double Xe, Ye, Ze;
   public double Xf, Yf, Zf;
   public double Xg, Yg, Zg;
   public double Xh, Yh, Zh;
   public void inputCoordinate(Scanner sa) {
      System.out.println("Enter 8 Coordinates for Ghan (Cube):");
      System.out.print("A - Enter garnus X,Y,Z: ");
      Xa = sa.nextDouble();
      Ya = sa.nextDouble();
      Za = sa.nextDouble();
      System.out.print("B - Enter garnus X,Y,Z: ");
      Xb = sa.nextDouble();
      Yb = sa.nextDouble();
      Zb = sa.nextDouble();
      System.out.print("C - Enter garnus X,Y,Z: ");
      Xc = sa.nextDouble();
      Yc = sa.nextDouble();
```

```
        Zc = sa.nextDouble();
        System.out.print("D - Enter garnus X,Y,Z: ");
        Xd = sa.nextDouble();
        Yd = sa.nextDouble();
        Zd = sa.nextDouble();
        System.out.print("E - Enter garnus X,Y,Z: ");
        Xe = sa.nextDouble();
        Ye = sa.nextDouble();
        Ze = sa.nextDouble();
        System.out.print("F - Enter garnus X,Y,Z: ");
        Xf = sa.nextDouble();
        Yf = sa.nextDouble();
        Zf = sa.nextDouble();
        System.out.print("G - Enter garnus X,Y,Z: ");
        Xg = sa.nextDouble();
        Yg = sa.nextDouble();
        Zg = sa.nextDouble();
        System.out.print("H - Enter garnus X,Y,Z: ");
        Xh = sa.nextDouble();
        Yh = sa.nextDouble();
        Zh = sa.nextDouble();
    }
    public double[][] getOriginalPoint() {
        return new double[][] {
                { Xa, Ya, Za },{ Xb, Yb, Zb },{ Xc, Yc, Zc }, { Xd, Yd, Zd },{ Xe, Ye, Ze },{
Xf, Yf, Zf },{ Xg, Yg, Zg },{ Xh, Yh, Zh }
        };
    }

    public void showOriginalCoordinate(double[][] BGPoint) {
        char[] BGName = { 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H' };
```

```
    System.out.println("\nOriginal Coordinates:");
    for (int Bikram = 0; Bikram < 8; Bikram++) {
        System.out.println(BGName[Bikram] + " = (" +
                BGPoint[Bikram][0] + ", " +
                BGPoint[Bikram][1] + ", " +
                BGPoint[Bikram][2] + ")");
    }
}
public void showTransformedCoordinate(double[][] BGPoint) {
    char[] BGName = { 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H' };
    System.out.println("\nTransformed Coordinates:");
    for (int Bikram = 0; Bikram < 8; Bikram++) {
        System.out.println(BGName[Bikram] + "' = (" +
                BGPoint[Bikram][0] + ", " +
                BGPoint[Bikram][1] + ", " +
                BGPoint[Bikram][2] + ")");
    }
}
public static void createFrame(String BGTitle, JPanel AshliPanel) {
    JFrame BGF = new JFrame(BGTitle);
    BGF.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    BGF.setSize(1000, 750);
    BGF.setLocationRelativeTo(null);
    BGF.add(AshliPanel);
    BGF.setVisible(true);
}
public static class CustomPanel3D extends JPanel {
    private double[][] BGOriginal;
    private double[][] BGTransformed;
    public CustomPanel3D(double[][] BGOriginal, double[][] BGTransformed) {
```

```
        this.BGOriginal = BGOriginal;
        this.BGTransformed = BGTransformed;
    }
    public int BGScreenX(double XCoordinate, double ZCoordinate) {
        return (int) Math.round(getWidth() / 2 + XCoordinate * 30 - ZCoordinate * 15);
    }
    public int BGScreenY(double YCoordinate, double ZCoordinate) {
        return (int) Math.round(getHeight() / 2 - YCoordinate * 30 + ZCoordinate * 15);
    }
    public void BGDrawAxes(Graphics2D g2) {
        int centerX = getWidth() / 2;
        int centerY = getHeight() / 2;
        g2.setStroke(new BasicStroke(2));
        g2.drawLine(0, centerY, getWidth(), centerY);
        g2.drawString("X", getWidth() - 25, centerY - 8);
        g2.drawLine(centerX, 0, centerX, getHeight());
        g2.drawString("Y", centerX + 8, 25);
        g2.drawLine(centerX, centerY, centerX - 180, centerY + 180);
        g2.drawString("Z", centerX - 200, centerY + 190);
        g2.drawString("O(0,0,0)", centerX + 8, centerY + 18);
    }
    public void BGDrawCube(Graphics2D g2, double[][] BGPoint, boolean
BGIsTransformed) {
        int[][] BGEdge = {{ 0, 1 }, { 1, 2 }, { 2, 3 }, { 3, 0 },
                { 4, 5 }, { 5, 6 }, { 6, 7 }, { 7, 4 },
                { 0, 4 }, { 1, 5 }, { 2, 6 }, { 3, 7 }
        };
        char[] BGName = { 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H' };
        g2.setStroke(new BasicStroke(2));
        for (int[] BGLine : BGEdge) {
            int BGFirst = BGLine[0];
```

```
            int BGSecond = BGLine[1];
            int X1 = BGScreenX(BGPoint[BGFirst][0], BGPoint[BGFirst][2]);
            int Y1 = BGScreenY(BGPoint[BGFirst][1], BGPoint[BGFirst][2]);
            int X2 = BGScreenX(BGPoint[BGSecond][0], BGPoint[BGSecond][2]);
            int Y2 = BGScreenY(BGPoint[BGSecond][1], BGPoint[BGSecond][2]);
            g2.drawLine(X1, Y1, X2, Y2);
        }
        for (int Bikram = 0; Bikram < 8; Bikram++) {
            int X = BGScreenX(BGPoint[Bikram][0], BGPoint[Bikram][2]);
            int Y = BGScreenY(BGPoint[Bikram][1], BGPoint[Bikram][2]);
            g2.fillOval(X - 4, Y - 4, 8, 8);
            String BGLabel = BGIsTransformed? BGName[Bikram] + "'": 
String.valueOf(BGName[Bikram]);
            g2.drawString(BGLabel, X + 6, Y - 6);
        }
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;

        BGDrawAxes(g2);
        BGDrawCube(g2, BGOriginal, false);
        BGDrawCube(g2, BGTransformed, true);

        g2.drawString("Original Cube: A,B,C,D,E,F,G,H", 20, 30);
        g2.drawString("Transformed Cube: A',B',C',D',E',F',G',H'", 20, 55);
    }
  }
}
```

## 2. 3D package for Library Function Programs.

**Code:**

```
package package3D;
public class BGhelper3DLibrary {
   public static double[][] BGIdentityMatrix() {
      return new double[][] {
            { 1, 0, 0, 0 },
            { 0, 1, 0, 0 },
            { 0, 0, 1, 0 },
            { 0, 0, 0, 1 }
      };
   }
   public static double[][] BGTranslationMatrix(double Tx, double Ty, double Tz) {
      return new double[][] {
            { 1, 0, 0, Tx },
            { 0, 1, 0, Ty },
            { 0, 0, 1, Tz },
            { 0, 0, 0, 1 }
      };
   }
   public static double[][] BGScalingMatrix(double Sx, double Sy, double Sz) {
      return new double[][] {
            { Sx, 0, 0, 0 },
            { 0, Sy, 0, 0 },
            { 0, 0, Sz, 0 },
            { 0, 0, 0, 1 }
      };
   }
   public static double[][] BGRotationXMatrix(double BGAngle) {
      double BGRadian = Math.toRadians(BGAngle);
```

```
    double BGCos = Math.cos(BGRadian);
    double BGSin = Math.sin(BGRadian);

    return new double[][] {
        { 1, 0, 0, 0 },{ 0, BGCos, -BGSin, 0 },{ 0, BGSin, BGCos, 0 },{ 0, 0, 0, 1 }
    };
}

public static double[][] BGRotationYMatrix(double BGAngle) {
    double BGRadian = Math.toRadians(BGAngle);
    double BGCos = Math.cos(BGRadian);
    double BGSin = Math.sin(BGRadian);

    return new double[][] {
        { BGCos, 0, BGSin, 0 },
        { 0, 1, 0, 0 },
        { -BGSin, 0, BGCos, 0 },
        { 0, 0, 0, 1 }
    };
}

public static double[][] BGRotationZMatrix(double BGAngle) {
    double BGRadian = Math.toRadians(BGAngle);
    double BGCos = Math.cos(BGRadian);
    double BGSin = Math.sin(BGRadian);

    return new double[][] {
        { BGCos, -BGSin, 0, 0 },
        { BGSin, BGCos, 0, 0 },
        { 0, 0, 1, 0 },
```

```
            { 0, 0, 0, 1 }
        };
    }

    public static double[][] BGReflectionZXMatrix() {
        return new double[][] {
            { 1, 0, 0, 0 },
            { 0, -1, 0, 0 },
            { 0, 0, 1, 0 },
            { 0, 0, 0, 1 }
        };
    }

    public static double[][] BGReflectionYZMatrix() {
        return new double[][] {
            { -1, 0, 0, 0 },
            { 0, 1, 0, 0 },
            { 0, 0, 1, 0 },
            { 0, 0, 0, 1 }
        };
    }

    public static double[][] BGReflectionXYMatrix() {
        return new double[][] {
            { 1, 0, 0, 0 },
            { 0, 1, 0, 0 },
            { 0, 0, -1, 0 },
            { 0, 0, 0, 1 }
        };
    }
```

```
public static double[][] BGShearingXMatrix(double b, double c) {
    return new double[][] {
            { 1, 0, 0, 0 },
            { b, 1, 0, 0 },
            { c, 0, 1, 0 },
            { 0, 0, 0, 1 }
    };
}

public static double[][] BGShearingYMatrix(double a, double c) {
    return new double[][] {
            { 1, a, 0, 0 },
            { 0, 1, 0, 0 },
            { 0, c, 1, 0 },
            { 0, 0, 0, 1 }
    };
}
public static double[][] BGShearingZMatrix(double a, double b) {
    return new double[][] {
            { 1, 0, a, 0 },
            { 0, 1, b, 0 },
            { 0, 0, 1, 0 },
            { 0, 0, 0, 1 }
    };
}
public static double[][] BGMultiplyMatrix(double[][] BGFirst, double[][] BGSecond) {
    double[][] BGResult = new double[4][4];
    for (int Bikram = 0; Bikram < 4; Bikram++) {
        for (int Gyawali = 0; Gyawali < 4; Gyawali++) {
```

```
            for (int Ashli = 0; Ashli < 4; Ashli++) {
                BGResult[Bikram][Gyawali] += BGFirst[Bikram][Ashli] *
BGSecond[Ashli][Gyawali];
            }
        }
    }
    return BGResult;
  }
  public static double[][] BGApplyMatrix(double[][] BGPoint, double[][] BGMatrix) {
    double[][] BGTransformed = new double[8][3];
    for (int Bikram = 0; Bikram < 8; Bikram++) {
      double[] BGCoordinate = {
          BGPoint[Bikram][0],
          BGPoint[Bikram][1],
          BGPoint[Bikram][2],
          1
      };
      double[] BGNaya = new double[4];
      for (int Gyawali = 0; Gyawali < 4; Gyawali++) {
        for (int Ashli = 0; Ashli < 4; Ashli++) {
          BGNaya[Gyawali] += BGMatrix[Gyawali][Ashli] * BGCoordinate[Ashli];
        }
      }
      BGTransformed[Bikram][0] = BGNaya[0];
      BGTransformed[Bikram][1] = BGNaya[1];
      BGTransformed[Bikram][2] = BGNaya[2];
    }
    return BGTransformed;
  }
}
```

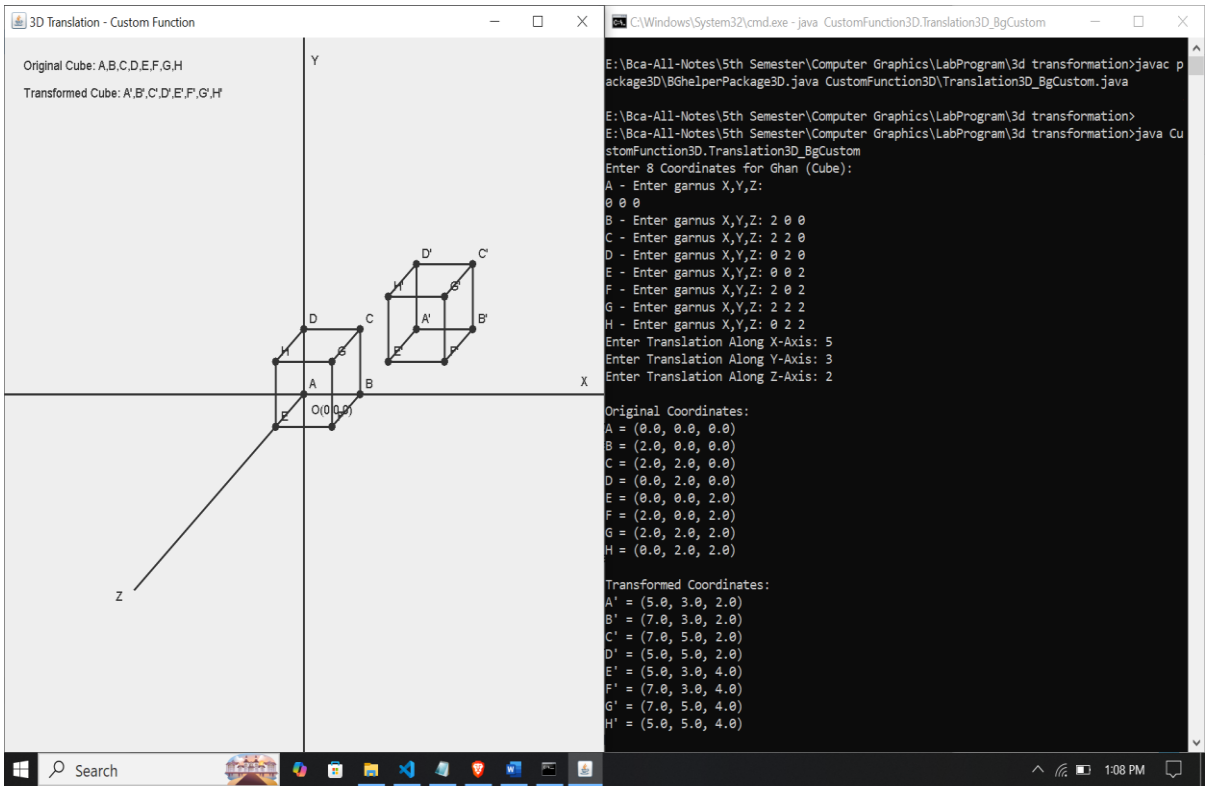
## 3. 3d Translation using Custom function.

**Code:**

```
package CustomFunction3D;
import package3D.BGhelperPackage3D;
import java.util.Scanner;
public class Translation3D_BgCustom {
   public double[][] BGTranslation(double[][] BGOriginal,double Tx,double Ty,double
Tz) {
      double[][] BGTransformed=new double[8][3];
      for(int Bikram=0;Bikram<8;Bikram++) {
         BGTransformed[Bikram][0]=BGOriginal[Bikram][0]+Tx;
         BGTransformed[Bikram][1]=BGOriginal[Bikram][1]+Ty;
         BGTransformed[Bikram][2]=BGOriginal[Bikram][2]+Tz;
      }
      return BGTransformed;
   }
   public void BGHehe() {
      Scanner sa=new Scanner(System.in);
      BGhelperPackage3D bg=new BGhelperPackage3D();
      bg.inputCoordinate(sa);
      System.out.print("Enter Translation Along X-Axis: ");
      double Tx=sa.nextDouble();
      System.out.print("Enter Translation Along Y-Axis: ");
      double Ty=sa.nextDouble();
      System.out.print("Enter Translation Along Z-Axis: ");
      double Tz=sa.nextDouble();
      double[][] BGOriginal=bg.getOriginalPoint();
      double[][] BGTransformed=BGTranslation(BGOriginal,Tx,Ty,Tz);
      bg.showOriginalCoordinate(BGOriginal);
      bg.showTransformedCoordinate(BGTransformed);
```

```
        BGhelperPackage3D.CustomPanel3D AshliPanel=
            new BGhelperPackage3D.CustomPanel3D(BGOriginal,BGTransformed);
        BGhelperPackage3D.createFrame("3D Translation - Custom Function",AshliPanel);
    }
    public static void main(String[] args) {
        new Translation3D_BgCustom().BGHehe();
    }
}
```

**Output:**



## 4. 3D Translation using Library Function.

**Code:**

```
package LibraryFunction3D;
import package3D.BGhelperPackage3D;
import package3D.BGhelper3DLibrary;
import java.util.Scanner;
public class Translation3D_BgLibrary {
   public void BGHehe() {
       Scanner sa = new Scanner(System.in);
       BGhelperPackage3D bg = new BGhelperPackage3D();
       bg.inputCoordinate(sa);

       System.out.print("Enter Translation Along X-Axis: ");
       double Tx = sa.nextDouble();

       System.out.print("Enter Translation Along Y-Axis: ");
       double Ty = sa.nextDouble();

       System.out.print("Enter Translation Along Z-Axis: ");
       double Tz = sa.nextDouble();

       double[][] BGOriginal = bg.getOriginalPoint();

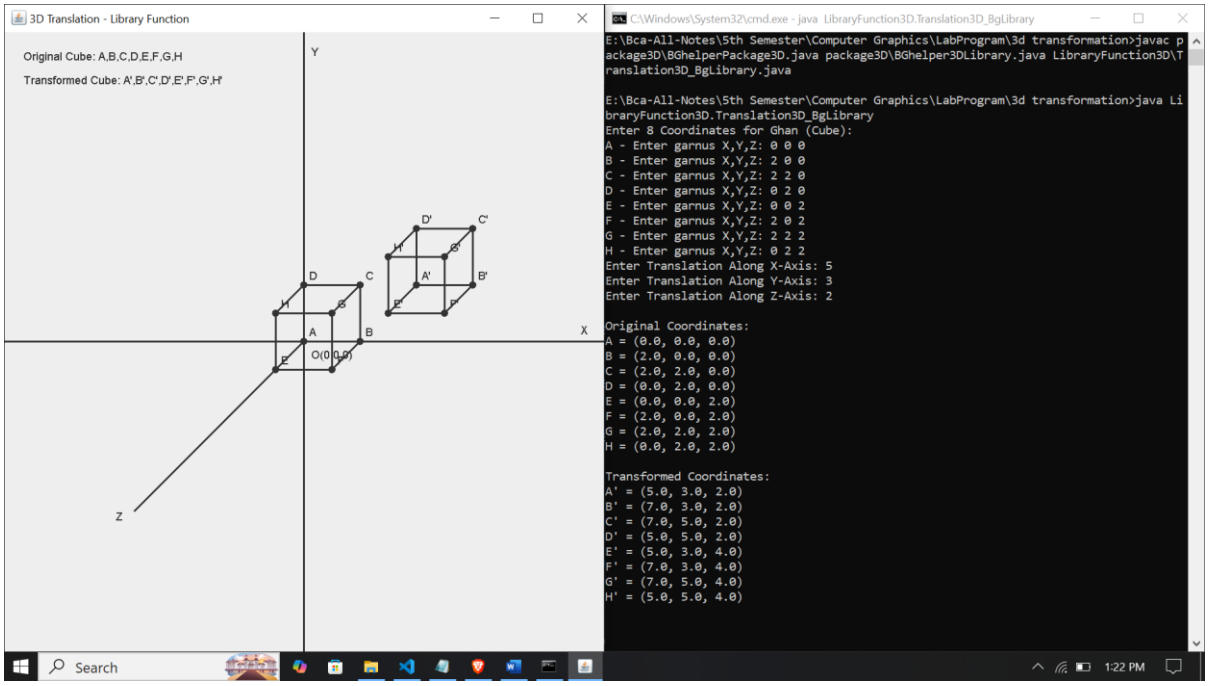
       double[][] BGMatrix = BGhelper3DLibrary.BGTranslationMatrix(Tx, Ty, Tz);
       double[][] BGTransformed = BGhelper3DLibrary.BGApplyMatrix(BGOriginal,
BGMatrix);

       bg.showOriginalCoordinate(BGOriginal);
       bg.showTransformedCoordinate(BGTransformed);
```

```
        BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);


        BGhelperPackage3D.createFrame("3D Translation - Library Function",
                AshliPanel);
    }
    public static void main(String[] args) {
        new Translation3D_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 5. 3D Scaling through origin using Custom Function.

**Code:**

```
package CustomFunction3D;
import package3D.BGhelperPackage3D;
import java.util.Scanner;
public class Scaling_Origin3D_BgCustom {
   public double[][] BGScaling(double[][] BGOriginal, double Sx, double Sy, double Sz)
{
      double[][] BGTransformed = new double[8][3];
      for (int Bikram = 0; Bikram < 8; Bikram++) {
         BGTransformed[Bikram][0] = BGOriginal[Bikram][0] * Sx;
         BGTransformed[Bikram][1] = BGOriginal[Bikram][1] * Sy;
         BGTransformed[Bikram][2] = BGOriginal[Bikram][2] * Sz;
      }
      return BGTransformed;
   }
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage3D bg = new BGhelperPackage3D();
      bg.inputCoordinate(sa);
      System.out.print("Enter Scaling Factor Along X-Axis: ");
      double Sx = sa.nextDouble();
      System.out.print("Enter Scaling Factor Along Y-Axis: ");
      double Sy = sa.nextDouble();
      System.out.print("Enter Scaling Factor Along Z-Axis: ");
      double Sz = sa.nextDouble();
      double[][] BGOriginal = bg.getOriginalPoint();
      double[][] BGTransformed = BGScaling(BGOriginal, Sx, Sy, Sz);
      bg.showOriginalCoordinate(BGOriginal);
```

```
        bg.showTransformedCoordinate(BGTransformed);

        BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);

        BGhelperPackage3D.createFrame("3D Scaling About Origin - Custom Function
With Bikram", AshliPanel);

    }

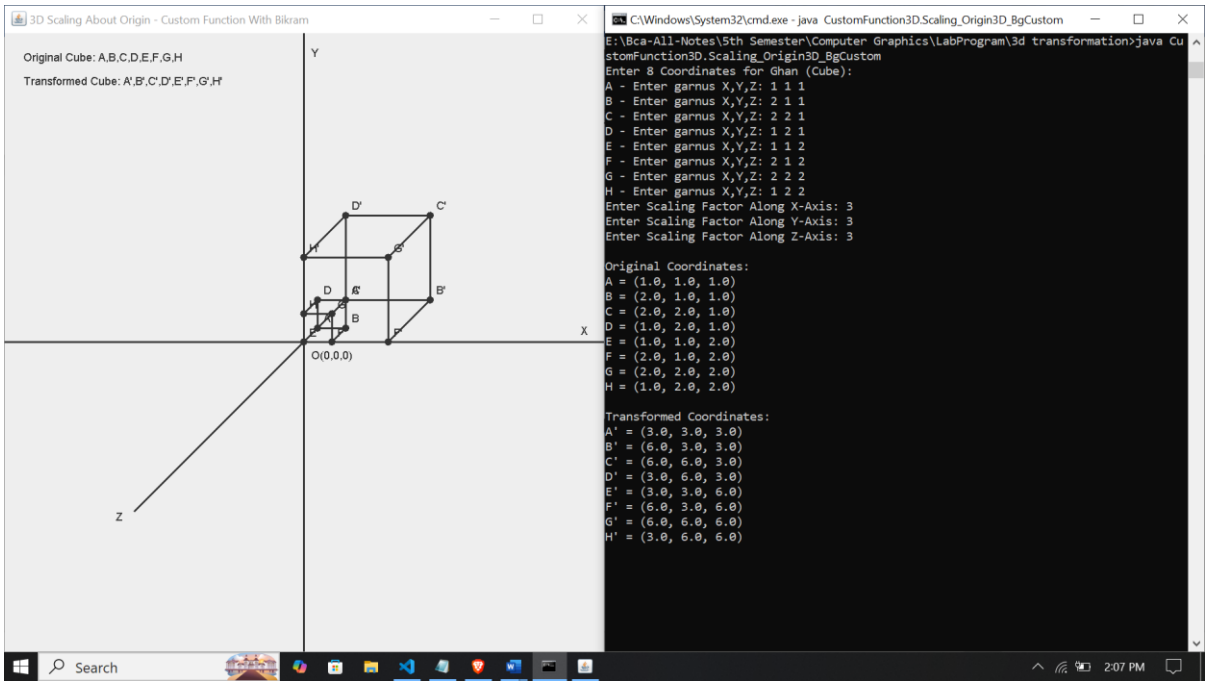
    public static void main(String[] args) {

        new Scaling_Origin3D_BgCustom().BGHehe();

    }

}
```

**Output:**



## 6. 3D Scaling through selected Fixed-point using Custom Function.

**Code:**

```
package CustomFunction3D;
import package3D.BGhelperPackage3D;
import java.util.Scanner;

public class Scaling_FixedPoint3D_BgCustom {
   public double[][] BGScalingFixedPoint(double[][] BGOriginal, double Sx, double Sy,
double Sz, double Xfixed,
         double Yfixed, double Zfixed) {
      double[][] BGTransformed = new double[8][3];
      for (int Bikram = 0; Bikram < 8; Bikram++) {
         double XTranslate = BGOriginal[Bikram][0] - Xfixed;
         double YTranslate = BGOriginal[Bikram][1] - Yfixed;
         double ZTranslate = BGOriginal[Bikram][2] - Zfixed;
         double XScale = XTranslate * Sx;
         double YScale = YTranslate * Sy;
         double ZScale = ZTranslate * Sz;

         BGTransformed[Bikram][0] = XScale + Xfixed;
         BGTransformed[Bikram][1] = YScale + Yfixed;
         BGTransformed[Bikram][2] = ZScale + Zfixed;
      }

      return BGTransformed;
   }

   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
```

```
        BGhelperPackage3D bg = new BGhelperPackage3D();

        bg.inputCoordinate(sa);

        System.out.print("Enter Scaling Factor Along X-Axis: ");
        double Sx = sa.nextDouble();

        System.out.print("Enter Scaling Factor Along Y-Axis: ");
        double Sy = sa.nextDouble();

        System.out.print("Enter Scaling Factor Along Z-Axis: ");
        double Sz = sa.nextDouble();

        System.out.print("Enter Selected Fixed Point X: ");
        double Xfixed = sa.nextDouble();

        System.out.print("Enter Selected Fixed Point Y: ");
        double Yfixed = sa.nextDouble();

        System.out.print("Enter Selected Fixed Point Z: ");
        double Zfixed = sa.nextDouble();

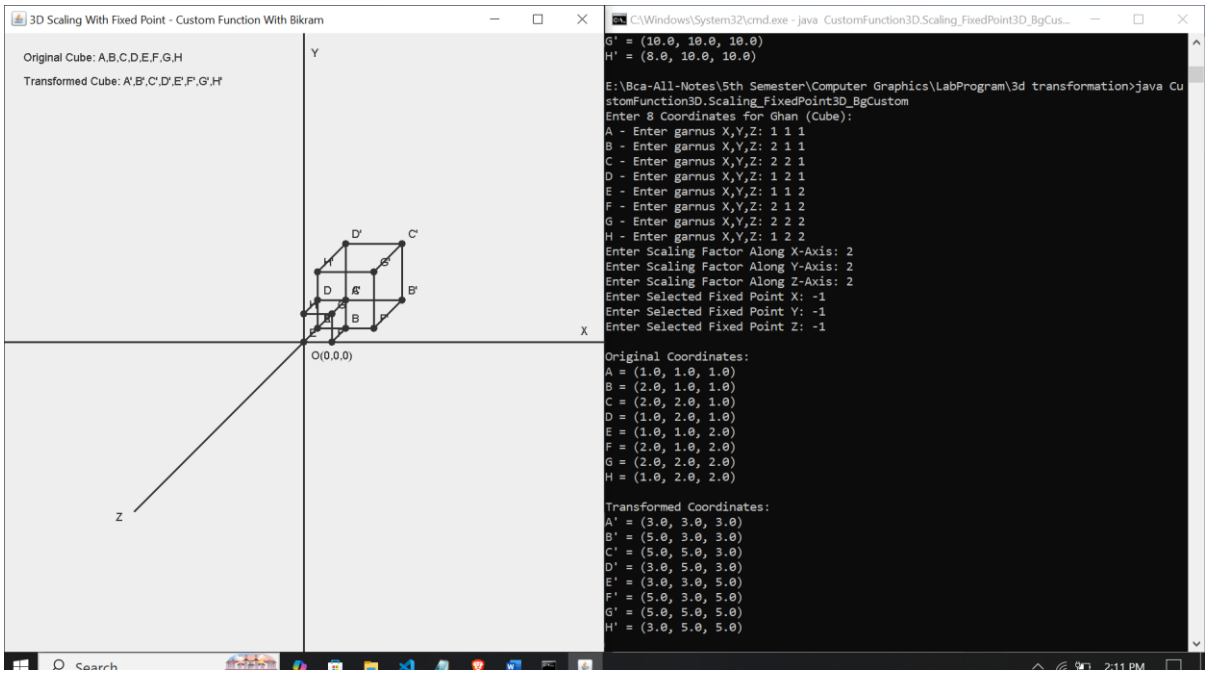
        double[][] BGOriginal = bg.getOriginalPoint();
        double[][] BGTransformed = BGScalingFixedPoint(BGOriginal, Sx, Sy, Sz, Xfixed,
Yfixed, Zfixed);

        bg.showOriginalCoordinate(BGOriginal);
        bg.showTransformedCoordinate(BGTransformed);
        BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);
```

```
        BGhelperPackage3D.createFrame("3D Scaling With Fixed Point - Custom Function
With Bikram", AshliPanel);
    }

    public static void main(String[] args) {
        new Scaling_FixedPoint3D_BgCustom().BGHehe();
    }
}
```

**Output:**



## 7. 3D Scaling through origin using Library Function.

**Code:**

```
package LibraryFunction3D;
import package3D.BGhelperPackage3D;
import package3D.BGhelper3DLibrary;
import java.util.Scanner;

public class Scaling_Origin3D_BgLibrary {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackage3D bg = new BGhelperPackage3D();
      bg.inputCoordinate(sa);

      System.out.print("Enter Scaling Factor Along X-Axis: ");
      double Sx = sa.nextDouble();

      System.out.print("Enter Scaling Factor Along Y-Axis: ");
      double Sy = sa.nextDouble();

      System.out.print("Enter Scaling Factor Along Z-Axis: ");
      double Sz = sa.nextDouble();

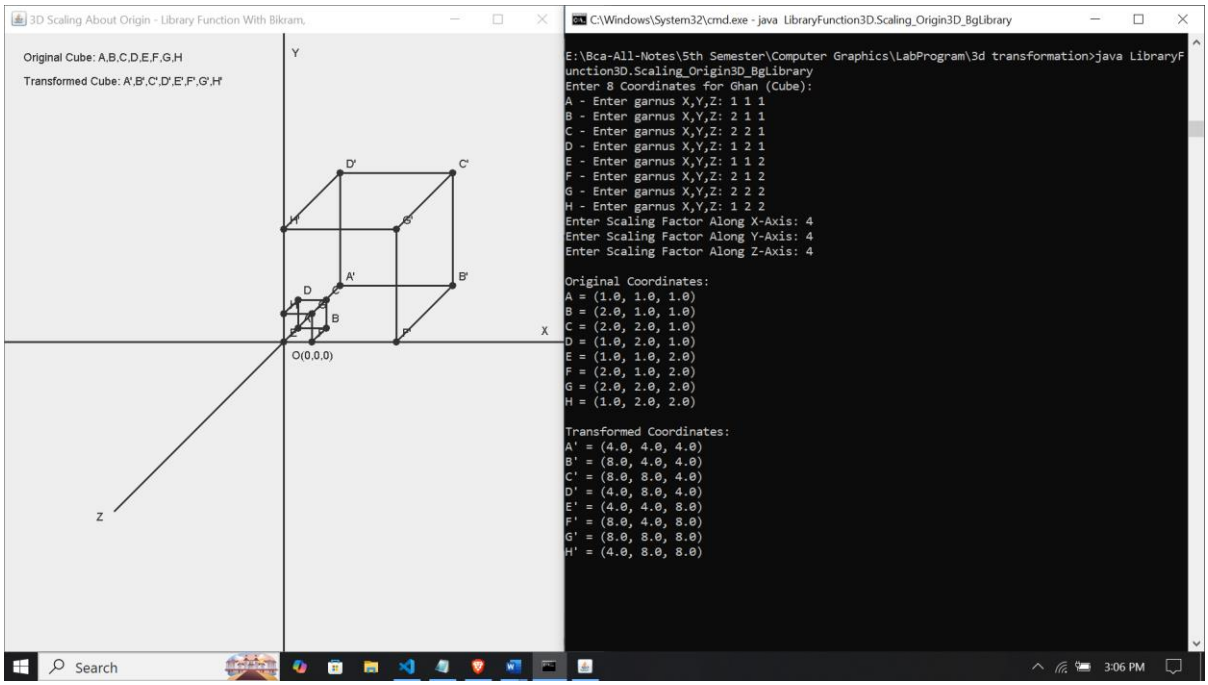
      double[][] BGOriginal = bg.getOriginalPoint();
      double[][] BGMatrix = BGhelper3DLibrary.BGScalingMatrix(Sx, Sy, Sz);
      double[][] BGTransformed = BGhelper3DLibrary.BGApplyMatrix(BGOriginal,
BGMatrix);

      bg.showOriginalCoordinate(BGOriginal);
      bg.showTransformedCoordinate(BGTransformed);
```

```
        BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);


        BGhelperPackage3D.createFrame( "3D Scaling About Origin - Library Function
With Bikram,",AshliPanel);        BGhelperPackage3D.createFrame(
                "3D Scaling About Origin - Library Function With Bikram,",
                AshliPanel);
    }
    public static void main(String[] args) {
        new Scaling_Origin3D_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 8. 3D Scaling though fixed-point using Library Function.

**Code:**

```
package LibraryFunction3D;
import package3D.BGhelperPackage3D;
import package3D.BGhelper3DLibrary;
import java.util.Scanner;
public class Scaling_FixedPoint3D_BgLibrary {
   public void BGHehe() {
       Scanner sa = new Scanner(System.in);
       BGhelperPackage3D bg = new BGhelperPackage3D();
       bg.inputCoordinate(sa);
       System.out.print("Enter Scaling Factor Along X-Axis: ");
       double Sx = sa.nextDouble();
       System.out.print("Enter Scaling Factor Along Y-Axis: ");
       double Sy = sa.nextDouble();
       System.out.print("Enter Scaling Factor Along Z-Axis: ");
       double Sz = sa.nextDouble();
       System.out.print("Enter Fixed Point X: ");
       double Xfixed = sa.nextDouble();
       System.out.print("Enter Fixed Point Y: ");
       double Yfixed = sa.nextDouble();
       System.out.print("Enter Fixed Point Z: ");
       double Zfixed = sa.nextDouble();
       double[][] BGOriginal = bg.getOriginalPoint();
       double[][] BGTranslate = BGhelper3DLibrary.BGTranslationMatrix( -Xfixed, -
Yfixed, -Zfixed);
       double[][] BGScale = BGhelper3DLibrary.BGScalingMatrix(Sx, Sy, Sz);
       double[][] BGInverseTranslate = BGhelper3DLibrary.BGTranslationMatrix(
             Xfixed, Yfixed, Zfixed);
```

```
        double[][] BGMatrix =
BGhelper3DLibrary.BGMultiplyMatrix( BGInverseTranslate, BGhelper3DLibrary.BGM
ultiplyMatrix( BGScale, BGTranslate));

        double[][] BGTransformed = BGhelper3DLibrary.BGApplyMatrix(BGOriginal,
BGMatrix);

        bg.showOriginalCoordinate(BGOriginal);

        bg.showTransformedCoordinate(BGTransformed);

        BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);

        BGhelperPackage3D.createFrame( "3D Scaling With Fixed Point - Library Function
With Bikram", AshliPanel);

    }

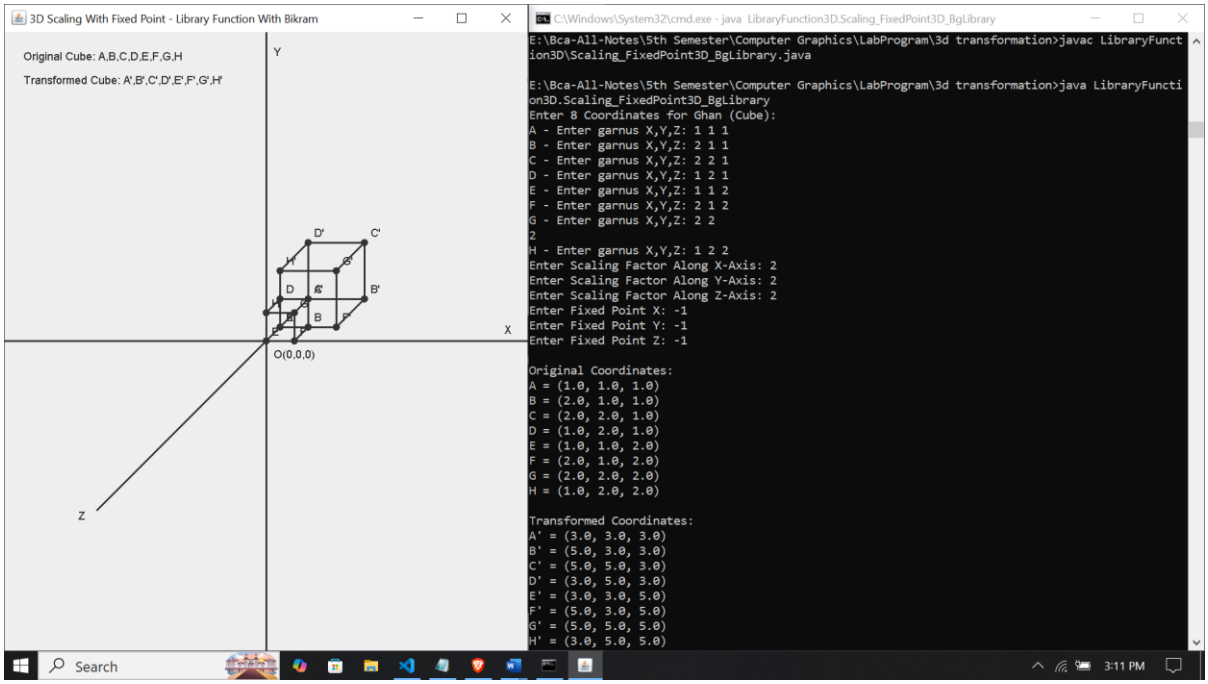

    public static void main(String[] args) {

        new Scaling_FixedPoint3D_BgLibrary().BGHehe();

    }

}
```

**Output:**



## 9. 3D Rotation through X-Axis using Custom Function

**Code:**

```
package CustomFunction3D;
import package3D.BGhelperPackage3D;
import java.util.Scanner;

public class Rotation_XAxis3D_BgCustom {
	public double[][] BGRotationX(double[][] BGOriginal, double BGAngle) {
		double[][] BGTransformed = new double[8][3];
		double BGRadian = Math.toRadians(BGAngle);
		for (int Bikram = 0; Bikram < 8; Bikram++) {
			double X = BGOriginal[Bikram][0];
			double Y = BGOriginal[Bikram][1];
			double Z = BGOriginal[Bikram][2];
			BGTransformed[Bikram][0] = X;
			BGTransformed[Bikram][1] = Y * Math.cos(BGRadian) - Z *
Math.sin(BGRadian);
			BGTransformed[Bikram][2] = Y * Math.sin(BGRadian) + Z *
Math.cos(BGRadian);
		}
		return BGTransformed;
	}
	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();
		bg.inputCoordinate(sa);
		System.out.print("Enter Rotation Angle in Degree: ");
		double BGAngle = sa.nextDouble();
		double[][] BGOriginal = bg.getOriginalPoint();
		double[][] BGTransformed = BGRotationX(BGOriginal, BGAngle);
```

```
                bg.showOriginalCoordinate(BGOriginal);

                bg.showTransformedCoordinate(BGTransformed);

                BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);

                BGhelperPackage3D.createFrame("3D Rotation Along X-Axis - Custom
Function With Bikram", AshliPanel);

        }

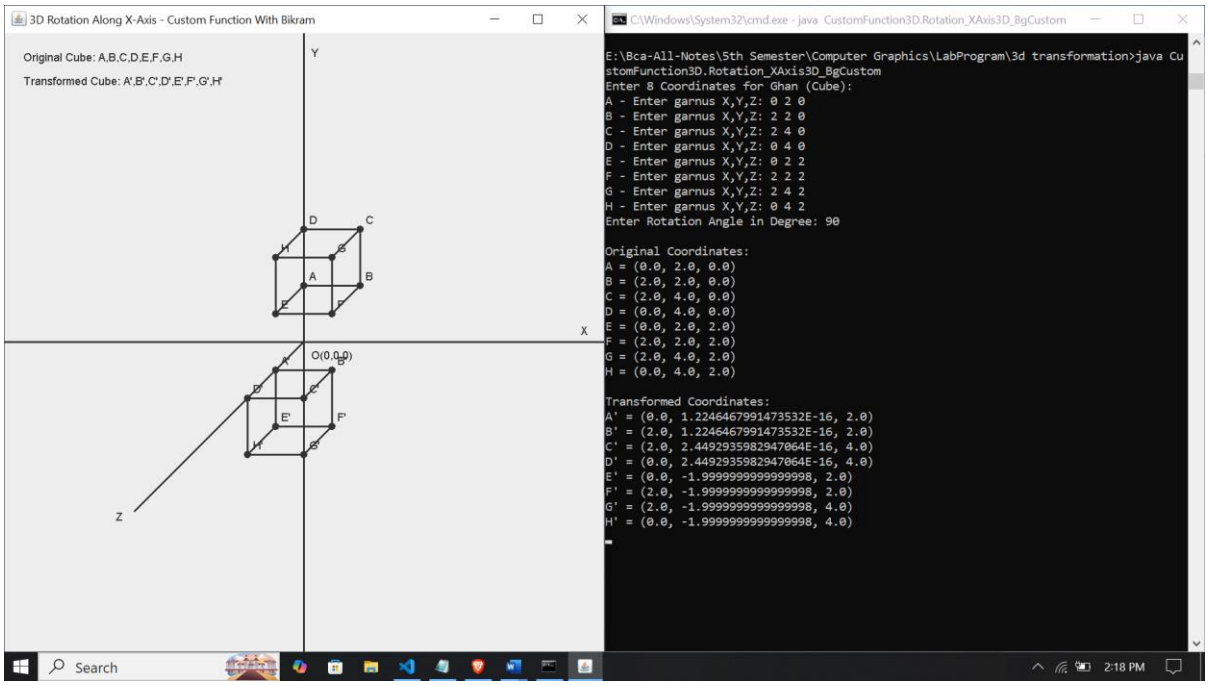
        public static void main(String[] args) {

                new Rotation_XAxis3D_BgCustom().BGHehe();

        }

}
```

**Output:**



## 10.3D Rotation through Y-Axis through Custom Function.

**Code:**

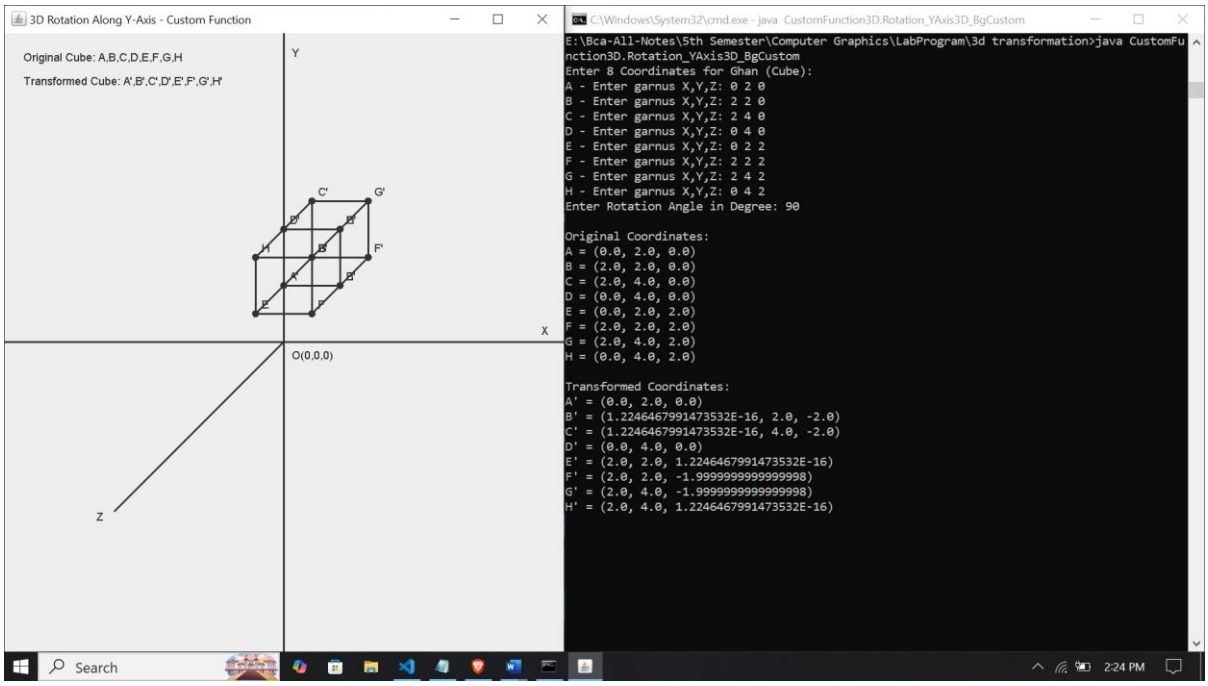
```
package CustomFunction3D;
import package3D.BGhelperPackage3D;
import java.util.Scanner;
public class Rotation_YAxis3D_BgCustom {
	public double[][] BGRotationY(double[][] BGOriginal, double BGAngle) {
		double[][] BGTransformed = new double[8][3];
		double BGRadian = Math.toRadians(BGAngle);
		for (int Bikram = 0; Bikram < 8; Bikram++) {
			double X = BGOriginal[Bikram][0];
			double Y = BGOriginal[Bikram][1];
			double Z = BGOriginal[Bikram][2];
			BGTransformed[Bikram][0] = X * Math.cos(BGRadian) + Z * Math.sin(BGRadian);
			BGTransformed[Bikram][1] = Y;
			BGTransformed[Bikram][2] = -X * Math.sin(BGRadian) + Z * Math.cos(BGRadian);
		}
		return BGTransformed;
	}

	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();
		bg.inputCoordinate(sa);
		System.out.print("Enter Rotation Angle in Degree: ");
		double BGAngle = sa.nextDouble();
		double[][] BGOriginal = bg.getOriginalPoint();
		double[][] BGTransformed = BGRotationY(BGOriginal, BGAngle);
```

```
        bg.showOriginalCoordinate(BGOriginal);
        bg.showTransformedCoordinate(BGTransformed);

        BGhelperPackage3D.CustomPanel3D AshliPanel = new BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);
        BGhelperPackage3D.createFrame("3D Rotation Along Y-Axis - Custom Function With Bikram", AshliPanel);
    }
    public static void main(String[] args) {
        new Rotation_YAxis3D_BgCustom().BGHehe();
    }
}
```

**Output:**



## 11.3D Rotation through Z-Axis using Custom Function.

**Code:**

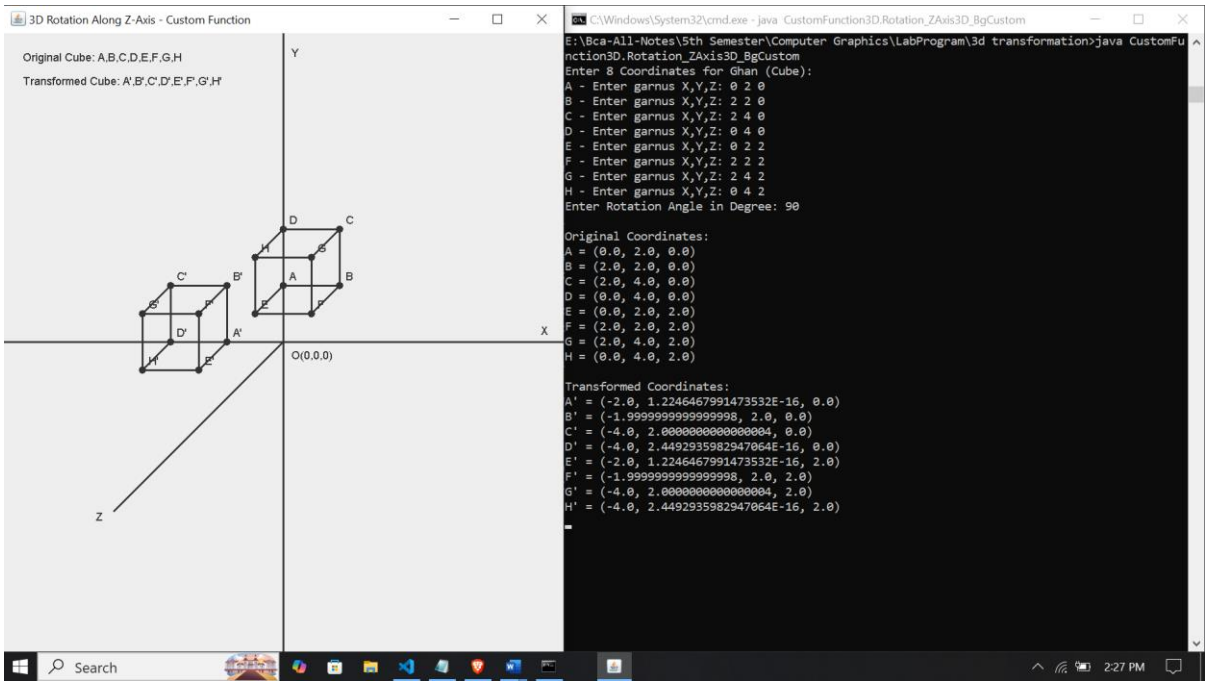
```
package CustomFunction3D;
import package3D.BGhelperPackage3D;
import java.util.Scanner;
public class Rotation_ZAxis3D_BgCustom {
	public double[][] BGRotationZ(double[][] BGOriginal, double BGAngle) {
		double[][] BGTransformed = new double[8][3];
		double BGRadian = Math.toRadians(BGAngle);

		for (int Bikram = 0; Bikram < 8; Bikram++) {
			double X = BGOriginal[Bikram][0];
			double Y = BGOriginal[Bikram][1];
			double Z = BGOriginal[Bikram][2];
			BGTransformed[Bikram][0] = X * Math.cos(BGRadian) - Y *
Math.sin(BGRadian);
			BGTransformed[Bikram][1] = X * Math.sin(BGRadian) + Y *
Math.cos(BGRadian);
			BGTransformed[Bikram][2] = Z;
		}
		return BGTransformed;
	}
	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();
		bg.inputCoordinate(sa);
		System.out.print("Enter Rotation Angle in Degree: ");
		double BGAngle = sa.nextDouble();
		double[][] BGOriginal = bg.getOriginalPoint();
		double[][] BGTransformed = BGRotationZ(BGOriginal, BGAngle);
```

```
                bg.showOriginalCoordinate(BGOriginal);
                bg.showTransformedCoordinate(BGTransformed);


                BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);
                BGhelperPackage3D.createFrame("3D Rotation Along Z-Axis - Custom
Function", AshliPanel);
        }
        public static void main(String[] args) {
                new Rotation_ZAxis3D_BgCustom().BGHehe();
        }
}
```

**Output:**



## 12.3D Rotation through X-Axis using Library Function.

**Code:**

```
package LibraryFunction3D;
import package3D.BGhelperPackage3D;
import package3D.BGhelper3DLibrary;
import java.util.Scanner;

public class Rotation_XAxis3D_BgLibrary {
	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();

		bg.inputCoordinate(sa);

		System.out.print("Enter Rotation Angle in Degree: ");
		double BGAngle = sa.nextDouble();

		double[][] BGOriginal = bg.getOriginalPoint();

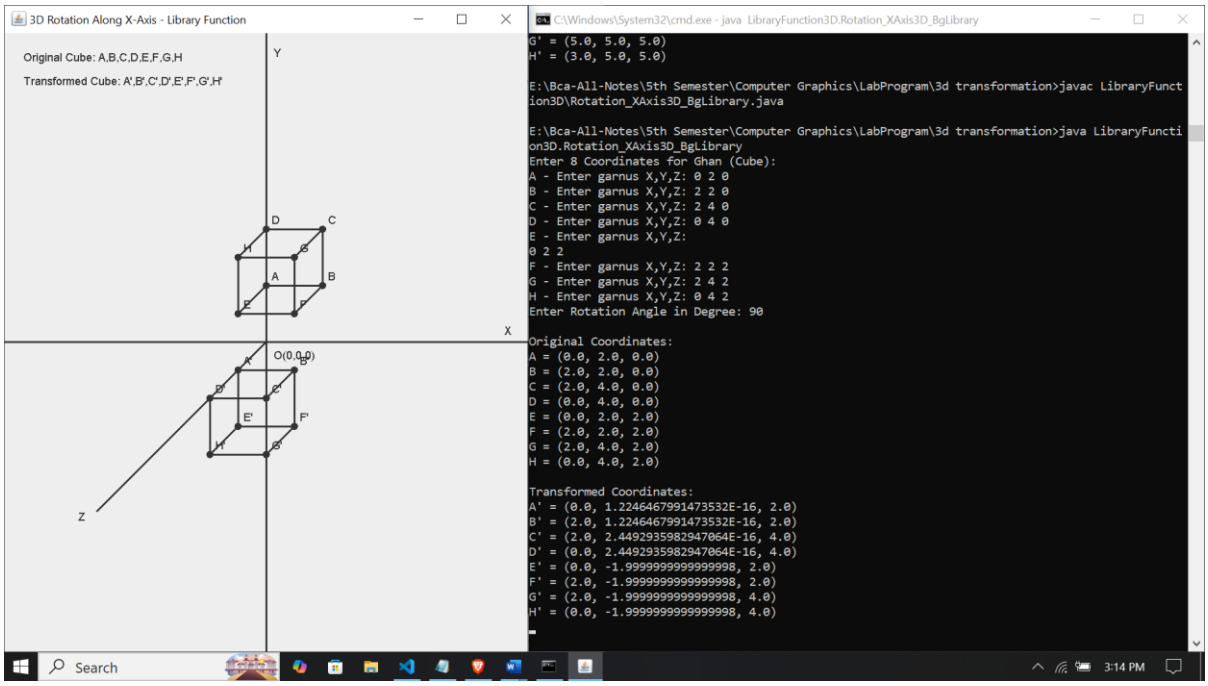
		double[][] BGMatrix =
BGhelper3DLibrary.BGRotationXMatrix(BGAngle);

		double[][] BGTransformed =
BGhelper3DLibrary.BGApplyMatrix(BGOriginal, BGMatrix);

		bg.showOriginalCoordinate(BGOriginal);
		bg.showTransformedCoordinate(BGTransformed);
		BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);
```

```
                BGhelperPackage3D.createFrame("3D Rotation Along X-Axis - Library
Function ",AshliPanel);
        }
        public static void main(String[] args) {
                new Rotation_XAxis3D_BgLibrary().BGHehe();
        }
}
```

**Output:**



## 13.3D Rotation through Y-Axis using Library Function.

**Code:**

```
package LibraryFunction3D;
import package3D.BGhelperPackage3D;
import package3D.BGhelper3DLibrary;
import java.util.Scanner;

public class Rotation_YAxis3D_BgLibrary {
	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();

		bg.inputCoordinate(sa);

		System.out.print("Enter Rotation Angle in Degree: ");
		double BGAngle = sa.nextDouble();

		double[][] BGOriginal = bg.getOriginalPoint();

		double[][] BGMatrix =
BGhelper3DLibrary.BGRotationYMatrix(BGAngle);

		double[][] BGTransformed =
BGhelper3DLibrary.BGApplyMatrix(BGOriginal, BGMatrix);

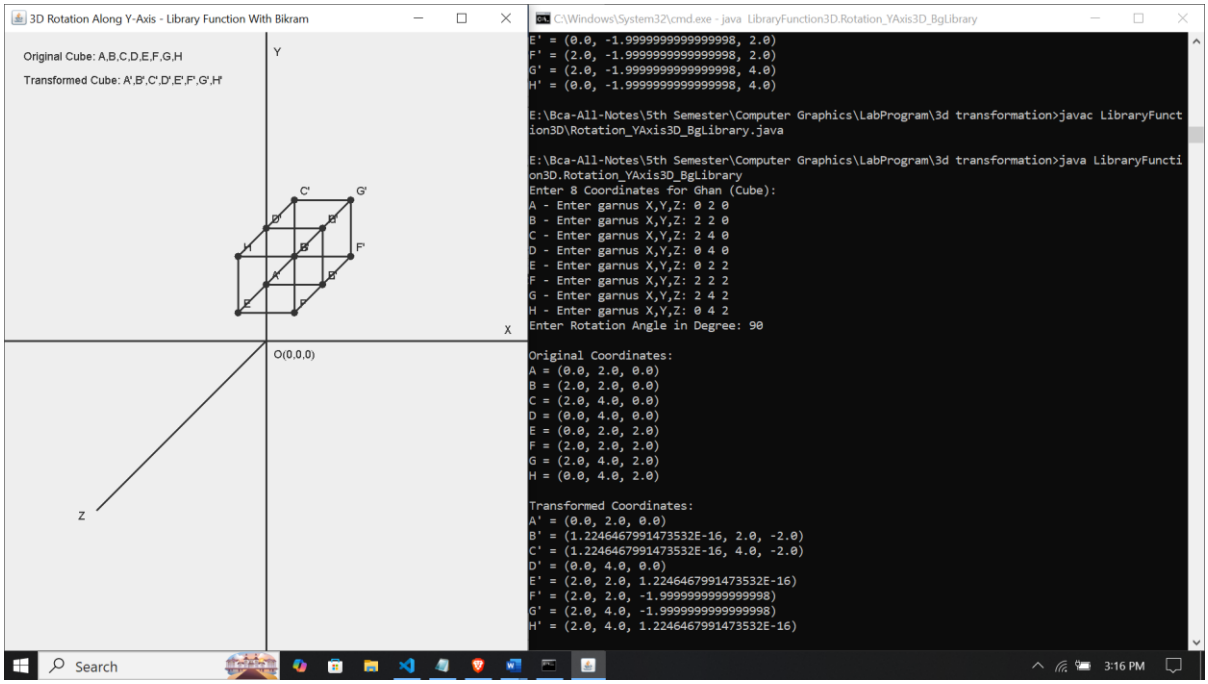
		bg.showOriginalCoordinate(BGOriginal);
		bg.showTransformedCoordinate(BGTransformed);

		BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);
```

```
                BGhelperPackage3D.createFrame("3D Rotation Along Y-Axis - Library
Function With Bikram",
                                AshliPanel);
        }

        public static void main(String[] args) {
                new Rotation_YAxis3D_BgLibrary().BGHehe();
        }
}
```

**Output:**



## 14.3D Rotation through Z-Axis using Library Function.

**Code:**

```
package LibraryFunction3D;
import package3D.BGhelperPackage3D;
import package3D.BGhelper3DLibrary;
import java.util.Scanner;

public class Rotation_ZAxis3D_BgLibrary {
	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();

		bg.inputCoordinate(sa);

		System.out.print("Enter Rotation Angle in Degree: ");
		double BGAngle = sa.nextDouble();

		double[][] BGOriginal = bg.getOriginalPoint();

		double[][] BGMatrix =
BGhelper3DLibrary.BGRotationZMatrix(BGAngle);

		double[][] BGTransformed =
BGhelper3DLibrary.BGApplyMatrix(BGOriginal, BGMatrix);

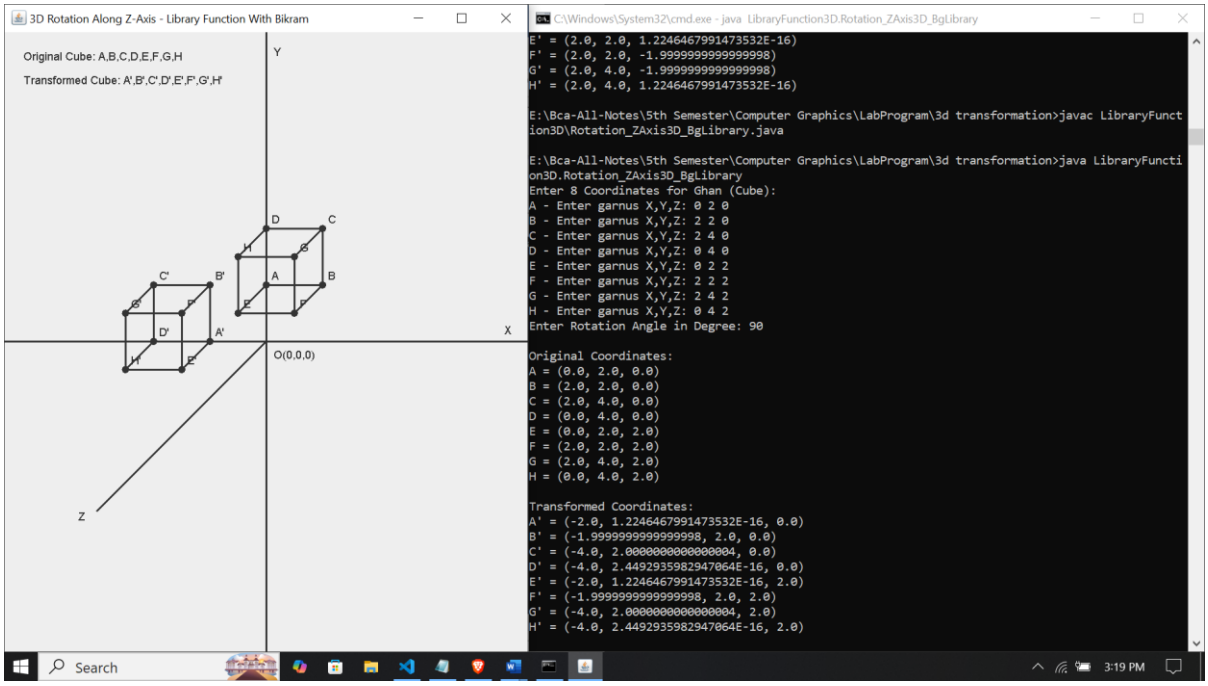
		bg.showOriginalCoordinate(BGOriginal);
		bg.showTransformedCoordinate(BGTransformed);

		BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);
```

```
                BGhelperPackage3D.createFrame("3D Rotation Along Z-Axis - Library
Function With Bikram", AshliPanel);
        }

        public static void main(String[] args) {
                new Rotation_ZAxis3D_BgLibrary().BGHehe();
        }
}
```

**Output:**



## 15.3D Reflection through ZX plane using Custom Function.

**Code:**

```
package CustomFunction3D;
import package3D.BGhelperPackage3D;
import java.util.Scanner;

public class Reflection_ZXPlane3D_BgCustom {
   public double[][] BGReflectionZX(double[][] BGOriginal) {
      double[][] BGTransformed=new double[8][3];

      for(int Bikram=0;Bikram<8;Bikram++) {
         BGTransformed[Bikram][0]=BGOriginal[Bikram][0];
         BGTransformed[Bikram][1]=-BGOriginal[Bikram][1];
         BGTransformed[Bikram][2]=BGOriginal[Bikram][2];
      }
      return BGTransformed;
   }

   public void BGHehe() {
      Scanner sa=new Scanner(System.in);
      BGhelperPackage3D bg=new BGhelperPackage3D();

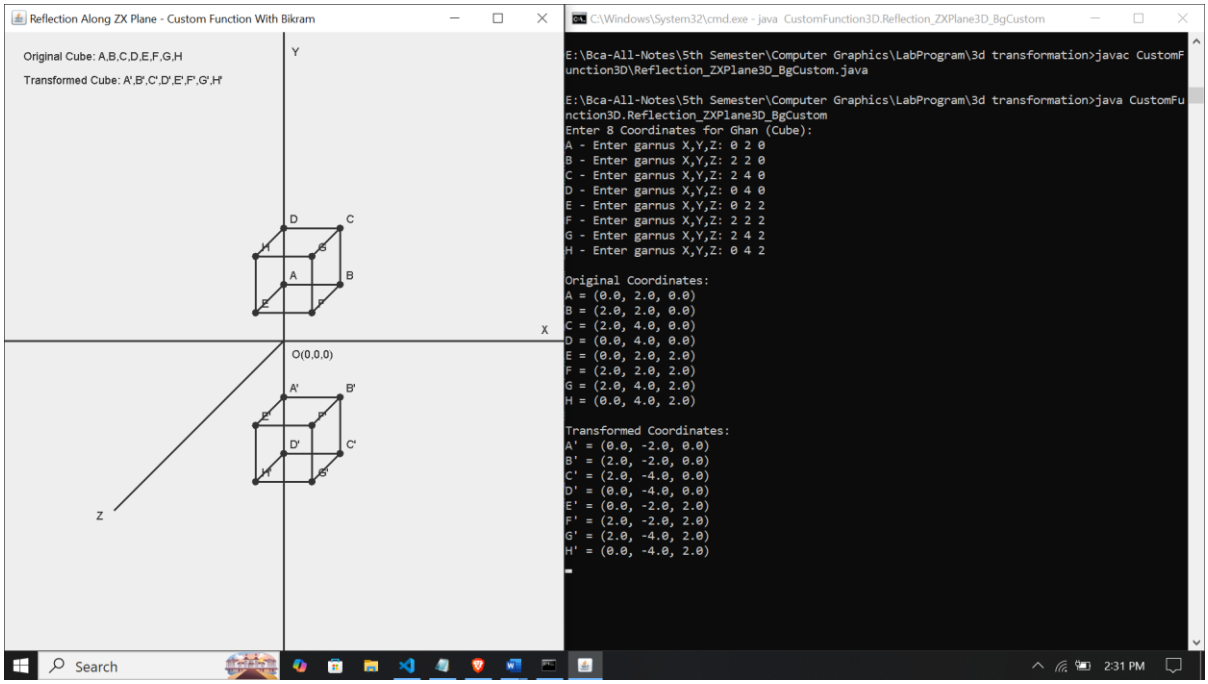
      bg.inputCoordinate(sa);

      double[][] BGOriginal=bg.getOriginalPoint();
      double[][] BGTransformed=BGReflectionZX(BGOriginal);

      bg.showOriginalCoordinate(BGOriginal);
      bg.showTransformedCoordinate(BGTransformed);
```

```
        BGhelperPackage3D.CustomPanel3D AshliPanel=
            new BGhelperPackage3D.CustomPanel3D(BGOriginal,BGTransformed);
        BGhelperPackage3D.createFrame("Reflection Along ZX Plane - Custom Function
With Bikram",AshliPanel);
    }
    public static void main(String[] args) {
        new Reflection_ZXPlane3D_BgCustom().BGHehe();
    }
}
```

**Output:**



## 16.3D Reflection using YZ plane using Custom Function.

**Code:**

```
package CustomFunction3D;
import package3D.BGhelperPackage3D;
import java.util.Scanner;
public class Reflection_YZPlane3D_BgCustom {
   public double[][] BGReflectionYZ(double[][] BGOriginal) {
      double[][] BGTransformed=new double[8][3];

      for(int Bikram=0;Bikram<8;Bikram++) {
         BGTransformed[Bikram][0]=-BGOriginal[Bikram][0];
         BGTransformed[Bikram][1]=BGOriginal[Bikram][1];
         BGTransformed[Bikram][2]=BGOriginal[Bikram][2];
      }
      return BGTransformed;
   }

   public void BGHehe() {
      Scanner sa=new Scanner(System.in);
      BGhelperPackage3D bg=new BGhelperPackage3D();

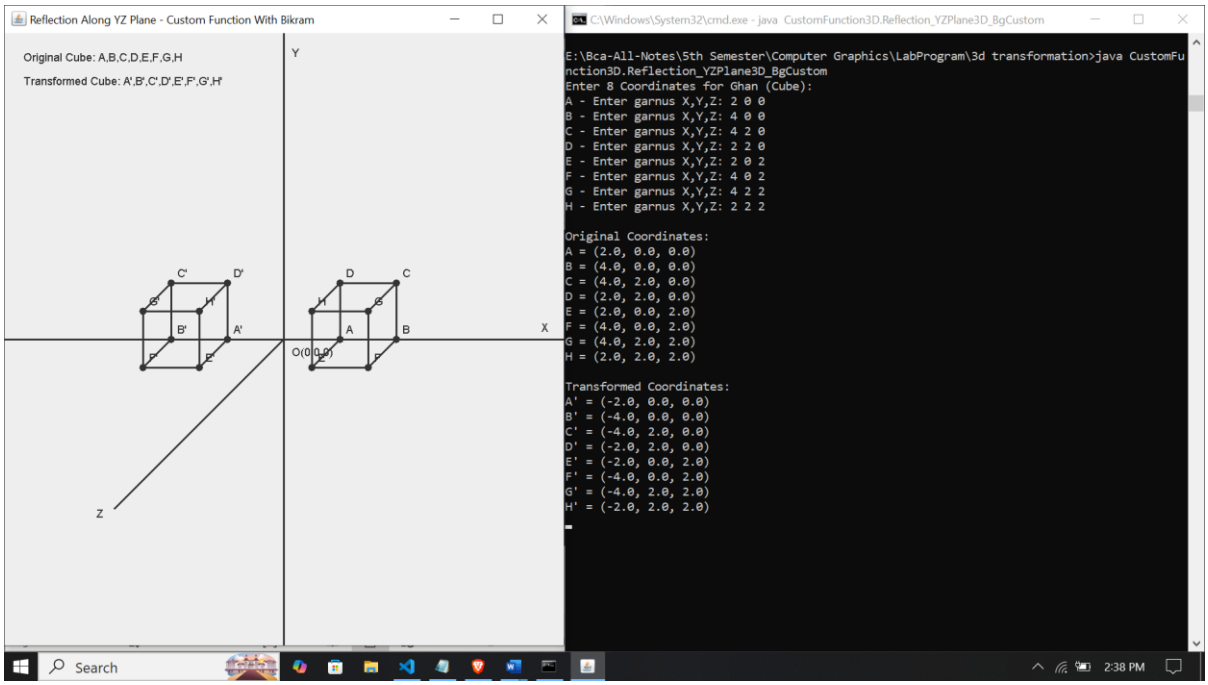
      bg.inputCoordinate(sa);
      double[][] BGOriginal=bg.getOriginalPoint();
      double[][] BGTransformed=BGReflectionYZ(BGOriginal);
      bg.showOriginalCoordinate(BGOriginal);
      bg.showTransformedCoordinate(BGTransformed);

      BGhelperPackage3D.CustomPanel3D AshliPanel=
         new BGhelperPackage3D.CustomPanel3D(BGOriginal,BGTransformed);
```

```
        BGhelperPackage3D.createFrame("Reflection Along YZ Plane - Custom Function
With Bikram",AshliPanel);
    }

    public static void main(String[] args) {
        new Reflection_YZPlane3D_BgCustom().BGHehe();
    }
}
```

**Output:**



## 17.3D Reflection through XY plane using Custom Function.

**Code:**

```
package CustomFunction3D;
import package3D.BGhelperPackage3D;
import java.util.Scanner;

public class Reflection_XYPlane3D_BgCustom {
	public double[][] BGReflectionXY(double[][] BGOriginal) {
		double[][] BGTransformed = new double[8][3];

		for (int Bikram = 0; Bikram < 8; Bikram++) {
			BGTransformed[Bikram][0] = BGOriginal[Bikram][0];
			BGTransformed[Bikram][1] = BGOriginal[Bikram][1];
			BGTransformed[Bikram][2] = -BGOriginal[Bikram][2];
		}
		return BGTransformed;
	}

	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();

		bg.inputCoordinate(sa);

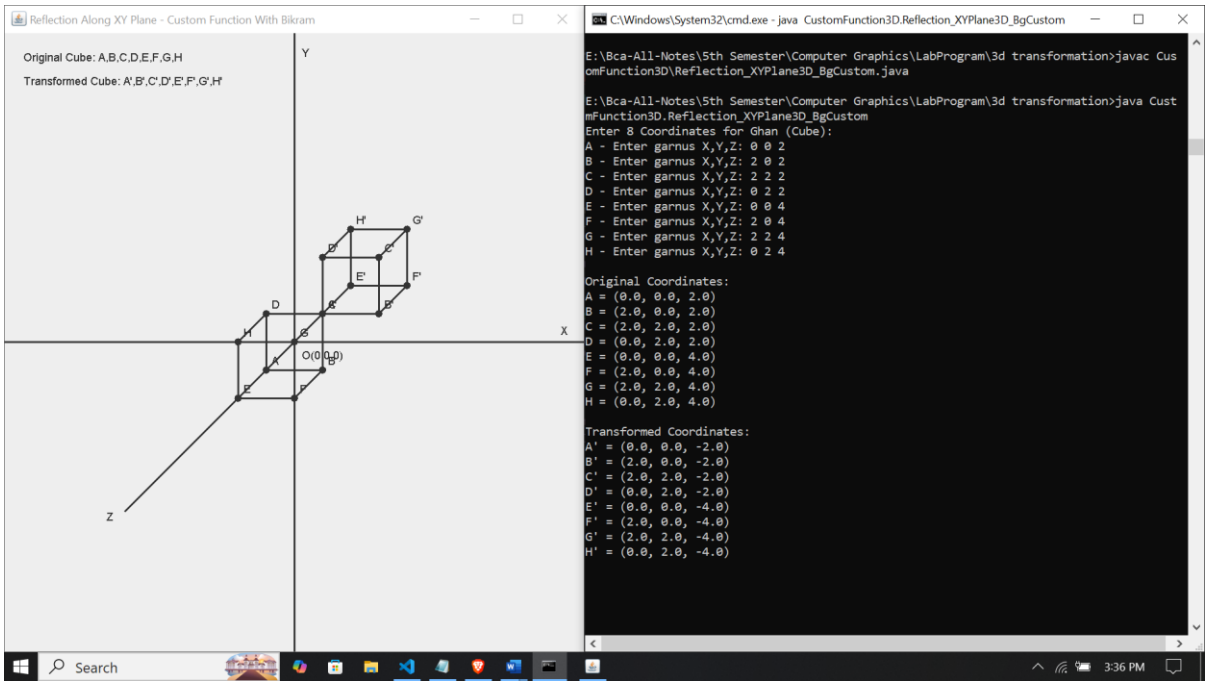
		double[][] BGOriginal = bg.getOriginalPoint();
		double[][] BGTransformed = BGReflectionXY(BGOriginal);

		bg.showOriginalCoordinate(BGOriginal);
		bg.showTransformedCoordinate(BGTransformed);
```

```
		BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);


		BGhelperPackage3D.createFrame("Reflection Along XY Plane - Custom
Function With Bikram", AshliPanel);
	}
	public static void main(String[] args) {
		new Reflection_XYPlane3D_BgCustom().BGHehe();
	}
}
```

**Output:**



## 18.3D Reflection through ZX plane using Library Function.

**Code:**

```
package LibraryFunction3D;
import package3D.BGhelperPackage3D;
import package3D.BGhelper3DLibrary;
import java.util.Scanner;

public class Reflection_ZXPlane3D_BgLibrary {
	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();

		bg.inputCoordinate(sa);

		double[][] BGOriginal = bg.getOriginalPoint();

		double[][] BGMatrix = BGhelper3DLibrary.BGReflectionZXMatrix();

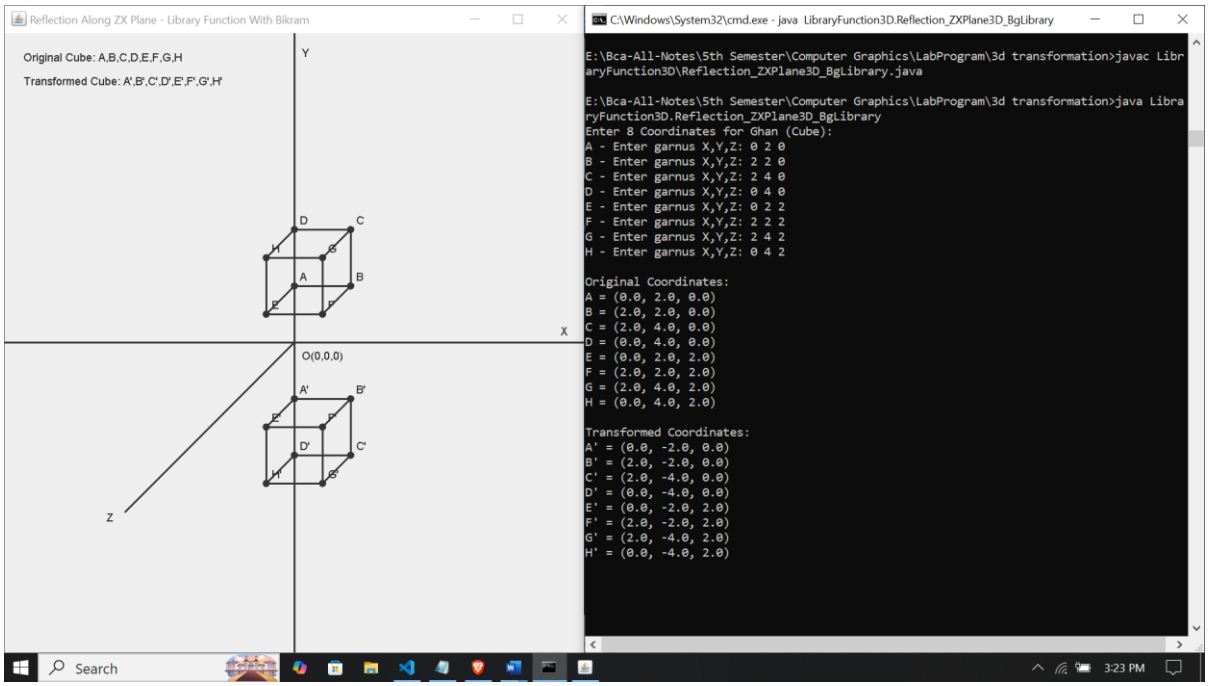
		double[][] BGTransformed =
BGhelper3DLibrary.BGApplyMatrix(BGOriginal, BGMatrix);

		bg.showOriginalCoordinate(BGOriginal);
		bg.showTransformedCoordinate(BGTransformed);

		BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);
BGhelperPackage3D.createFrame("Reflection Along ZX Plane - Library Function With
Bikram",AshliPanel);
	}
```

```
        public static void main(String[] args) {
                new Reflection_ZXPlane3D_BgLibrary().BGHehe();
        }
}
```

**Output:**



## 19.3D Reflection through YZ plane using Library Function.

**Code:**

```
package LibraryFunction3D;
import package3D.BGhelperPackage3D;
import package3D.BGhelper3DLibrary;
import java.util.Scanner;

public class Reflection_YZPlane3D_BgLibrary {
	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();

		bg.inputCoordinate(sa);

		double[][] BGOriginal = bg.getOriginalPoint();

		double[][] BGMatrix = BGhelper3DLibrary.BGReflectionYZMatrix();

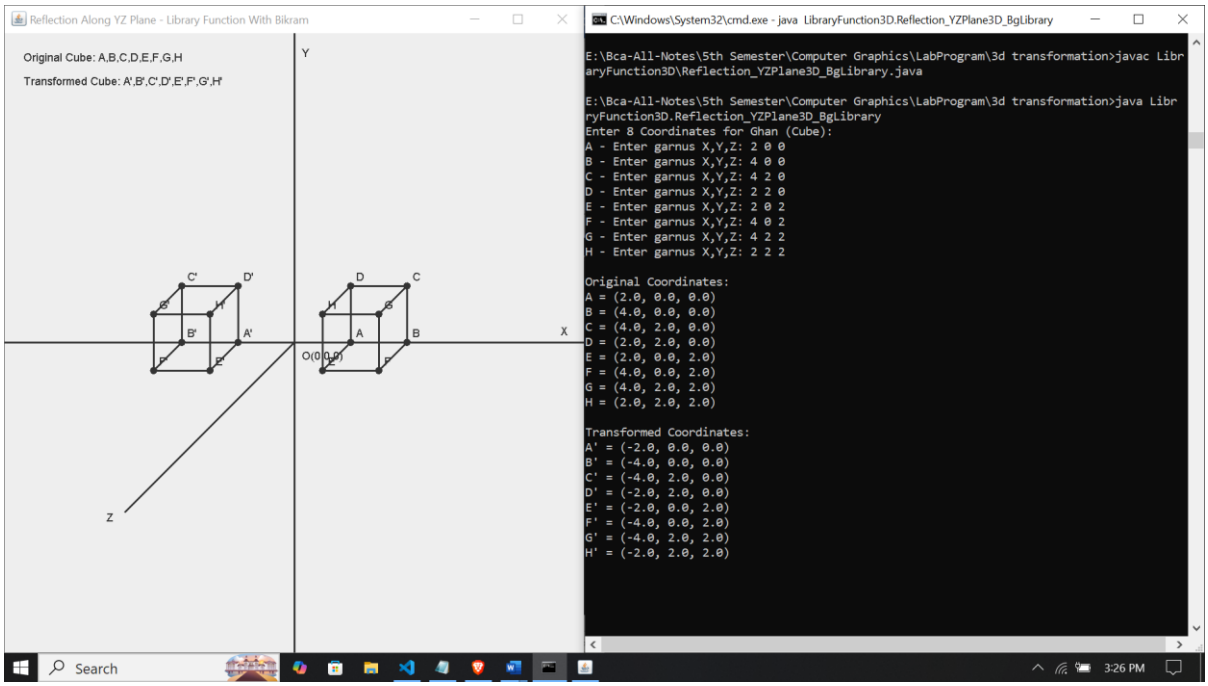
		double[][] BGTransformed =
BGhelper3DLibrary.BGApplyMatrix(BGOriginal, BGMatrix);

		bg.showOriginalCoordinate(BGOriginal);
		bg.showTransformedCoordinate(BGTransformed);
		BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);

		BGhelperPackage3D.createFrame("Reflection Along YZ Plane - Library
Function With Bikram", AshliPanel);
	}
```

```
        public static void main(String[] args) {
                new Reflection_YZPlane3D_BgLibrary().BGHehe();
        }
}
```

**Output:**



## 20.3D Reflection through XY plane using Library Function.

**Code:**

```
package LibraryFunction3D;
import package3D.BGhelperPackage3D;
import package3D.BGhelper3DLibrary;
import java.util.Scanner;

public class Reflection_XYPlane3D_BgLibrary {
	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();

		bg.inputCoordinate(sa);

		double[][] BGOriginal = bg.getOriginalPoint();

		double[][] BGMatrix = BGhelper3DLibrary.BGReflectionXYMatrix();

		double[][] BGTransformed =
BGhelper3DLibrary.BGApplyMatrix(BGOriginal, BGMatrix);

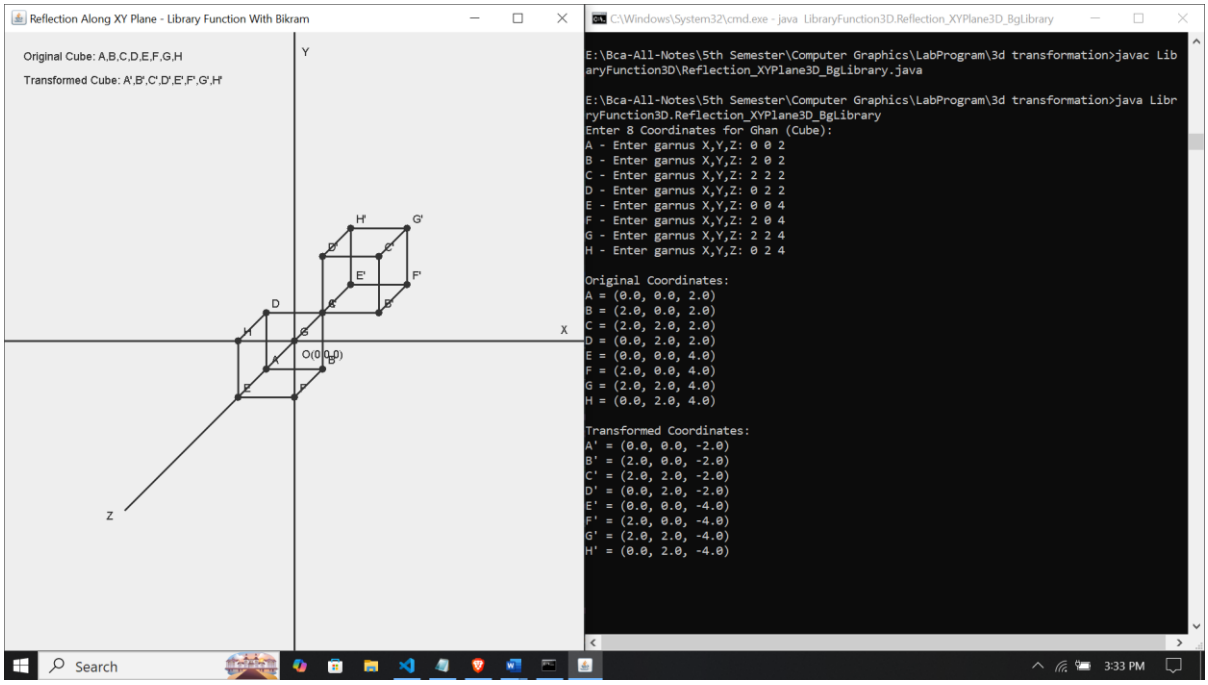
		bg.showOriginalCoordinate(BGOriginal);
		bg.showTransformedCoordinate(BGTransformed);

		BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);

		BGhelperPackage3D.createFrame("Reflection Along XY Plane - Library
Function With Bikram",AshliPanel);
	}
```

```
        public static void main(String[] args) {
                new Reflection_XYPlane3D_BgLibrary().BGHehe();
        }
}
```

**Output:**



## 21.3D Shearing along X-Axis using Custom Function.

**Code:**

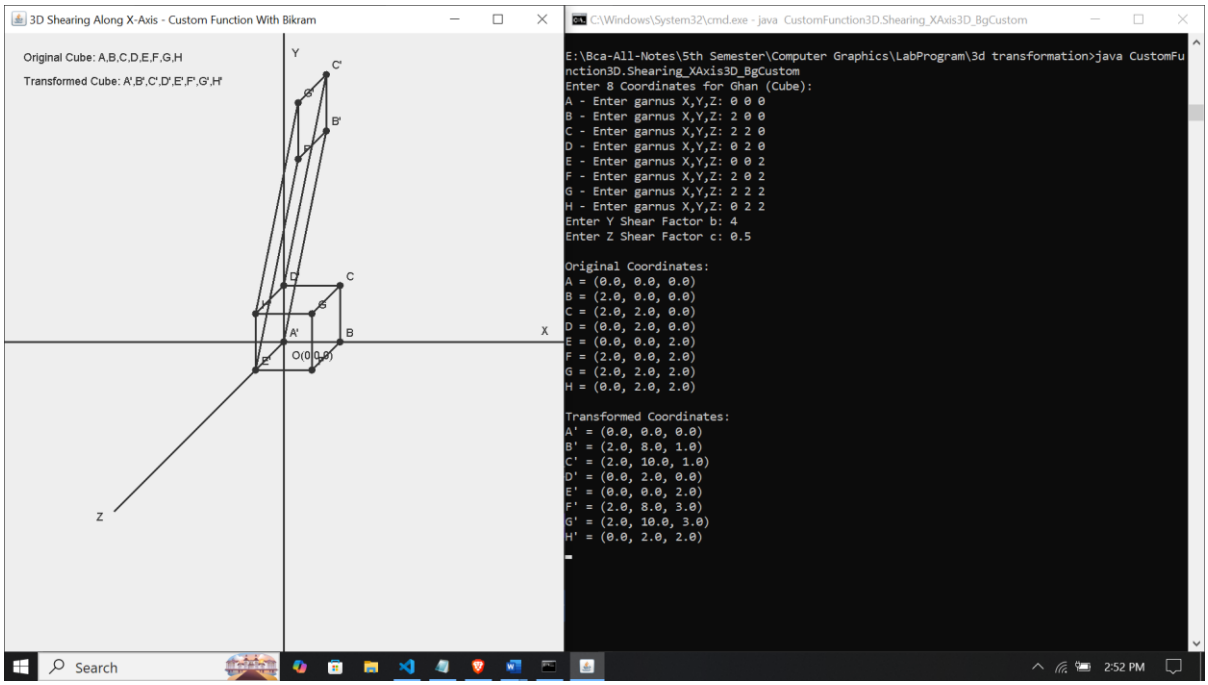
```
package CustomFunction3D;
import package3D.BGhelperPackage3D;
import java.util.Scanner;
public class Shearing_XAxis3D_BgCustom {
	public double[][] BGShearingX(double[][] BGOriginal, double b, double c) {
		double[][] BGTransformed = new double[8][3];

		for (int Bikram = 0; Bikram < 8; Bikram++) {
			double X = BGOriginal[Bikram][0];
			double Y = BGOriginal[Bikram][1];
			double Z = BGOriginal[Bikram][2];
			BGTransformed[Bikram][0] = X;
			BGTransformed[Bikram][1] = Y + b * X;
			BGTransformed[Bikram][2] = Z + c * X;
		}
		return BGTransformed;
	}

	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();
		bg.inputCoordinate(sa);
		System.out.print("Enter Y Shear Factor b: ");
		double b = sa.nextDouble();
		System.out.print("Enter Z Shear Factor c: ");
		double c = sa.nextDouble();
		double[][] BGOriginal = bg.getOriginalPoint();
```

```
                double[][] BGTransformed = BGShearingX(BGOriginal, b, c);
                bg.showOriginalCoordinate(BGOriginal);
                bg.showTransformedCoordinate(BGTransformed);
                BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);
                BGhelperPackage3D.createFrame("3D Shearing Along X-Axis - Custom
Function With Bikram", AshliPanel);
        }
        public static void main(String[] args) {
                new Shearing_XAxis3D_BgCustom().BGHehe();
        }
}
```

**Output:**



## 22.3D Shearing through Y-Axis using Custom Function.

**Code:**

```
package CustomFunction3D;
import package3D.BGhelperPackage3D;
import java.util.Scanner;
public class Shearing_YAxis3D_BgCustom {
	public double[][] BGShearingY(double[][] BGOriginal, double a, double c) {
		double[][] BGTransformed = new double[8][3];
		for (int Bikram = 0; Bikram < 8; Bikram++) {
			double X = BGOriginal[Bikram][0];
			double Y = BGOriginal[Bikram][1];
			double Z = BGOriginal[Bikram][2];
			BGTransformed[Bikram][0] = X + a * Y;
			BGTransformed[Bikram][1] = Y;
			BGTransformed[Bikram][2] = Z + c * Y;
		}
		return BGTransformed;
	}
	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();
		bg.inputCoordinate(sa);
		System.out.print("Enter X Shear Factor a: ");
		double a = sa.nextDouble();
		System.out.print("Enter Z Shear Factor c: ");
		double c = sa.nextDouble();
		double[][] BGOriginal = bg.getOriginalPoint();
		double[][] BGTransformed = BGShearingY(BGOriginal, a, c);
		bg.showOriginalCoordinate(BGOriginal);
```

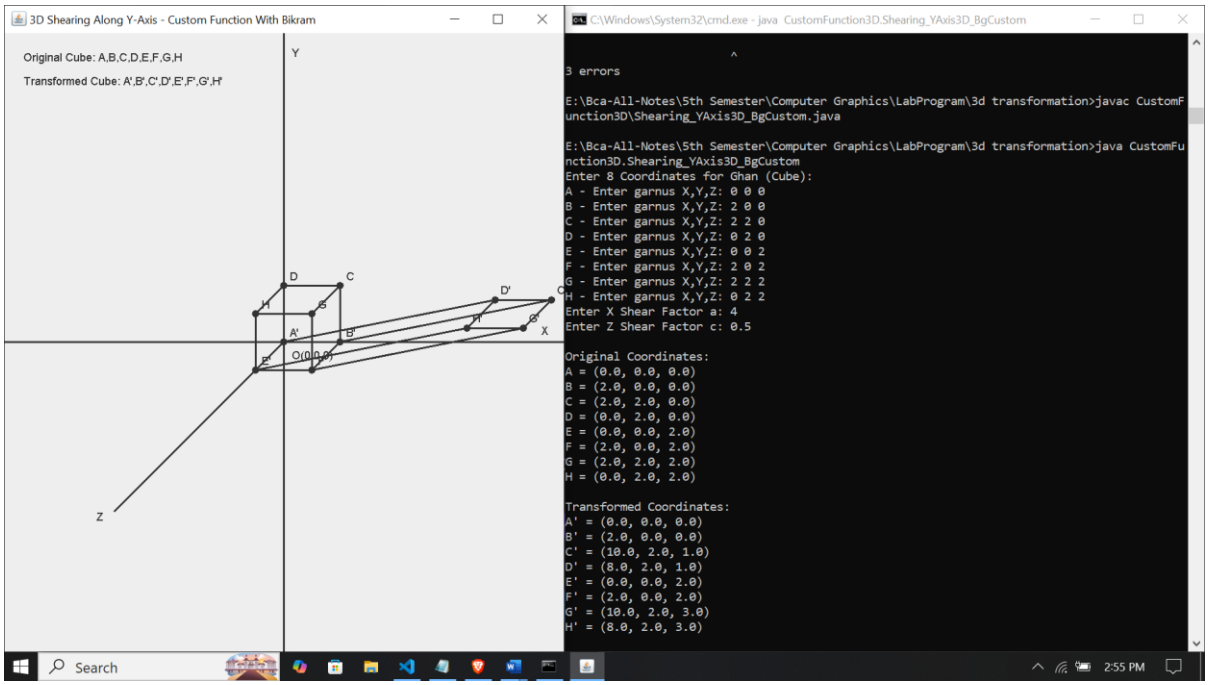
```
                bg.showTransformedCoordinate(BGTransformed);

                BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);

                BGhelperPackage3D.createFrame("3D Shearing Along Y-Axis - Custom
Function With Bikram", AshliPanel);
        }


        public static void main(String[] args) {
                new Shearing_YAxis3D_BgCustom().BGHehe();
        }
}
```

**Output:**



## 23.3D Shearing along Z-Axis using Custom Function.

**Code:**

```
package CustomFunction3D;
import package3D.BGhelperPackage3D;
import java.util.Scanner;
public class Shearing_ZAxis3D_BgCustom {
	public double[][] BGShearingZ(double[][] BGOriginal, double a, double b) {
		double[][] BGTransformed = new double[8][3];
		for (int Bikram = 0; Bikram < 8; Bikram++) {
			double X = BGOriginal[Bikram][0];
			double Y = BGOriginal[Bikram][1];
			double Z = BGOriginal[Bikram][2];
			BGTransformed[Bikram][0] = X + a * Z;
			BGTransformed[Bikram][1] = Y + b * Z;
			BGTransformed[Bikram][2] = Z;
		}
		return BGTransformed;
	}

	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();
		bg.inputCoordinate(sa);
		System.out.print("Enter X Shear Factor a: ");
		double a = sa.nextDouble();
		System.out.print("Enter Y Shear Factor b: ");
		double b = sa.nextDouble();
		double[][] BGOriginal = bg.getOriginalPoint();
		double[][] BGTransformed = BGShearingZ(BGOriginal, a, b);
```

```
                bg.showOriginalCoordinate(BGOriginal);

                bg.showTransformedCoordinate(BGTransformed);

                BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);

                BGhelperPackage3D.createFrame("3D Shearing Along Z-Axis - Custom
Function", AshliPanel);

        }

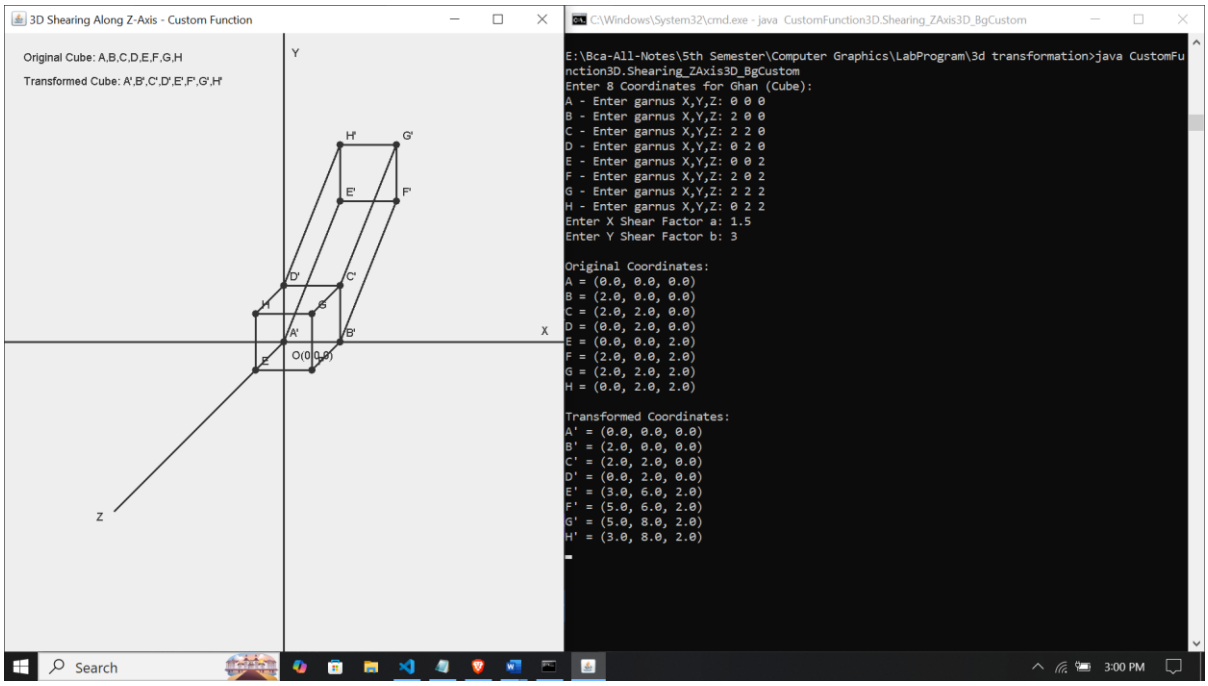

        public static void main(String[] args) {

                new Shearing_ZAxis3D_BgCustom().BGHehe();

        }

}
```

**Output:**



## 24. 3D Shearing along X-Axis using Library Function.

**Code:**

```
package LibraryFunction3D;
import package3D.BGhelperPackage3D;
import package3D.BGhelper3DLibrary;
import java.util.Scanner;

public class Shearing_XAxis3D_BgLibrary {
	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();
		bg.inputCoordinate(sa);

		System.out.print("Enter Y Shear Factor b: ");
		double b = sa.nextDouble();

		System.out.print("Enter Z Shear Factor c: ");
		double c = sa.nextDouble();

		double[][] BGOriginal = bg.getOriginalPoint();

		double[][] BGMatrix = BGhelper3DLibrary.BGShearingXMatrix(b, c);

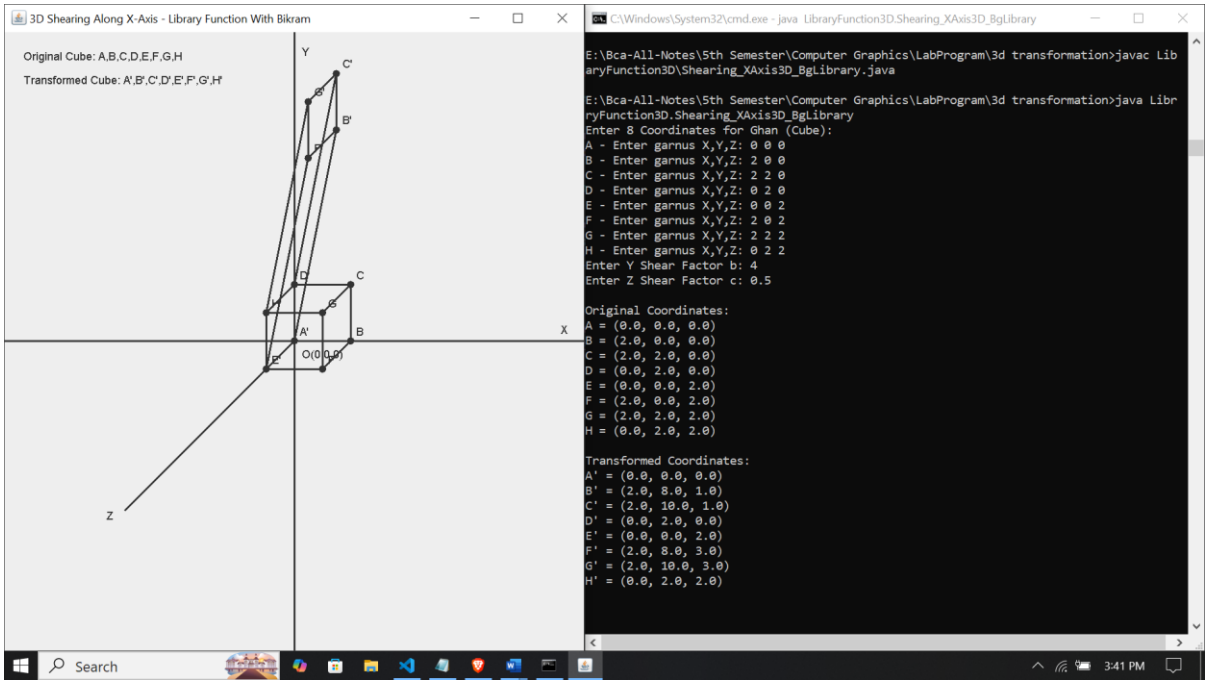
		double[][] BGTransformed =
BGhelper3DLibrary.BGApplyMatrix(BGOriginal, BGMatrix);
		bg.showOriginalCoordinate(BGOriginal);
		bg.showTransformedCoordinate(BGTransformed);
```

```
        BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);


        BGhelperPackage3D.createFrame("3D Shearing Along X-Axis - Library
Function With Bikram", AshliPanel);
    }


    public static void main(String[] args) {
        new Shearing_XAxis3D_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 25.3D Shearing along Y-Axis using Library Function.

**Code:**

```
package LibraryFunction3D;
import package3D.BGhelperPackage3D;
import package3D.BGhelper3DLibrary;
import java.util.Scanner;

public class Shearing_YAxis3D_BgLibrary {
	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();

		bg.inputCoordinate(sa);

		System.out.print("Enter X Shear Factor a: ");
		double a = sa.nextDouble();

		System.out.print("Enter Z Shear Factor c: ");
		double c = sa.nextDouble();

		double[][] BGOriginal = bg.getOriginalPoint();

		double[][] BGMatrix = BGhelper3DLibrary.BGShearingYMatrix(a, c);

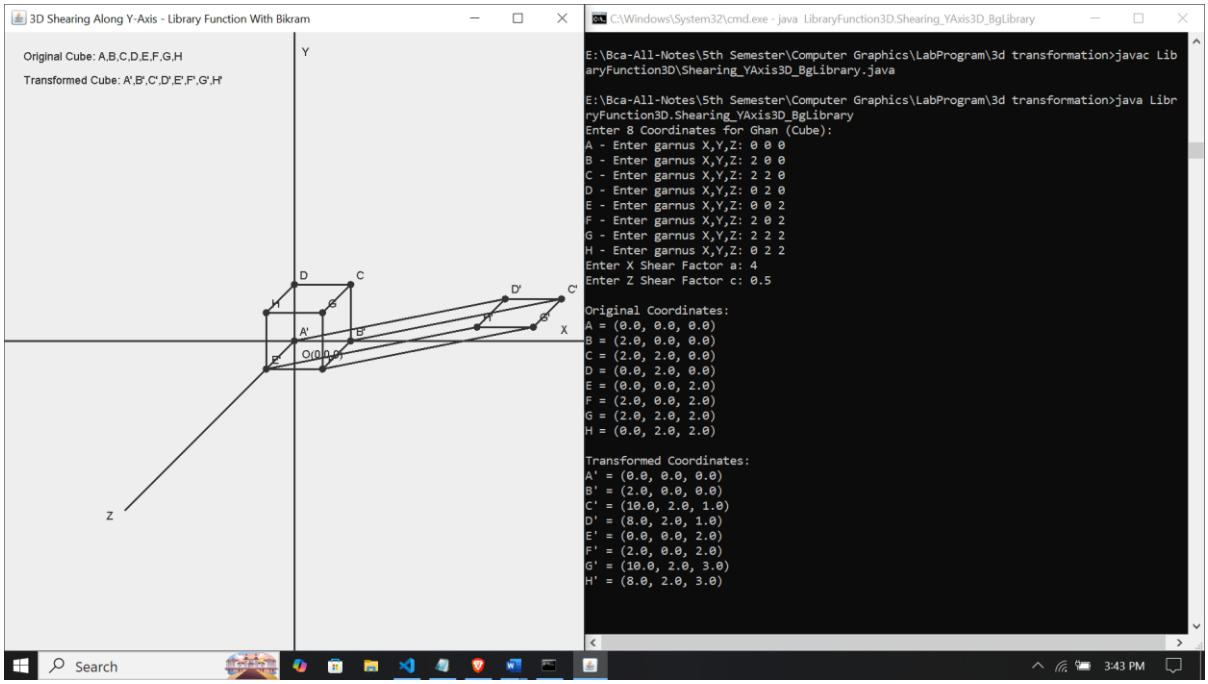
		double[][] BGTransformed =
BGhelper3DLibrary.BGApplyMatrix(BGOriginal, BGMatrix);
		bg.showOriginalCoordinate(BGOriginal);
		bg.showTransformedCoordinate(BGTransformed);
```

```
        BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);


        BGhelperPackage3D.createFrame("3D Shearing Along Y-Axis - Library
Function With Bikram", AshliPanel);
    }


    public static void main(String[] args) {
        new Shearing_YAxis3D_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 26.3D Shearing along Z-Axis using Library Function.

**Code:**

```
package LibraryFunction3D;
import package3D.BGhelperPackage3D;
import package3D.BGhelper3DLibrary;
import java.util.Scanner;

public class Shearing_ZAxis3D_BgLibrary {
	public void BGHehe() {
		Scanner sa = new Scanner(System.in);
		BGhelperPackage3D bg = new BGhelperPackage3D();

		bg.inputCoordinate(sa);

		System.out.print("Enter X Shear Factor a: ");
		double a = sa.nextDouble();

		System.out.print("Enter Y Shear Factor b: ");
		double b = sa.nextDouble();

		double[][] BGOriginal = bg.getOriginalPoint();

		double[][] BGMatrix = BGhelper3DLibrary.BGShearingZMatrix(a, b);

		double[][] BGTransformed =
BGhelper3DLibrary.BGApplyMatrix(BGOriginal, BGMatrix);

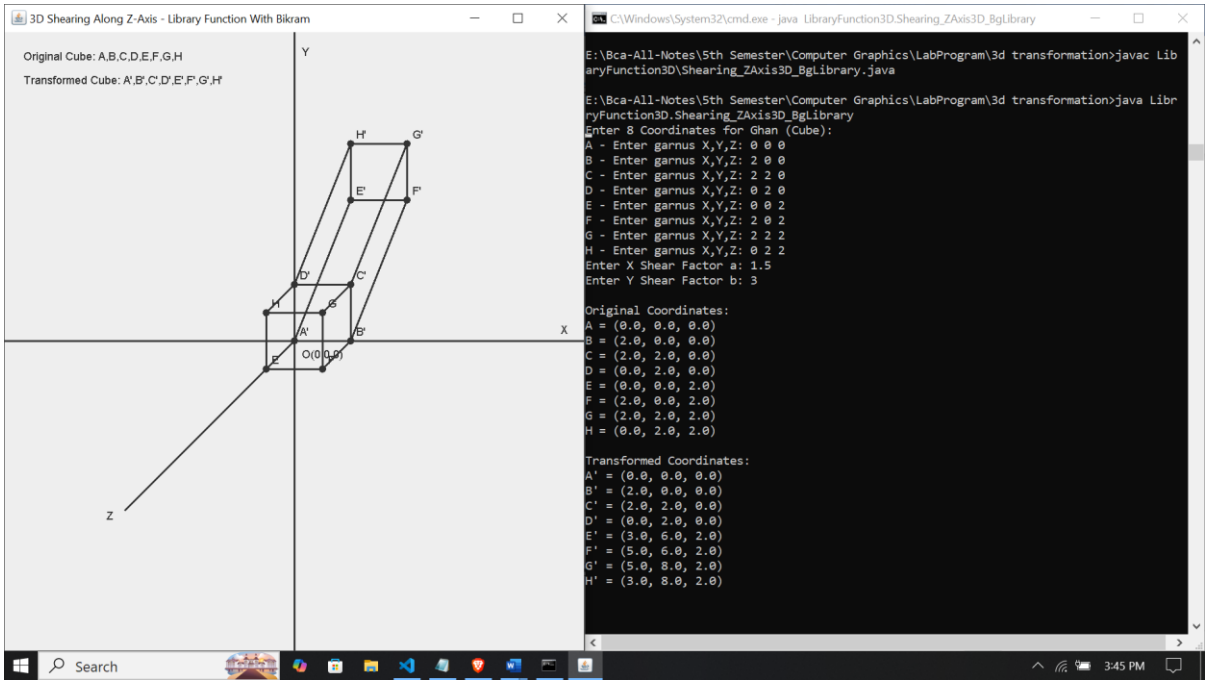
		bg.showOriginalCoordinate(BGOriginal);
		bg.showTransformedCoordinate(BGTransformed);
```

```
        BGhelperPackage3D.CustomPanel3D AshliPanel = new
BGhelperPackage3D.CustomPanel3D(BGOriginal, BGTransformed);


        BGhelperPackage3D.createFrame("3D Shearing Along Z-Axis - Library
Function With Bikram", AshliPanel);
    }


    public static void main(String[] args) {
        new Shearing_ZAxis3D_BgLibrary().BGHehe();
    }
}
```

**Output:**



# III. Line Drawing Algorithm

## 1. Program to create Package for input and swing.

**Code:**

```
package packageLine;
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;

public class BGhelperPackageLine {
   public int Xa,Ya,Xb,Yb;
   public void inputCoordinate(Scanner sa) {
      System.out.println("Enter Start and End Coordinates:");
      System.out.print("Suru X Coordinate enter garnus hos: ");
      Xa=sa.nextInt();
      System.out.print("Suru Y Coordinate enter garnus hos: ");
      Ya=sa.nextInt();
      System.out.print("Antim X Coordinate enter garnus hos: ");
      Xb=sa.nextInt();
      System.out.print("Antim Y Coordinate enter garnus hos: ");
      Yb=sa.nextInt();
   }

   public void showCoordinate() {
      System.out.println("\nInput Coordinates:");
      System.out.println("Suru = ("+Xa+","+Ya+")");
      System.out.println("Antim = ("+Xb+","+Yb+")");
   }
   public static void createFrame(String BGTitle,JPanel AshliPanel) {
```

```
    JFrame BGF=new JFrame(BGTitle);
    BGF.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    BGF.setSize(750,550);
    BGF.setLocationRelativeTo(null);
    BGF.add(AshliPanel);
    BGF.setVisible(true);
}

public static class PointPanel extends JPanel {
    private int Xa,Ya,Xb,Yb;
    private int[][] BGPoints;
    private boolean BGLibrary;
    public PointPanel(int Xa,int Ya,int Xb,int Yb,int[][] BGPoints) {
        this.Xa=Xa;
        this.Ya=Ya;
        this.Xb=Xb;
        this.Yb=Yb;
        this.BGPoints=BGPoints;
        this.BGLibrary=false;
    }
    public PointPanel(int Xa,int Ya,int Xb,int Yb) {
        this.Xa=Xa;
        this.Ya=Ya;
        this.Xb=Xb;
        this.Yb=Yb;
        this.BGLibrary=true;
    }
    private int BGScreenX(int XCoordinate) {
        return getWidth()/2+XCoordinate*20;
    }
```

```
    private int BGScreenY(int YCoordinate) {
        return getHeight()/2-YCoordinate*20;
    }
    private void BGDrawAxes(Graphics2D g2) {
        int centerX=getWidth()/2;
        int centerY=getHeight()/2;
        g2.drawLine(0,centerY,getWidth(),centerY);
        g2.drawLine(centerX,0,centerX,getHeight());
        g2.drawString("X-Axis",getWidth()-60,centerY-8);
        g2.drawString("Y-Axis",centerX+8,25);
        g2.drawString("O(0,0)",centerX+8,centerY+18);
    }

    private void BGDrawCalculatedLine(Graphics2D g2) {
  int X1=BGScreenX(Xa);
  int Y1=BGScreenY(Ya);
  int X2=BGScreenX(Xb);
  int Y2=BGScreenY(Yb);
  g2.drawLine(X1,Y1,X2,Y2);
}

    private void BGDrawLibraryLine(Graphics2D g2) {
        int X1=BGScreenX(Xa);
        int Y1=BGScreenY(Ya);
        int X2=BGScreenX(Xb);
        int Y2=BGScreenY(Yb);
        g2.drawLine(X1,Y1,X2,Y2);
    }

    @Override
```

```
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            Graphics2D g2=(Graphics2D)g;
            g2.setStroke(new BasicStroke(2));
            BGDrawAxes(g2);
            if(BGLibrary) {
               BGDrawLibraryLine(g2);
            } else {
               BGDrawCalculatedLine(g2);
            }

            int SX=BGScreenX(Xa);
            int SY=BGScreenY(Ya);
            int EX=BGScreenX(Xb);
            int EY=BGScreenY(Yb);

            g2.drawString("("+Xa+","+Ya+")",SX+8,SY+20);
            g2.drawString("("+Xb+","+Yb+")",EX+8,EY-10);
        }
    }
}
```

## 2. DDA Algorithm using Custom Function

**Code:**

```
package CustomFunction;
import packageLine.BGhelperPackageLine;
import java.util.Scanner;

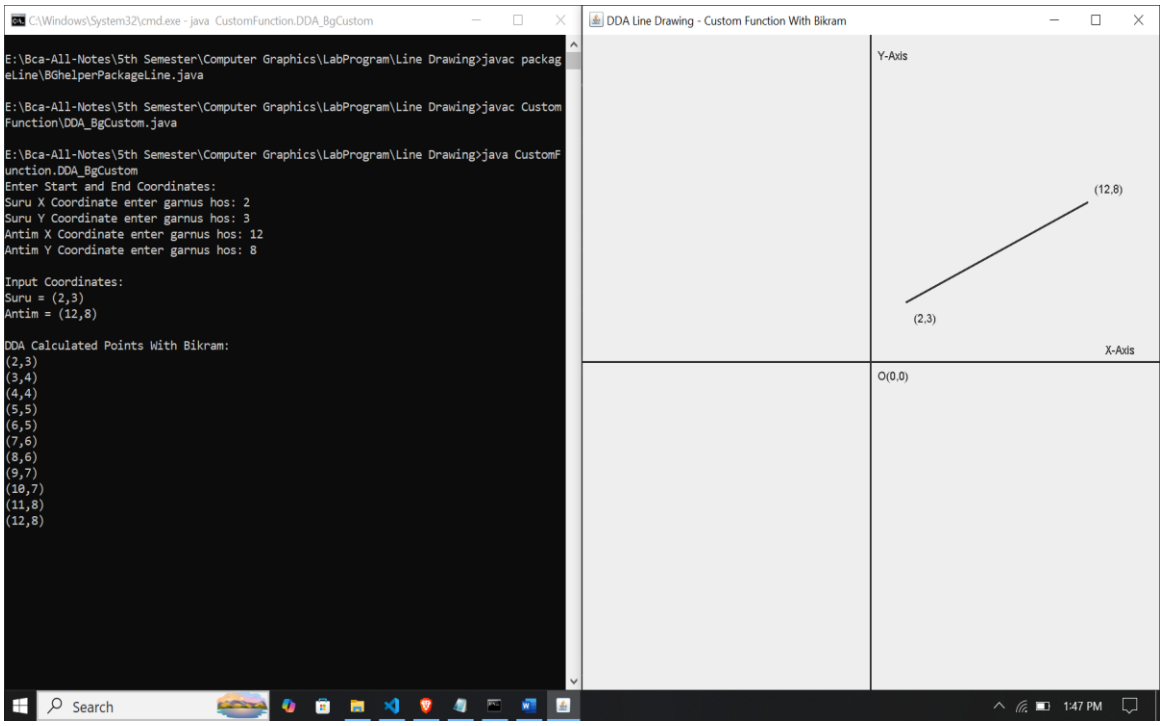
public class DDA_BgCustom {
    public int[][] BGDDA(int Xa, int Ya, int Xb, int Yb) {
        int delx = Xb - Xa;
        int dely = Yb - Ya;
        int steps = Math.max(Math.abs(delx), Math.abs(dely));
        double XIncrement = (double) delx / steps;
        double YIncrement = (double) dely / steps;
        double XCurrent = Xa;
        double YCurrent = Ya;
        int[][] BGPoints = new int[steps + 1][2];

        System.out.println("\nDDA Calculated Points With Bikram:");
        for (int Bikram = 0; Bikram <= steps; Bikram++) {
            BGPoints[Bikram][0] = (int) Math.round(XCurrent);
            BGPoints[Bikram][1] = (int) Math.round(YCurrent);
            System.out.println("(" + BGPoints[Bikram][0] + "," + BGPoints[Bikram][1] +
")");
            XCurrent += XIncrement;
            YCurrent += YIncrement;
        }
        return BGPoints;
    }
    public void BGHehe() {
```

```
        Scanner sa = new Scanner(System.in);
        BGhelperPackageLine bg = new BGhelperPackageLine();
        bg.inputCoordinate(sa);
        bg.showCoordinate();
        int[][] BGPoints = BGDDA(bg.Xa, bg.Ya, bg.Xb, bg.Yb);
        BGhelperPackageLine.PointPanel AshliPanel = new
BGhelperPackageLine.PointPanel(bg.Xa, bg.Ya, bg.Xb, bg.Yb,
                BGPoints);
        BGhelperPackageLine.createFrame("DDA Line Drawing - Custom Function With
Bikram", AshliPanel);
    }

    public static void main(String[] args) {
        new DDA_BgCustom().BGHehe();
    }
}
```

**Output:**



## 3. DDA Algorithm using Library Function

**Code:**

```
package LibraryFunction;

import packageLine.BGhelperPackageLine;

import java.util.Scanner;

public class DDA_BgLibrary {

   public void BGHehe() {

      Scanner sa = new Scanner(System.in);

      BGhelperPackageLine bg = new BGhelperPackageLine();

      bg.inputCoordinate(sa);

      bg.showCoordinate();

      System.out.println("\nLibrary Line with Bikram:");

      System.out.println("Suru = (" + bg.Xa + "," + bg.Ya + ")");

      System.out.println("Antim = (" + bg.Xb + "," + bg.Yb + ")");

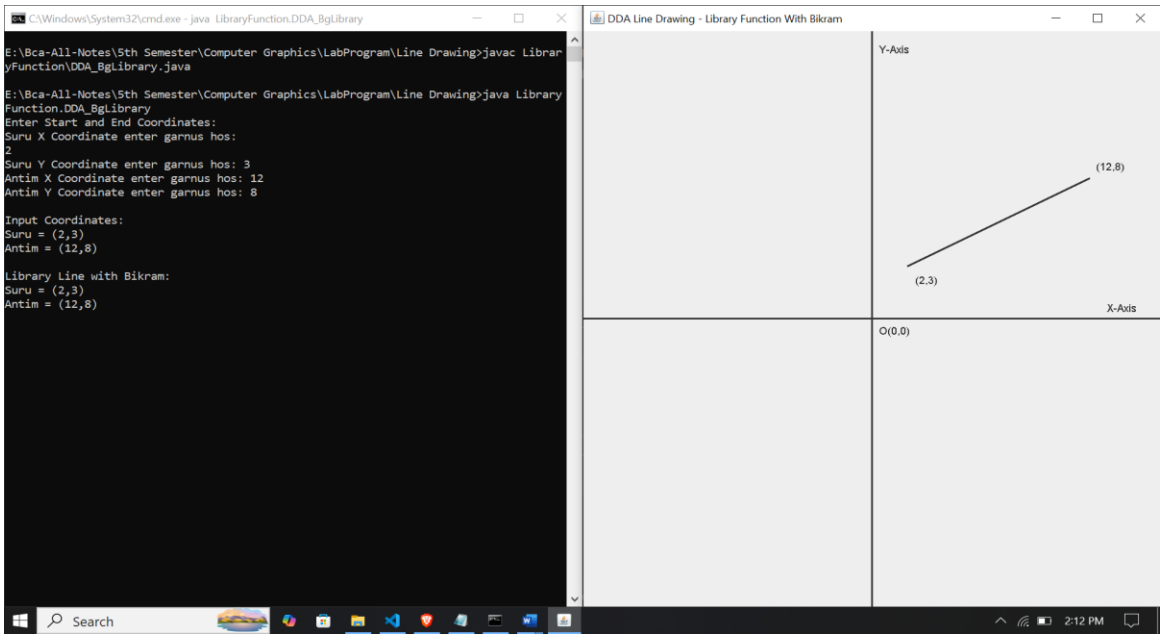
      BGhelperPackageLine.PointPanel AshliPanel = new
BGhelperPackageLine.PointPanel(bg.Xa, bg.Ya, bg.Xb, bg.Yb);

      BGhelperPackageLine.createFrame("DDA Line Drawing - Library Function With
Bikram", AshliPanel);    }

   public static void main(String[] args) {

      new DDA_BgLibrary().BGHehe();

   }}
```



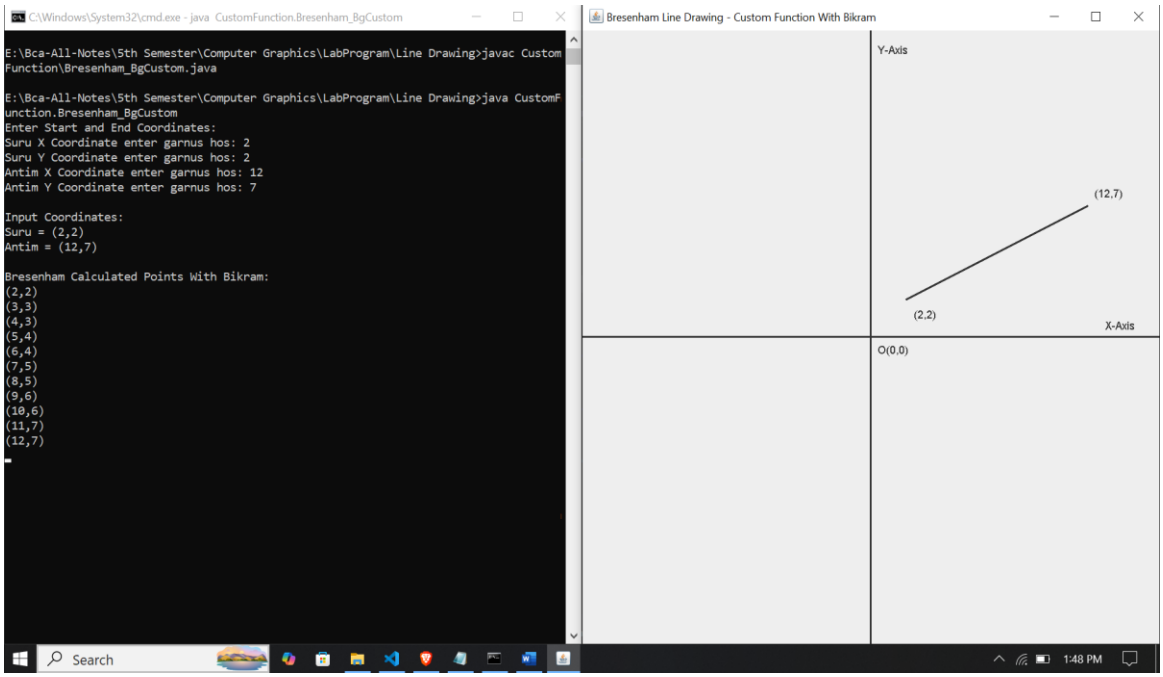
## 4. Bresenham Line Drawing Algorithm using Custom Function.

**Code:**

```
package CustomFunction;
import packageLine.BGhelperPackageLine;
import java.util.Scanner;
public class Bresenham_BgCustom {
   public int[][] BGBresenham(int Xa, int Ya, int Xb, int Yb) {
      int delx = Math.abs(Xb - Xa);
      int dely = Math.abs(Yb - Ya);
      int p = 2 * dely - delx;
      int x, y, xantim;
      if (Xb > Xa) {
         x = Xa;          y = Ya;          xantim = Xb;
      } else {
         x = Xb;          y = Yb;          xantim = Xa;
      }
      int steps = xantim - x;
      int[][] BGPoints = new int[steps + 1][2];
      int Bikram = 0;
      System.out.println("\nBresenham Calculated Points With Bikram:");
      BGPoints[Bikram][0] = x;
      BGPoints[Bikram][1] = y;
      System.out.println("(" + x + "," + y + ")");
      while (x < xantim) {
         x++;
         if (p < 0) {
            p = p + 2 * dely;
         } else {
            y++;
            p = p + 2 * dely - 2 * delx; }
         Bikram++;
```

```
            BGPoints[Bikram][0] = x;      BGPoints[Bikram][1] = y;
            System.out.println("(" + x + "," + y + ")");
        }return BGPoints;}
    public void BGHehe() {
        Scanner sa = new Scanner(System.in);
        BGhelperPackageLine bg = new BGhelperPackageLine();
        bg.inputCoordinate(sa);
        bg.showCoordinate();
        int[][] BGPoints = BGBresenham(bg.Xa, bg.Ya, bg.Xb, bg.Yb);
        BGhelperPackageLine.PointPanel AshliPanel = new
BGhelperPackageLine.PointPanel(bg.Xa, bg.Ya, bg.Xb, bg.Yb,BGPoints);
        BGhelperPackageLine.createFrame("Bresenham Line Drawing - Custom Function
With Bikram", AshliPanel);
    }
    public static void main(String[] args) {
        new Bresenham_BgCustom().BGHehe();
    }
}
```

**Output:**



## 5. Bresenham Line Drawing using Library Funcition.

**Code:**

```
package LibraryFunction;

import packageLine.BGhelperPackageLine;

import java.util.Scanner;

public class DDA_BgLibrary {

   public void BGHehe() {

      Scanner sa = new Scanner(System.in);

      BGhelperPackageLine bg = new BGhelperPackageLine();

      bg.inputCoordinate(sa);bg.showCoordinate();

      System.out.println("\nLibrary Line with Bikram:");

      System.out.println("Suru = (" + bg.Xa + "," + bg.Ya + ")");

      System.out.println("Antim = (" + bg.Xb + "," + bg.Yb + ")");

                  BGhelperPackageLine.PointPanel AshliPanel = new BGhelperPackageLine.PointPanel(bg.Xa, bg.Ya, bg.Xb, bg.Yb);

       BGhelperPackageLine.createFrame("DDA Line Drawing - Library Function With Bikram", AshliPanel);

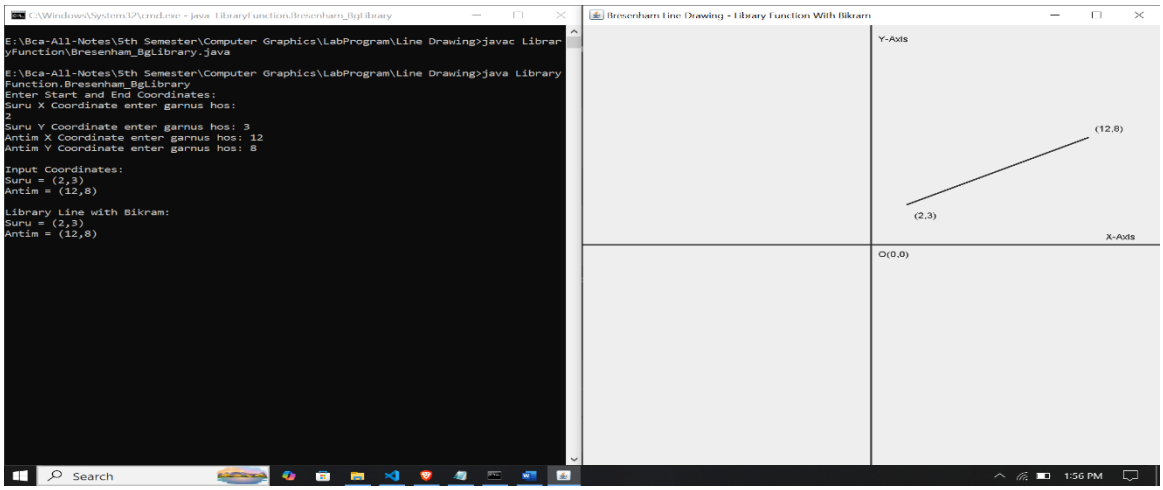
   }

   public static void main(String[] args) {

      new DDA_BgLibrary().BGHehe();

   }

}
```



# IV. Circle Drawing Algorithm

## 1. Program to create package use for input and swing.

**Code:**

```
package packageCircle;
import java.util.Scanner;
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Graphics;
public class BGhelperPackage {
   public static Scanner suru = new Scanner(System.in);
   public static int getRadius() {
      System.out.print("Enter radius: ");
      return suru.nextInt();
   }
   public static void showCircle(int[][] gyawali, String butwal) {
      JFrame antim = new JFrame(butwal);
      antim.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      antim.setSize(700, 700);
      antim.setLocationRelativeTo(null);
      antim.add(new CirclePanel(gyawali));
      antim.setVisible(true);
   }
   public static void showLibraryCircle(int radius, String butwal) {
      JFrame antim = new JFrame(butwal);
      antim.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      antim.setSize(700, 700);
      antim.setLocationRelativeTo(null);
      antim.add(new LibraryCirclePanel(radius));
      antim.setVisible(true);}
   static class CirclePanel extends JPanel {
```

```
    private int[][] gyawali;
    public CirclePanel(int[][] gyawali) {
        this.gyawali = gyawali;
    }
    @Override
    protected void paintComponent(Graphics ashli) {
        super.paintComponent(ashli);
        int suru = getWidth() / 2;
        int antim = getHeight() / 2;
        ashli.drawLine(0, antim, getWidth(), antim);
        ashli.drawLine(suru, 0, suru, getHeight());
        for (int[] butwal : gyawali) {  int x = suru + butwal[0];int y = antim - butwal[1];
            ashli.fillRect(x, y, 1, 1);}
        ashli.drawString("X Axis", getWidth() - 60, antim - 5);
        ashli.drawString("Y Axis", suru + 5, 20);
    }}
static class LibraryCirclePanel extends JPanel {
    private int radius;
    public LibraryCirclePanel(int radius) {
        this.radius = radius;}
    @Override
    protected void paintComponent(Graphics ashli) {
        super.paintComponent(ashli);
        int suru = getWidth() / 2; int antim = getHeight() / 2;
        ashli.drawLine(0, antim, getWidth(), antim);
        ashli.drawLine(suru, 0, suru, getHeight());
        ashli.drawOval(suru - radius, antim - radius, 2 * radius, 2 * radius);
        ashli.drawString("X Axis", getWidth() - 60, antim - 5);
        ashli.drawString("Y Axis", suru + 5, 20);
    }}}
```

## 2. General Circle Drawing using Custom Function.

**Code:**

```
package CustomFunction;

import packageCircle.BGhelperPackage;
import java.util.ArrayList;

public class GeneralCircle_BgCustom {
    public static void main(String[] args) {
        int radius = BGhelperPackage.getRadius();
        ArrayList<int[]> gyawali = new ArrayList<>();

        System.out.println("\nCalculated Points With Bikram:");

        for (int suru = 0; suru <= radius; suru++) {
            int antim = (int) Math.round(Math.sqrt(radius * radius - suru * suru));

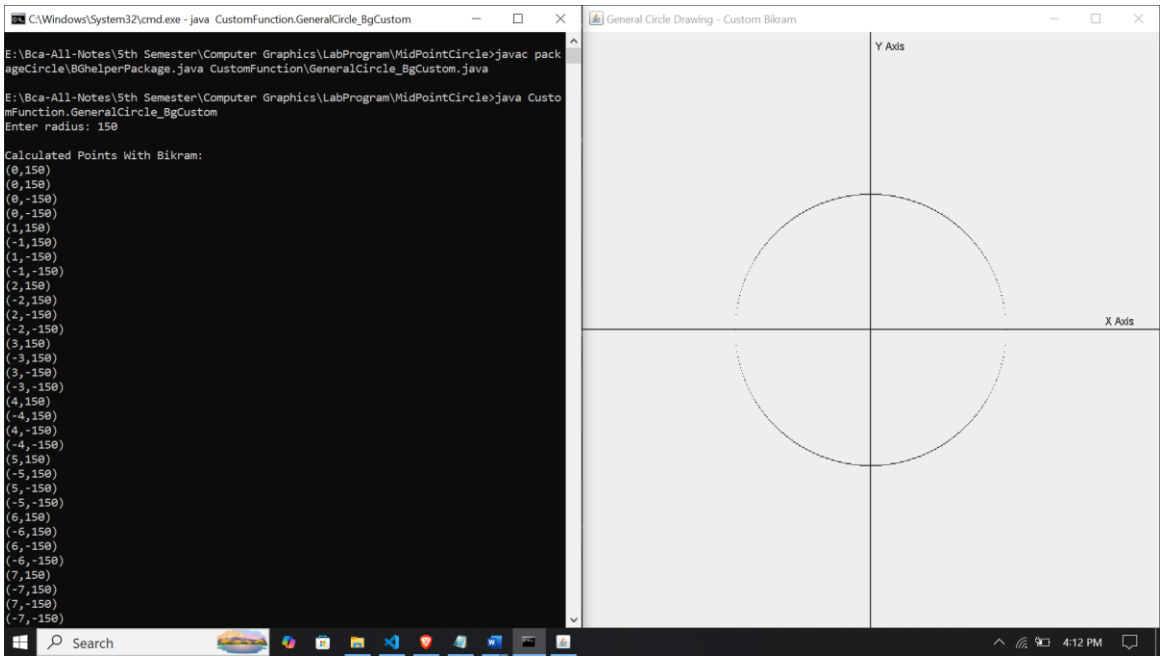
            int[][] butwal = {
                    { suru, antim },
                    { -suru, antim },
                    { suru, -antim },
                    { -suru, -antim }
            };

            for (int[] ashli : butwal) {
                System.out.println("(" + ashli[0] + "," + ashli[1] + ")");
                gyawali.add(ashli);
            }
        }
        System.out.println("Circle drawing completed.");
```

```
        int[][] bghehe = new int[gyawali.size()][2];

        for (int suru = 0; suru < gyawali.size(); suru++) {
            bghehe[suru] = gyawali.get(suru);
        }
        BGhelperPackage.showCircle(bghehe, "General Circle Drawing - Custom Bikram");
    }
}
```

**Output:**



## 3. General Circle Drawing using Library Function

**Code:**

```
package LibraryFunction;
import packageCircle.BGhelperPackage;
import java.util.ArrayList;

public class GeneralCircle_BgLibrary {
   public static void main(String[] args) {
      int radius=BGhelperPackage.getRadius();
      ArrayList<int[]> gyawali=new ArrayList<>();

      System.out.println("\nCalculated Points With Bikram:");


      for(int suru=0;suru<=radius;suru++) {
         int antim=(int)Math.round(Math.sqrt(radius*radius-suru*suru));

         int[][] butwal={
            {suru,antim},
            {-suru,antim},
            {suru,-antim},
            {-suru,-antim}
         };

         for(int[] ashli:butwal) {
            System.out.println("(" + ashli[0] + "," + ashli[1] + ")");
            gyawali.add(ashli);
         }
      }
```

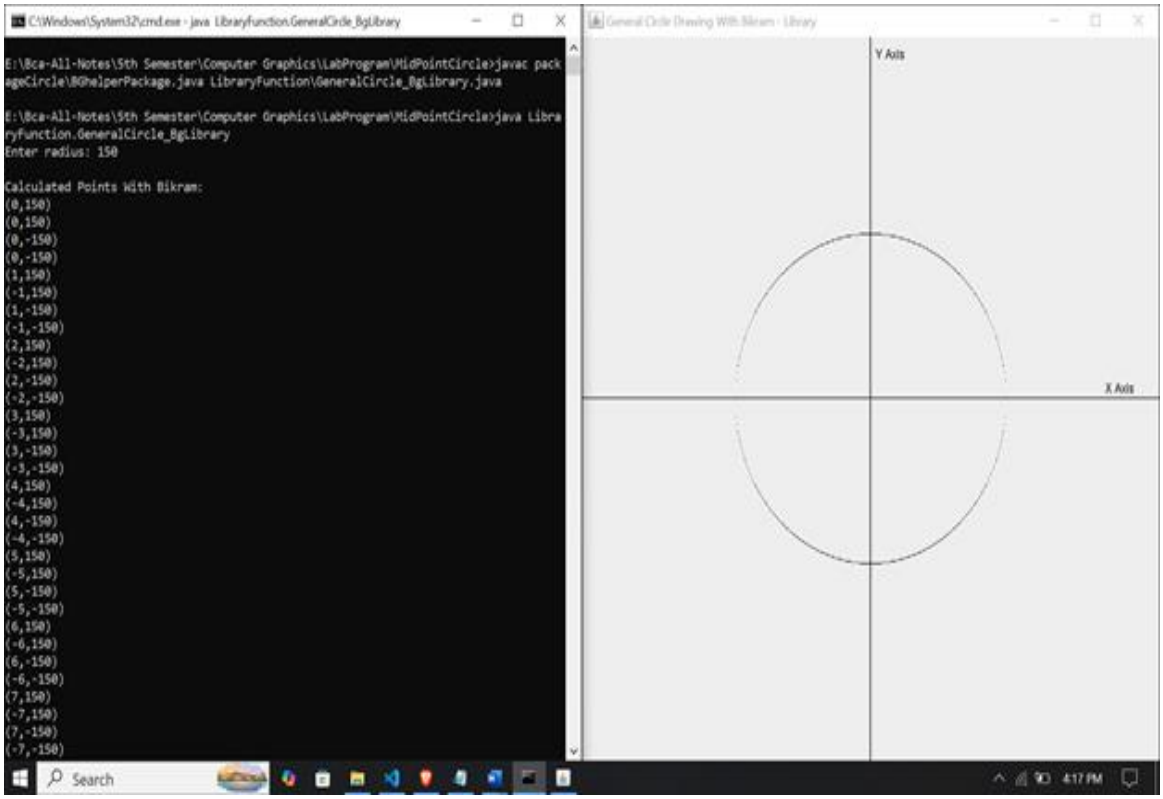
```
        System.out.println("Circle drawing completed.");


        int[][] bghehe=new int[gyawali.size()][2];


        for(int suru=0;suru<gyawali.size();suru++) {
            bghehe[suru]=gyawali.get(suru);
        }


        BGhelperPackage.showCircle(bghehe,"General Circle Drawing With Bikram -
Library");
    }
}
```

**Output:**



## 4. Mid-Point Circle Drawing using Custom Function.

**Code:**

```
package CustomFunction;
import packageCircle.BGhelperPackage;
import java.util.ArrayList;

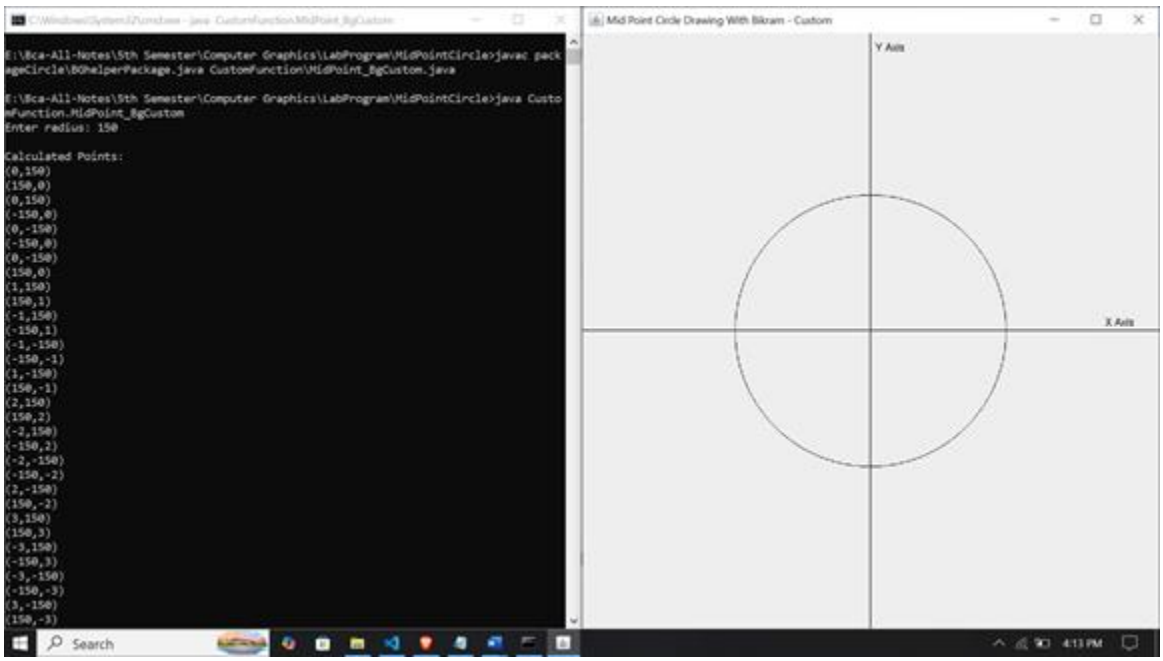
public class MidPoint_BgCustom {
   public static void main(String[] args) {
      int radius = BGhelperPackage.getRadius();
      int suru = 0;
      int antim = radius;
      int ashli = 1 - radius;
      ArrayList<int[]> gyawali = new ArrayList<>();
      System.out.println("\nCalculated Points:");

      while (suru <= antim) {
         int[][] butwal = {
               { suru, antim },
               { antim, suru },
               { -suru, antim },
               { -antim, suru },
               { -suru, -antim },
               { -antim, -suru },
               { suru, -antim },
               { antim, -suru }
         };

         for (int[] bghehe : butwal) {
            System.out.println("(" + bghehe[0] + "," + bghehe[1] + ")");
            gyawali.add(bghehe); }
```

```
            if (ashli < 0) {
                ashli = ashli + 2 * suru + 3;
            } else {
                ashli = ashli + 2 * (suru - antim) + 5;
                antim--;
}
            suru++;
        }
        System.out.println("Circle drawing completed.");
        int[][] bghehe = new int[gyawali.size()][2];
        for (int nsuru = 0; nsuru < gyawali.size(); nsuru++) {
            bghehe[nsuru] = gyawali.get(nsuru);
        }
BGhelperPackage.showCircle(bghehe, "Mid Point Circle Drawing With Bikram - Custom");
    }
}
```

**Output:**

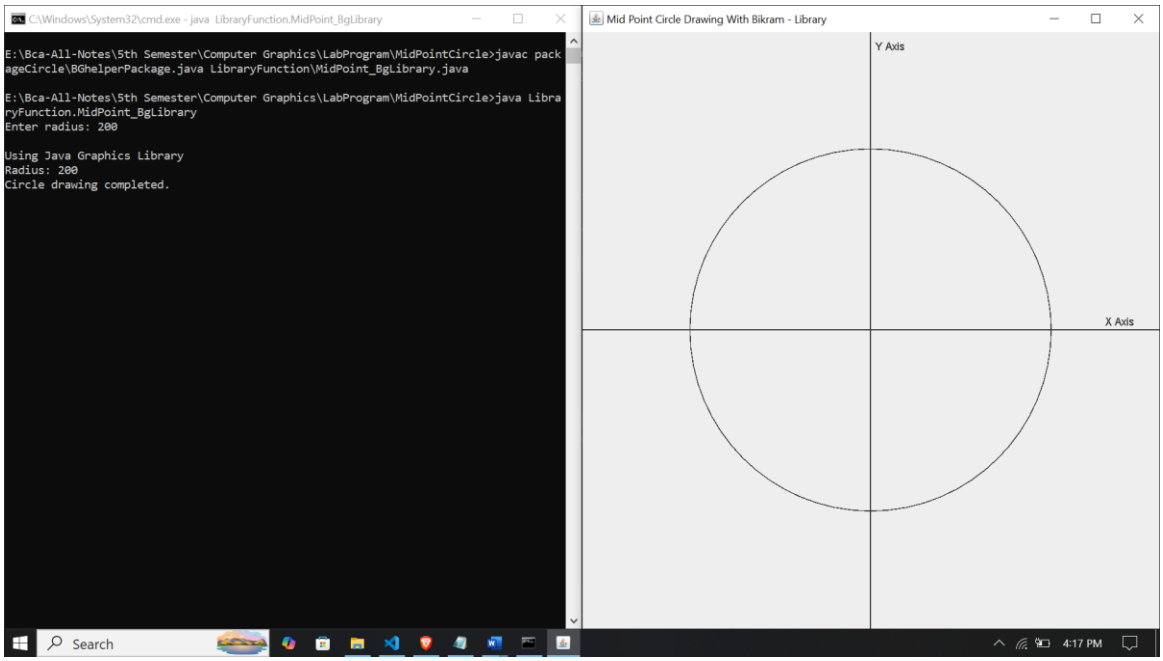


## 5. Mid-Point Circle Drawing using Library Function.

**Code:**

```
package LibraryFunction;
import packageCircle.BGhelperPackage;
public class MidPoint_BgLibrary {
    public static void main(String[] args) {
        int radius = BGhelperPackage.getRadius();
        System.out.println("\nUsing Java Graphics Library");
        System.out.println("Radius: " + radius);
        System.out.println("Circle drawing completed.");
        BGhelperPackage.showLibraryCircle(radius, "Mid Point Circle Drawing With Bikram - Library");
    }
}
```

**Output:**



# V. Mid-Point Ellipse Drawing Algorithm

## 1. Program to create package for input and swing.

**Code:**

```
package packageEllipse;
import java.util.Scanner;
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Graphics;

public class BGhelperPackage {
   public static Scanner suru = new Scanner(System.in);
   public static int getCenterX() {
      System.out.print("Enter center X: ");
      return suru.nextInt();
   }
   public static int getCenterY() {
      System.out.print("Enter center Y: ");
      return suru.nextInt();
   }
   public static int getXRadius() {
      System.out.print("Enter X radius: ");
      return suru.nextInt();
   }
   public static int getYRadius() {
      System.out.print("Enter Y radius: ");
      return suru.nextInt();
   }
   public static void showEllipse(int[][] gyawali, String butwal) {
      JFrame antim = new JFrame(butwal);
```

```
    antim.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    antim.setSize(700, 700);
    antim.setLocationRelativeTo(null);
    antim.add(new EllipsePanel(gyawali));
    antim.setVisible(true);
}
public static void showLibraryEllipse(int Xc, int Yc, int rx, int ry, String butwal) {
    JFrame antim = new JFrame(butwal);
    antim.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    antim.setSize(700, 700);
    antim.setLocationRelativeTo(null);
    antim.add(new LibraryEllipsePanel(Xc, Yc, rx, ry));
    antim.setVisible(true);
}
static class EllipsePanel extends JPanel {
    private int[][] gyawali;
    public EllipsePanel(int[][] gyawali) {
        this.gyawali = gyawali;
    }
    @Override
    protected void paintComponent(Graphics ashli) {
        super.paintComponent(ashli);
        int suru = getWidth() / 2;
        int antim = getHeight() / 2;
        ashli.drawLine(0, antim, getWidth(), antim);
        ashli.drawLine(suru, 0, suru, getHeight());
        for (int[] butwal : gyawali) {
            int x = suru + butwal[0];     int y = antim - butwal[1];
        ashli.fillRect(x, y, 1, 1);
        }
        ashli.drawString("X Axis", getWidth() - 60, antim - 5);
```

```
            ashli.drawString("Y Axis", suru + 5, 20);
        }
    }
    static class LibraryEllipsePanel extends JPanel {
        private int Xc;     private int Yc;
            private int rx;  private int ry;

        public LibraryEllipsePanel(int Xc, int Yc, int rx, int ry) {
            this.Xc = Xc;
            this.Yc = Yc;
            this.rx = rx;
            this.ry = ry;
        }

        @Override
        protected void paintComponent(Graphics ashli) {
            super.paintComponent(ashli);
            int suru = getWidth() / 2;
            int antim = getHeight() / 2;
            ashli.drawLine(0, antim, getWidth(), antim);
            ashli.drawLine(suru, 0, suru, getHeight());
            int centerX = suru + Xc;
            int centerY = antim - Yc;
            ashli.drawOval(centerX - rx, centerY - ry, 2 * rx, 2 * ry);
            ashli.drawString("X Axis", getWidth() - 60, antim - 5);
            ashli.drawString("Y Axis", suru + 5, 20);
        }
    }
}
```

## 2. Ellipse Drawing using Custom Function.

**Code:**

```
package CustomFunction;
import packageEllipse.BGhelperPackage;
import java.util.ArrayList;

public class MidPointEllipse_BgCustom {
   public static void main(String[] args) {
      int Xc = BGhelperPackage.getCenterX();
      int Yc = BGhelperPackage.getCenterY();
      int rx = BGhelperPackage.getXRadius();
      int ry = BGhelperPackage.getYRadius();
      ArrayList<int[]> gyawali = new ArrayList<>();
      int suru = 0;
      int antim = ry;

      double ashli = (double) (ry * ry) - (double) (rx * rx) * ry + (double) (rx * rx) / 4;
      double rx2 = (double) rx * rx;
      double ry2 = (double) ry * ry;

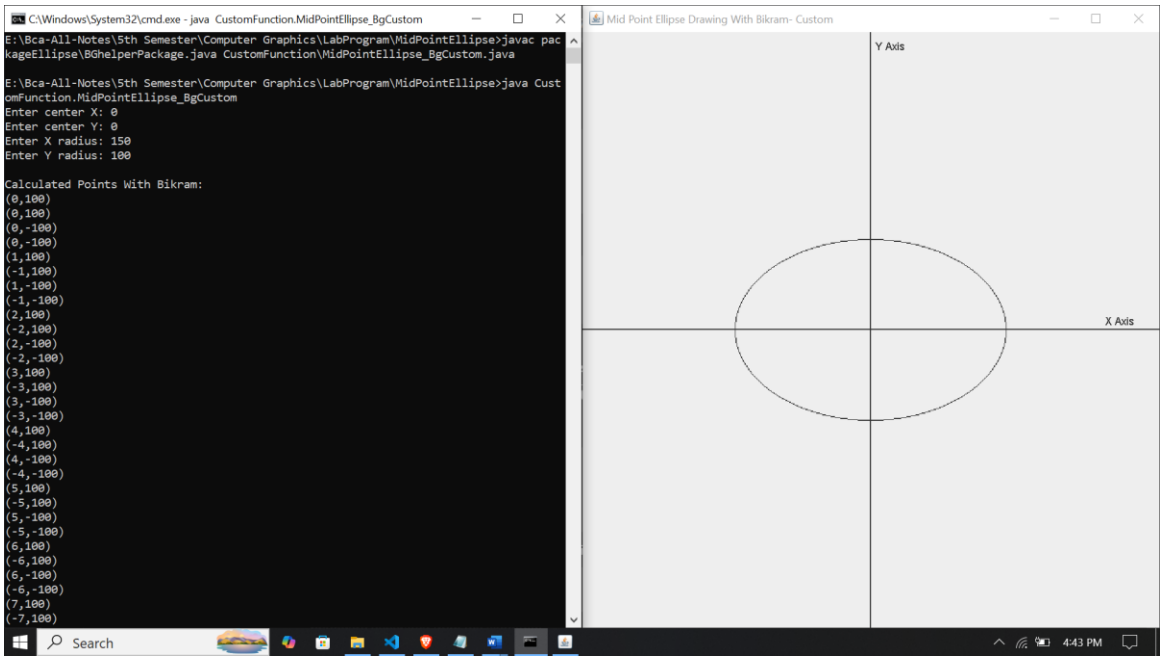
      System.out.println("\nCalculated Points With Bikram:");
      while (2 * ry2 * suru < 2 * rx2 * antim) {
         int[][] butwal = {
               { Xc + suru, Yc + antim },
               { Xc - suru, Yc + antim },
               { Xc + suru, Yc - antim },
               { Xc - suru, Yc - antim }
         };

         for (int[] bghehe : butwal) {
```

```
        System.out.println("(" + bghehe[0] + "," + bghehe[1] + ")");
        gyawali.add(bghehe);
    }
    if (ashli < 0) {
        suru++;
        ashli = ashli + 2 * ry2 * suru + ry2;
    } else {
        suru++;
        antim--;
        ashli = ashli + 2 * ry2 * suru - 2 * rx2 * antim + ry2;
    }
}
double butwalDecision = ry2 * (suru + 0.5) * (suru + 0.5)
        + rx2 * (antim - 1) * (antim - 1)
        - rx2 * ry2;
while (antim >= 0) {
    int[][] bghehe = {
            { Xc + suru, Yc + antim },
            { Xc - suru, Yc + antim },
            { Xc + suru, Yc - antim },
            { Xc - suru, Yc - antim }
    };
    for (int[] ashliPoint : bghehe) {
        System.out.println("(" + ashliPoint[0] + "," + ashliPoint[1] + ")");
        gyawali.add(ashliPoint);
    }
    if (butwalDecision > 0) {
        antim--;
        butwalDecision = butwalDecision - 2 * rx2 * antim + rx2;
    } else {
```

```
            suru++;
            antim--;
            butwalDecision = butwalDecision + 2 * ry2 * suru - 2 * rx2 * antim + rx2;
        }
    }
    int[][] bghehe = new int[gyawali.size()][2];
    for (int suruIndex = 0; suruIndex < gyawali.size(); suruIndex++) {
        bghehe[suruIndex] = gyawali.get(suruIndex);
    }
    BGhelperPackage.showEllipse(bghehe, "Mid Point Ellipse Drawing With Bikram-Custom");
  }
}
```

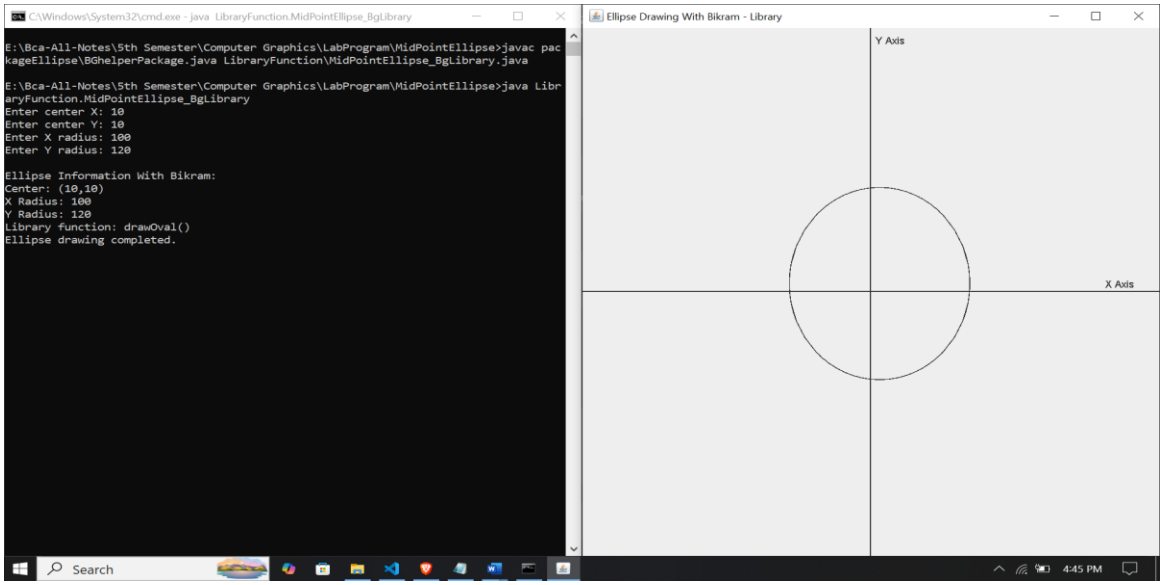
**Output:**



## 3. Ellipse Drawing using Library Function.

**Code:**

```
package LibraryFunction;
import packageEllipse.BGhelperPackage;
public class MidPointEllipse_BgLibrary {
    public static void main(String[] args) {
        int Xc = BGhelperPackage.getCenterX();
        int Yc = BGhelperPackage.getCenterY();
        int rx = BGhelperPackage.getXRadius();
        int ry = BGhelperPackage.getYRadius();
        System.out.println("\nEllipse Information With Bikram:");
        System.out.println("Center: (" + Xc + "," + Yc + ")");
        System.out.println("X Radius: " + rx);
        System.out.println("Y Radius: " + ry);
        System.out.println("Library function: drawOval()");
        System.out.println("Ellipse drawing completed.");
        BGhelperPackage.showLibraryEllipse(
                Xc, Yc, rx, ry, "Ellipse Drawing With Bikram - Library");
    }
}
```



# VI. Clipping Algorithm

## 1. Program to create package for input and swing.

**Code:**

```
package packageClipping;
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;
public class BGhelperPackageClipping {
   public double xmin, ymin, xmax, ymax;
   public double Xa, Ya, Xb, Yb;
   public int plapa;
   public void inputWindow(Scanner sa) {
      System.out.println("\nEnter Clipping Window Coordinates:");
      System.out.print("xmin enter garnu hos: ");
      xmin = sa.nextDouble();
      System.out.print("ymin enter garnu hos: ");
      ymin = sa.nextDouble();
      System.out.print("xmax enter garnu hos: ");
      xmax = sa.nextDouble();
      System.out.print("ymax enter garnu hos: ");
      ymax = sa.nextDouble();
   }
   public void inputPoint(Scanner sa) {
      System.out.print("\nNumber of Points enter garnu hos: ");
      plapa = sa.nextInt();
   }
   public double[][] inputPoints(Scanner sa) {
      double[][] bikramG = new double[plapa][2];
```

```
    for (int Bikram = 0; Bikram < plapa; Bikram++) {
        System.out.print("Point " + (Bikram + 1) + " X enter garnu hos: ");
        bikramG[Bikram][0] = sa.nextDouble();
        System.out.print("Point " + (Bikram + 1) + " Y enter garnu hos: ");
        bikramG[Bikram][1] = sa.nextDouble();
    }
    return bikramG;
}
public void inputLine(Scanner sa) {
    System.out.println("\nEnter Line Coordinates:");
    System.out.print("Suru X Coordinate enter garnu hos: ");
    Xa = sa.nextDouble();
    System.out.print("Suru Y Coordinate enter garnu hos: ");
    Ya = sa.nextDouble();
    System.out.print("Antim X Coordinate enter garnu hos: ");
    Xb = sa.nextDouble();
    System.out.print("Antim Y Coordinate enter garnu hos: ");
    Yb = sa.nextDouble();
}
public double[][] inputPolygon(Scanner sa) {
    System.out.print("\nNumber of Polygon Points enter garnu hos: ");
    plapa = sa.nextInt();
    double[][] gyawaliB = new double[plapa][2];
    for (int Bikram = 0; Bikram < plapa; Bikram++) {
        System.out.print("Point " + (Bikram + 1) + " X enter garnu hos: ");
        gyawaliB[Bikram][0] = sa.nextDouble();
        System.out.print("Point " + (Bikram + 1) + " Y enter garnu hos: ");
        gyawaliB[Bikram][1] = sa.nextDouble();
    }
    return gyawaliB;
```

```
    }
    public static void createFrame(String BGTitle, JPanel AshliPanel) {
        JFrame Bikram = new JFrame(BGTitle + " With Bikram");
        Bikram.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Bikram.setSize(1100, 650);
        Bikram.setLocationRelativeTo(null);
        Bikram.add(AshliPanel);
        Bikram.setVisible(true);
    }
    public static int BGScreenX(double x, double scale, int width) {
        return (int) Math.round(width / 2 + x * scale);
    }
    public static int BGScreenY(double y, double scale, int height) {
        return (int) Math.round(height / 2 - y * scale);
    }
    public static void BGDrawAxes(Graphics2D g2, JPanel panel, double scale) {
        int centerX = panel.getWidth() / 2;
        int centerY = panel.getHeight() / 2;
        g2.drawLine(0, centerY, panel.getWidth(), centerY);
        g2.drawLine(centerX, 0, centerX, panel.getHeight());
        g2.drawString("X-Axis", panel.getWidth() - 60, centerY - 8);
        g2.drawString("Y-Axis", centerX + 8, 20);
        g2.drawString("O(0,0)", centerX + 8, centerY + 18);
    }
    public static void BGDrawWindow(Graphics2D g2, double xmin, double ymin, double
xmax, double ymax, JPanel panel,
            double scale) {
        int X1 = BGScreenX(xmin, scale, panel.getWidth());
        int Y1 = BGScreenY(ymax, scale, panel.getHeight());
        int X2 = BGScreenX(xmax, scale, panel.getWidth());
        int Y2 = BGScreenY(ymin, scale, panel.getHeight());
```

```
    g2.drawRect(X1, Y1, X2 - X1, Y2 - Y1);
    g2.drawString("Clipping Window", X1, Y1 - 8);
}
public static class TwoBoxPanel extends JPanel {
    protected double xmin, ymin, xmax, ymax;
    protected double scale = 22;
    public TwoBoxPanel(double xmin, double ymin, double xmax, double ymax) {
        this.xmin = xmin;
        this.ymin = ymin;
        this.xmax = xmax;
        this.ymax = ymax;
    }
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
        int half = getWidth() / 2;
        g2.drawLine(half, 0, half, getHeight());
        g2.drawString("BEFORE CLIPPING", half / 2 - 65, 25);
        g2.drawString("AFTER CLIPPING", half + half / 2 - 60, 25);
    }
    protected void BGBeforeAxes(Graphics2D g2, int width) {
        int centerX = width / 2;
        int centerY = getHeight() / 2;
        g2.drawLine(0, centerY, width, centerY);
        g2.drawLine(centerX, 0, centerX, getHeight());
        g2.drawString("X-Axis", width - 55, centerY - 8);
        g2.drawString("Y-Axis", centerX + 8, 20);
        g2.drawString("O(0,0)", centerX + 8, centerY + 18);
    }
    protected int BGX(double x, int boxWidth) {
```

```
        return BGScreenX(x, scale, boxWidth);
    }
    protected int BGY(double y) {
        return BGScreenY(y, scale, getHeight());
    }
    protected void BGWindow(Graphics2D g2, int offset, int boxWidth) {
        int X1 = offset + BGX(xmin, boxWidth);
        int Y1 = BGY(ymax);
        int X2 = offset + BGX(xmax, boxWidth);
        int Y2 = BGY(ymin);
        g2.drawRect(X1, Y1, X2 - X1, Y2 - Y1);
        g2.drawString("Clipping Window", X1, Y1 - 8);
    }
  }
  public static class PointPanel extends TwoBoxPanel {
    private double[][] BGOriginal;
    private double[][] BGClipped;
    public PointPanel(double xmin, double ymin, double xmax, double ymax, double[][]
BGOriginal,double[][] BGClipped) {
        super(xmin, ymin, xmax, ymax);
        this.BGOriginal = BGOriginal;
        this.BGClipped = BGClipped;
    }
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
        int half = getWidth() / 2;
        int boxWidth = half - 10;
        BGBeforeAxes(g2, boxWidth);
        BGWindow(g2, 0, boxWidth);
        BGWindow(g2, half, boxWidth);
```

```
        for (int Bikram = 0; Bikram < BGOriginal.length; Bikram++) {
            int X = BGX(BGOriginal[Bikram][0], boxWidth);
            int Y = BGY(BGOriginal[Bikram][1]);
            g2.fillOval(X - 4, Y - 4, 8, 8);
            g2.drawString("P" + (Bikram + 1) + " (" + BGOriginal[Bikram][0] + "," +
BGOriginal[Bikram][1] + ")",X + 6, Y - 6);
        }
        for (int Bikram = 0; Bikram < BGClipped.length; Bikram++) {
            int X = half + BGX(BGClipped[Bikram][0], boxWidth);
            int Y = BGY(BGClipped[Bikram][1]);
            g2.fillOval(X - 4, Y - 4, 8, 8);
            g2.drawString("P" + (Bikram + 1) + " (" + BGClipped[Bikram][0] + "," +
BGClipped[Bikram][1] + ")",X + 6, Y - 6);
        }
    }
  }
  public static class LinePanel extends TwoBoxPanel {
    private double Xa, Ya, Xb, Yb;
    private double[] BGClipped;
    public LinePanel(double xmin, double ymin, double xmax, double ymax, double Xa,
double Ya, double Xb, double Yb,double[] BGClipped) {
        super(xmin, ymin, xmax, ymax);
        this.Xa = Xa;
        this.Ya = Ya;
        this.Xb = Xb;
        this.Yb = Yb;
        this.BGClipped = BGClipped;
    }
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
```

```
            int half = getWidth() / 2;
            int boxWidth = half - 10;
            BGBeforeAxes(g2, boxWidth);
            BGWindow(g2, 0, boxWidth);
            BGWindow(g2, half, boxWidth);
            int SX = BGX(Xa, boxWidth);
            int SY = BGY(Ya);
            int EX = BGX(Xb, boxWidth);
            int EY = BGY(Yb);
            g2.drawLine(SX, SY, EX, EY);
            g2.drawString("Suru (" + Xa + "," + Ya + ")", SX + 8, SY + 20);
            g2.drawString("Antim (" + Xb + "," + Yb + ")", EX + 8, EY - 8);
            if (BGClipped != null) {
               int CX1 = half + BGX(BGClipped[0], boxWidth);
               int CY1 = BGY(BGClipped[1]);
               int CX2 = half + BGX(BGClipped[2], boxWidth);
               int CY2 = BGY(BGClipped[3]);
               g2.drawLine(CX1, CY1, CX2, CY2);
               g2.drawString("Suru (" + BGClipped[0] + "," + BGClipped[1] + ")", CX1 + 8,
CY1 + 20);
               g2.drawString("Antim (" + BGClipped[2] + "," + BGClipped[3] + ")", CX2 + 8,
CY2 - 8);
            }
         }
      }
      public static class PolygonPanel extends TwoBoxPanel {
         private double[][] BGOriginal;
         private double[][] BGClipped;
         public PolygonPanel(double xmin, double ymin, double xmax, double ymax,
double[][] BGOriginal, double[][] BGClipped) {
            super(xmin, ymin, xmax, ymax);
```

```
        this.BGOriginal = BGOriginal;
        this.BGClipped = BGClipped;
    }
    private void BGDrawPolygon(Graphics2D g2, double[][] points, int offset, int
boxWidth) {
        if (points == null || points.length == 0)
            return;
        for (int Bikram = 0; Bikram < points.length; Bikram++) {
            int Antim = (Bikram + 1) % points.length;
            int X1 = offset + BGX(points[Bikram][0], boxWidth);
            int Y1 = BGY(points[Bikram][1]);
            int X2 = offset + BGX(points[Antim][0], boxWidth);
            int Y2 = BGY(points[Antim][1]);
            g2.drawLine(X1, Y1, X2, Y2);
            g2.fillOval(X1 - 3, Y1 - 3, 6, 6);
            g2.drawString("P" + (Bikram + 1), X1 + 6, Y1 - 6);
        }
    }
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
        int half = getWidth() / 2;
        int boxWidth = half - 10;
        BGBeforeAxes(g2, boxWidth);
        BGWindow(g2, 0, boxWidth);
        BGWindow(g2, half, boxWidth);
        BGDrawPolygon(g2, BGOriginal, 0, boxWidth);
        BGDrawPolygon(g2, BGClipped, half, boxWidth);
    }
  }
}
```

## 2. Point Clipping using Custom Function

**Code:**

```
package PointClipping;
import packageClipping.BGhelperPackageClipping;
import java.util.ArrayList;
import java.util.Scanner;
public class PointClipping_BgCustom {
   public boolean BGPointClip(double x, double y, double xmin, double ymin, double
xmax, double ymax) {return x >= xmin && x <= xmax && y >= ymin && y <= ymax;
   }
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackageClipping bg = new BGhelperPackageClipping();
      bg.inputWindow(sa);      bg.inputPoint(sa);
      double[][] Suru = bg.inputPoints(sa);
      ArrayList<double[]> gyawaliB = new ArrayList<>();
      System.out.println("\nPoint Clipping Result:");
      for (int Bikram = 0; Bikram < Suru.length; Bikram++) {
         boolean butwal = BGPointClip(Suru[Bikram][0], Suru[Bikram][1], bg.xmin,
bg.ymin, bg.xmax, bg.ymax);
         System.out.println("P" + (Bikram + 1) + " = (" + Suru[Bikram][0] + "," +
Suru[Bikram][1] + ") -> "+ (butwal ? "ACCEPTED" : "REJECTED"));
         if (butwal)
            gyawaliB.add(Suru[Bikram]);
      }
      double[][] Antim = new double[gyawaliB.size()][2];
      for (int Bikram = 0; Bikram < gyawaliB.size(); Bikram++)
         Antim[Bikram] = gyawaliB.get(Bikram);
      BGhelperPackageClipping.PointPanel AshliPanel =
new BGhelperPackageClipping.PointPanel(bg.xmin, bg.ymin,bg.xmax, bg.ymax, Suru,
Antim);
```

```
        BGhelperPackageClipping.createFrame("Point Clipping - Custom Function",
AshliPanel);
    }
    public static void main(String[] args) {
        new PointClipping_BgCustom().BGHehe();
    }
}
```

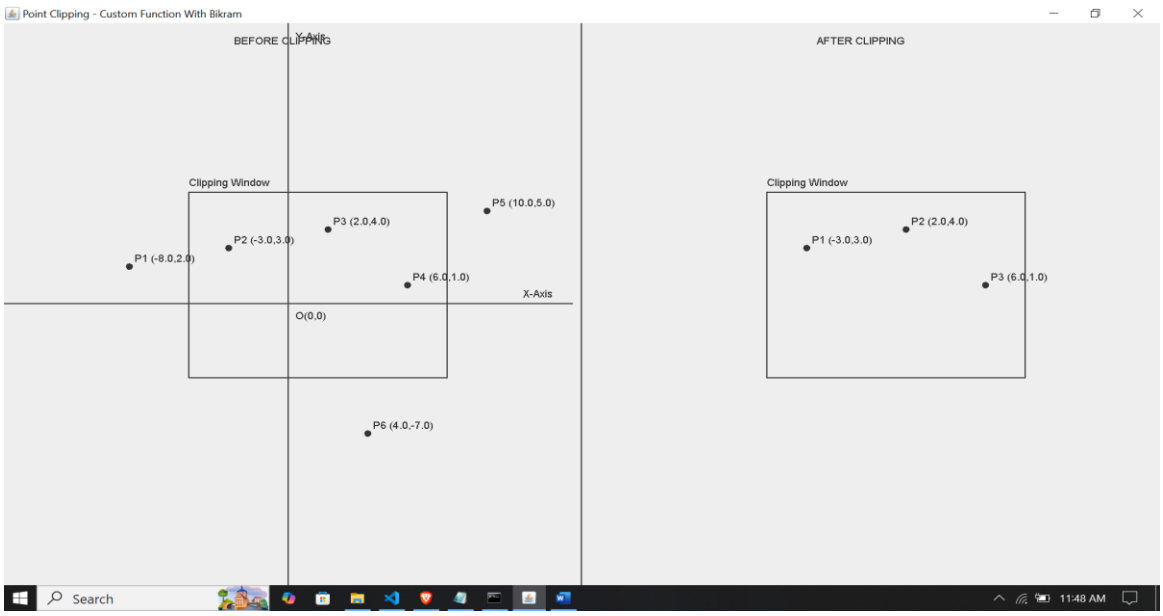
**Output:**

```
C:\Windows\System32\cmd.exe - java  PointClipping.PointClipping_BgCustom
^C
E:\Bca-All-Notes\5th Semester\Computer Graphics\LabProgram\Clipping>java PointClipping.PointClipping_BgCustom

Enter Clipping Window Coordinates:
xmin enter garnu hos: -5
ymin enter garnu hos: -4
xmax enter garnu hos: 8
ymax enter garnu hos: 6

Number of Points enter garnu hos: 6
Point 1 X enter garnu hos: -8
Point 1 Y enter garnu hos: 2
Point 2 X enter garnu hos: -3
Point 2 Y enter garnu hos: 3
Point 3 X enter garnu hos: 2
Point 3 Y enter garnu hos: 4
Point 4 X enter garnu hos: 6
Point 4 Y enter garnu hos: 1
Point 5 X enter garnu hos: 10
Point 5 Y enter garnu hos: 5
Point 6 X enter garnu hos: 4
Point 6 Y enter garnu hos: -7

Point Clipping Result:
P1 = (-8.0,2.0) -> REJECTED
P2 = (-3.0,3.0) -> ACCEPTED
P3 = (2.0,4.0) -> ACCEPTED
P4 = (6.0,1.0) -> ACCEPTED
P5 = (10.0,5.0) -> REJECTED
P6 = (4.0,-7.0) -> REJECTED
```



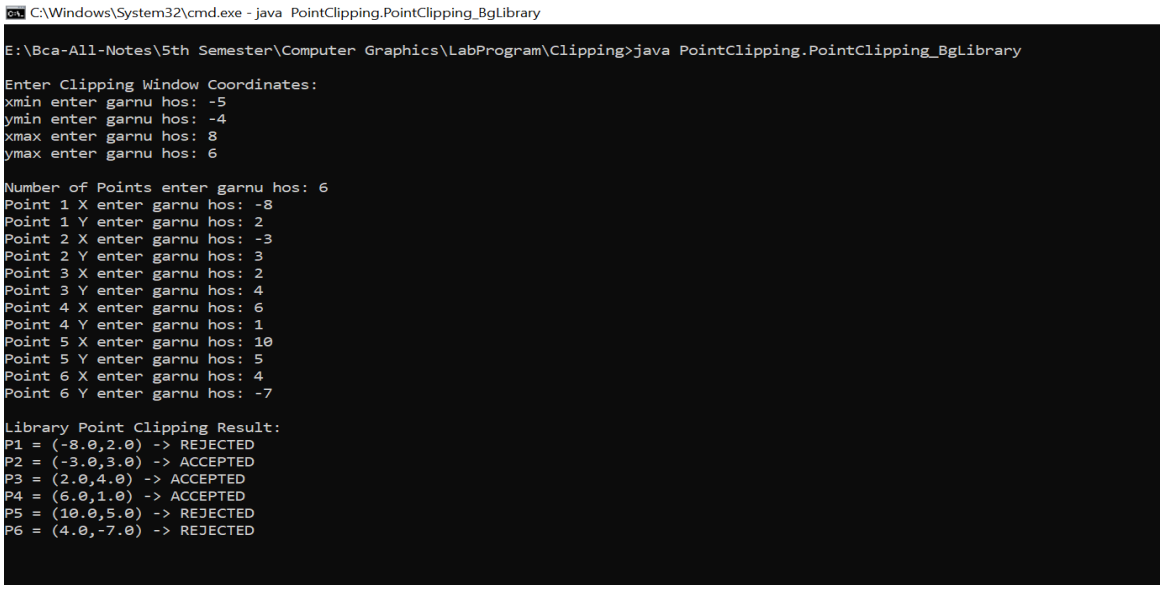
## 3. Point Clipping using Library Function

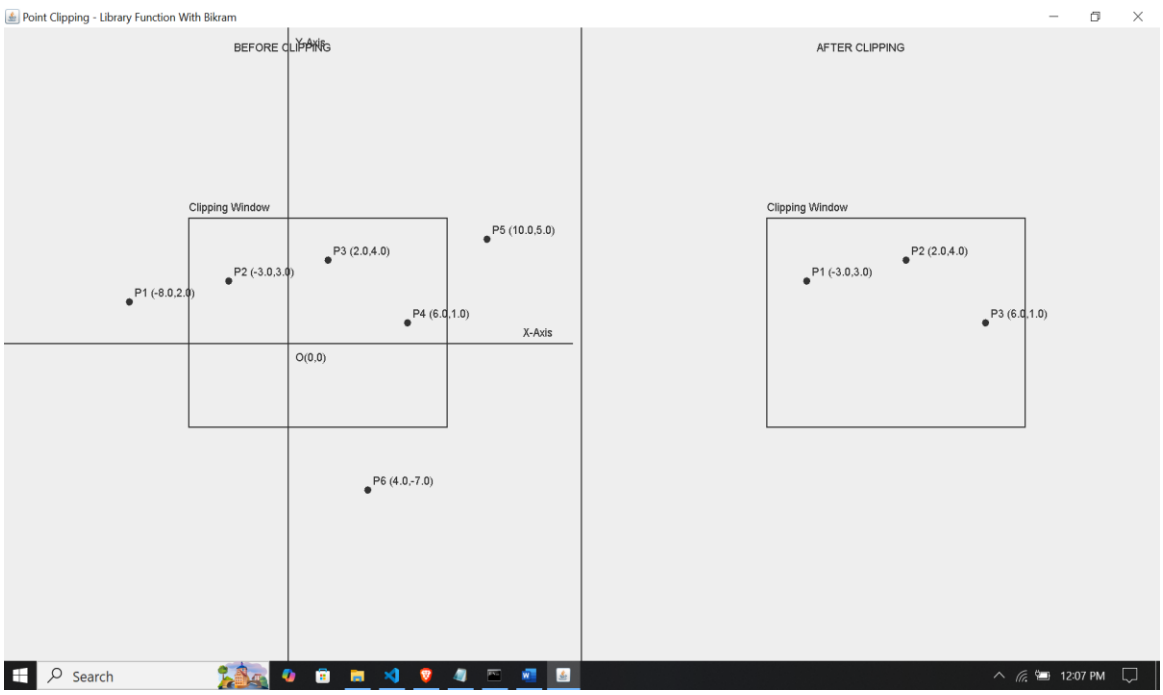
**Code:**

```
package PointClipping;
import packageClipping.BGhelperPackageClipping;
import java.awt.geom.Rectangle2D;
import java.util.ArrayList;
import java.util.Scanner;
public class PointClipping_BgLibrary {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackageClipping bg = new BGhelperPackageClipping();
      bg.inputWindow(sa);      bg.inputPoint(sa);
      double[][] Suru = bg.inputPoints(sa);
      Rectangle2D bikramG = new Rectangle2D.Double(bg.xmin, bg.ymin, bg.xmax -
bg.xmin, bg.ymax - bg.ymin);
      ArrayList<double[]> gyawaliB = new ArrayList<>();
      System.out.println("\nLibrary Point Clipping Result:");
      for (int Bikram = 0; Bikram < Suru.length; Bikram++) {
         boolean butwal = bikramG.contains(Suru[Bikram][0], Suru[Bikram][1]);
         System.out.println("P" + (Bikram + 1) + " = (" + Suru[Bikram][0] + "," +
Suru[Bikram][1] + ") -> "       + (butwal ? "ACCEPTED" : "REJECTED"));
         if (butwal)
            gyawaliB.add(Suru[Bikram]);
      }
      double[][] Antim = new double[gyawaliB.size()][2];
      for (int Bikram = 0; Bikram < gyawaliB.size(); Bikram++)
         Antim[Bikram] = gyawaliB.get(Bikram);
      BGhelperPackageClipping.PointPanel AshliPanel = new
BGhelperPackageClipping.PointPanel(bg.xmin, bg.ymin,bg.xmax, bg.ymax, Suru,
Antim);
```

```
        BGhelperPackageClipping.createFrame("Point Clipping - Library Function",
AshliPanel);
    }
    public static void main(String[] args) {
        new PointClipping_BgLibrary().BGHehe();
    }
}
```

**Output:**



## 4. Cohen Sutherland Line Clipping using Custom Function

**Code:**

```
package LineClipping;
import packageClipping.BGhelperPackageClipping;
import java.util.Scanner;
public class CohenSutherland_BgCustom {
   private final int INSIDE=0;          private final int LEFT=1;
   private final int RIGHT=2; private final int BOTTOM=4;
   private final int TOP=8;
   public int BGRegionCode(double x,double y,double xmin,double ymin,double
xmax,double ymax) {
      int bikram=INSIDE;
      if(x<xmin) bikram|=LEFT;
      else if(x>xmax) bikram|=RIGHT;
      if(y<ymin) bikram|=BOTTOM;
      else if(y>ymax) bikram|=TOP;
      return bikram;
   }
   public double[] BGCohenSutherland(double Xa,double Ya,double Xb,double
Yb,double xmin,double ymin,double xmax,double ymax) {
      int Suru=BGRegionCode(Xa,Ya,xmin,ymin,xmax,ymax);
      int Antim=BGRegionCode(Xb,Yb,xmin,ymin,xmax,ymax);
      while(true) {
         if((Suru|Antim)==0) return new double[]{Xa,Ya,Xb,Yb};
         if((Suru&Antim)!=0) return null;
         double x,y;
         int plapa=(Suru!=0)?Suru:Antim;
         if((plapa&TOP)!=0) {
            x=Xa+(Xb-Xa)*(ymax-Ya)/(Yb-Ya);        y=ymax;
         } else if((plapa&BOTTOM)!=0) {
```

```
            x=Xa+(Xb-Xa)*(ymin-Ya)/(Yb-Ya);
            y=ymin;} else if((plapa&RIGHT)!=0) {
            y=Ya+(Yb-Ya)*(xmax-Xa)/(Xb-Xa);
            x=xmax;} else {
            y=Ya+(Yb-Ya)*(xmin-Xa)/(Xb-Xa);  x=xmin;
         }
         if(plapa==Suru) {
            Xa=x;  Ya=y;          Suru=BGRegionCode(Xa,Ya,xmin,ymin,xmax,ymax);
         } else {
            Xb=x;         Yb=y;  Antim=BGRegionCode(Xb,Yb,xmin,ymin,xmax,ymax);
         }}}
   public void BGHehe() {
      Scanner sa=new Scanner(System.in);
      BGhelperPackageClipping bg=new BGhelperPackageClipping();
      bg.inputWindow(sa);      bg.inputLine(sa);
      int bikramG=BGRegionCode(bg.Xa,bg.Ya,bg.xmin,bg.ymin,bg.xmax,bg.ymax);
      int gyawaliB=BGRegionCode(bg.Xb,bg.Yb,bg.xmin,bg.ymin,bg.xmax,bg.ymax);
      System.out.println("\nCohen-Sutherland Region Codes:");
      System.out.println("Suru Code = "+bikramG);
      System.out.println("Antim Code = "+gyawaliB);
double[]butwal=BGCohenSutherland(bg.Xa,bg.Ya,bg.Xb,bg.Yb,bg.xmin,bg.ymin,bg.xma
x,bg.ymax);
      if(butwal==null) {
         System.out.println("Line is REJECTED.");
      } else {
         System.out.println("\nClipped Line Points:");
         System.out.println("Suru = ("+butwal[0]+","+butwal[1]+")");
         System.out.println("Antim = ("+butwal[2]+","+butwal[3]+")");
         System.out.println("Line is ACCEPTED.");
      }
```

```
        BGhelperPackageClipping.LinePanel AshliPanel=new
BGhelperPackageClipping.LinePanel(bg.xmin,bg.ymin,bg.xmax,bg.ymax,bg.Xa,bg.Ya,b
g.Xb,bg.Yb,butwal);

        BGhelperPackageClipping.createFrame("Cohen-Sutherland Line Clipping - Custom
Function",AshliPanel);

    }

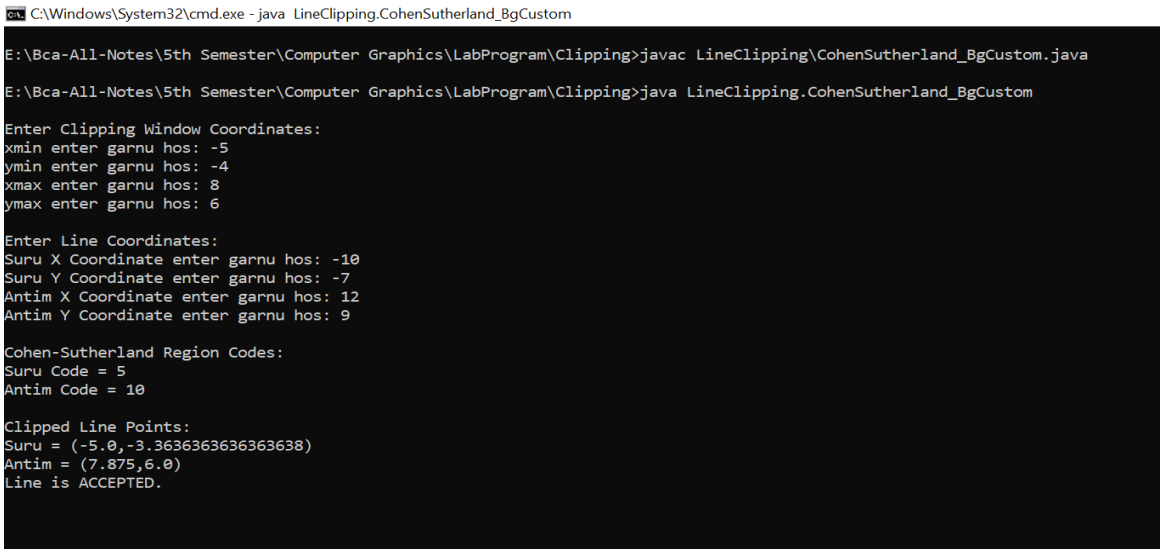
    public static void main(String[] args) {

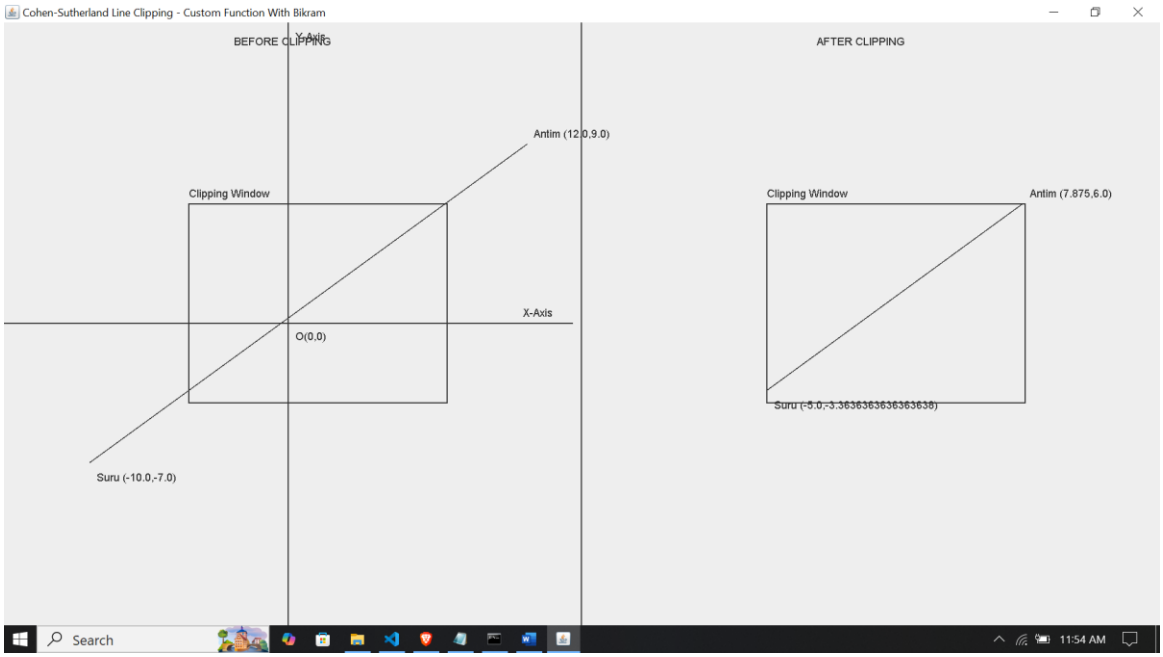
        new CohenSutherland_BgCustom().BGHehe();

    }

}
```

**Output:**



## 5. Cohen Sutherland Line Clipping Algorithm

**Code:**

```
package LineClipping;
import packageClipping.BGhelperPackageClipping;
import java.awt.geom.Line2D;
import java.util.Scanner;

public class CohenSutherland_BgLibrary {
   public void BGHehe() {
      Scanner sa = new Scanner(System.in);
      BGhelperPackageClipping bg = new BGhelperPackageClipping();
      bg.inputWindow(sa);
      bg.inputLine(sa);
      System.out.println("\nLibrary Line Clipping:");
      System.out.println("Suru = (" + bg.Xa + "," + bg.Ya + ")");
      System.out.println("Antim = (" + bg.Xb + "," + bg.Yb + ")");
      System.out.println("Line clipping is handled using Java Graphics clipping.");
      double[] butwal = null;
      Line2D gyawaliB = new Line2D.Double(bg.Xa, bg.Ya, bg.Xb, bg.Yb);

      if (gyawaliB.intersects(bg.xmin, bg.ymin, bg.xmax - bg.xmin, bg.ymax - bg.ymin)
|| bg.xmin <= bg.Xa && bg.Xa <= bg.xmax && bg.ymin <= bg.Ya && bg.Ya <=
bg.ymax
|| bg.xmin <= bg.Xb && bg.Xb <= bg.xmax && bg.ymin <= bg.Yb && bg.Yb <=
bg.ymax) {
         butwal = new double[] { bg.Xa, bg.Ya, bg.Xb, bg.Yb };
      }
      BGhelperPackageClipping.LinePanel AshliPanel = new
BGhelperPackageClipping.LinePanel(bg.xmin, bg.ymin, bg.xmax,
            bg.ymax, bg.Xa, bg.Ya, bg.Xb, bg.Yb, butwal);
```

```
        BGhelperPackageClipping.createFrame("Cohen-Sutherland Line Clipping - Library Function", AshliPanel);
    }

    public static void main(String[] args) {
        new CohenSutherland_BgLibrary().BGHehe();
    }
}
```

**Output:**

```
C:\Windows\System32\cmd.exe - java  LineClipping.CohenSutherland_BgLibrary

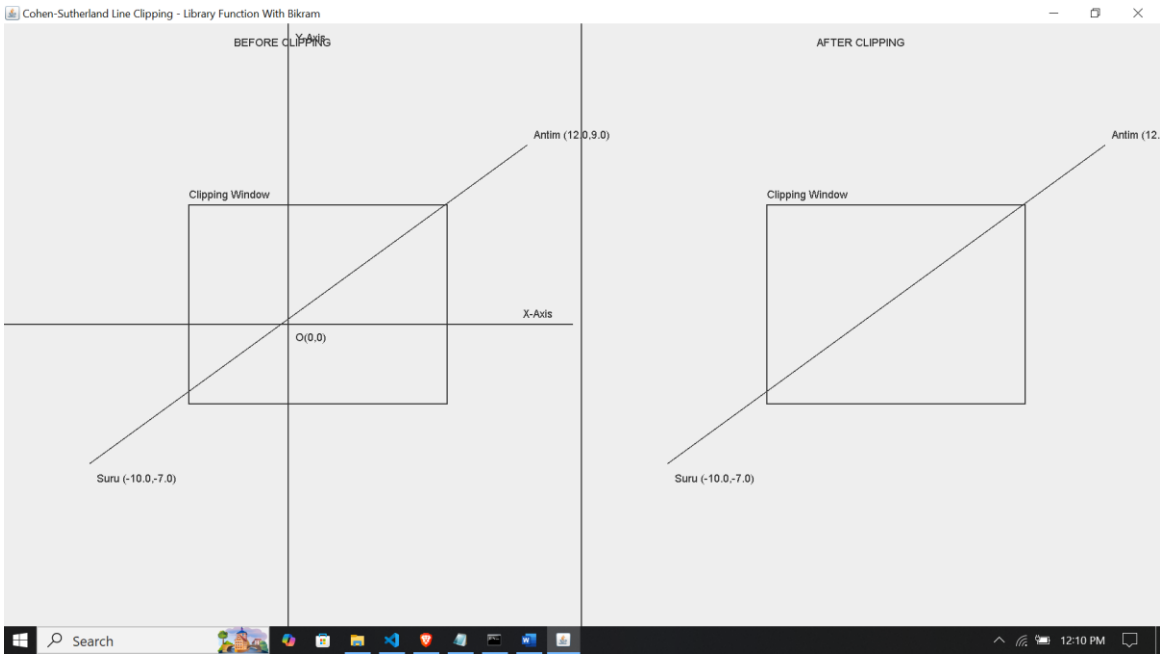
E:\Bca-All-Notes\5th Semester\Computer Graphics\LabProgram\Clipping>javac LineClipping\CohenSutherland_BgLibrary.java

E:\Bca-All-Notes\5th Semester\Computer Graphics\LabProgram\Clipping>java LineClipping.CohenSutherland_BgLibrary

Enter Clipping Window Coordinates:
xmin enter garnu hos: -5
ymin enter garnu hos: -4
xmax enter garnu hos: 8
ymax enter garnu hos: 6

Enter Line Coordinates:
Suru X Coordinate enter garnu hos: -10
Suru Y Coordinate enter garnu hos: -7
Antim X Coordinate enter garnu hos: 12
Antim Y Coordinate enter garnu hos: 9

Library Line Clipping:
Suru = (-10.0,-7.0)
Antim = (12.0,9.0)
Line clipping is handled using Java Graphics clipping.
```



## 6. Sutherland Hodgman Polygon Clipping Algorithm

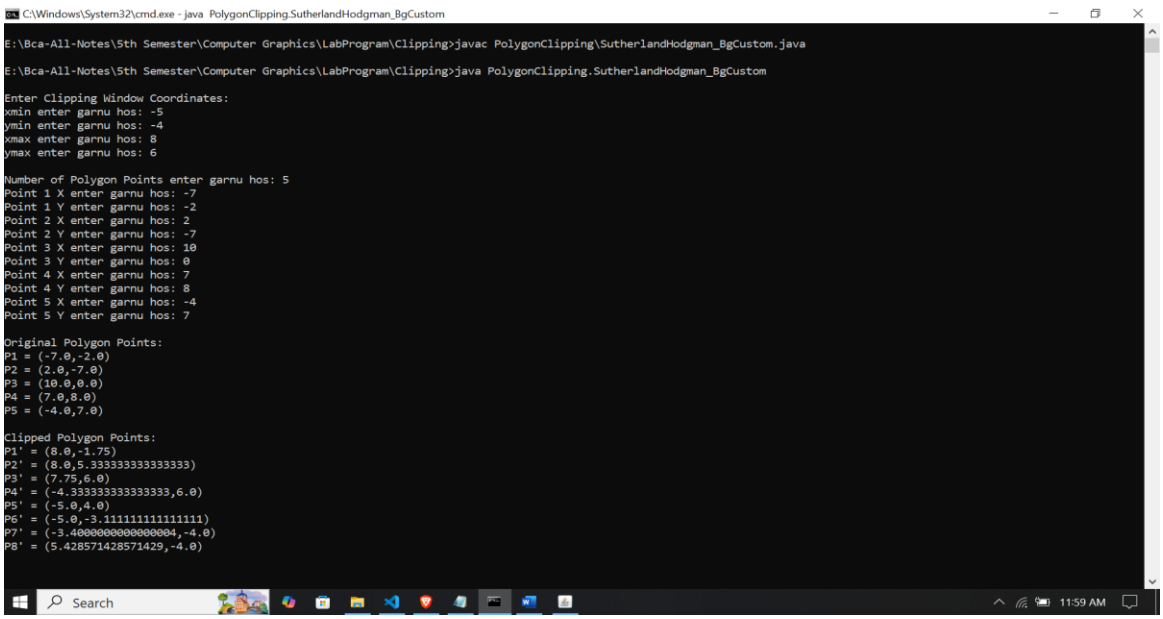
**Code:**

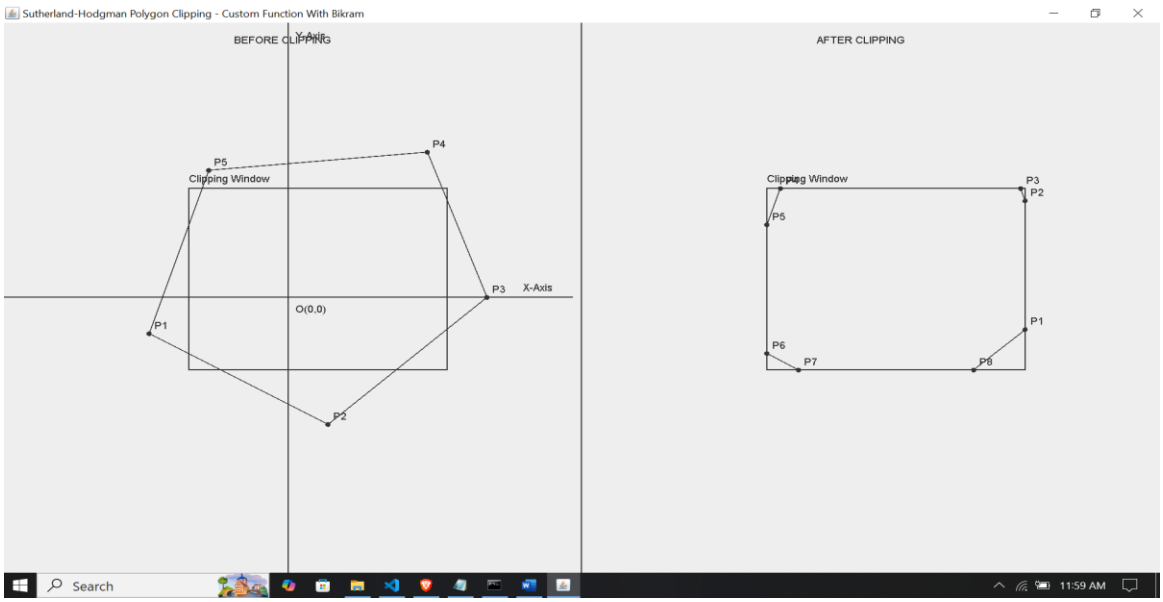
```
package PolygonClipping;
import packageClipping.BGhelperPackageClipping;
import java.util.ArrayList;
import java.util.Scanner;
public class SutherlandHodgman_BgCustom {
   private boolean BGInside(double[] point, int edge, double value) {
      if (edge == 0)      return point[0] >= value;
      if (edge == 1)      return point[0] <= value;
      if (edge == 2)      return point[1] >= value;
      return point[1] <= value;           }
   private double[] BGIntersection(double[] Suru, double[] Antim, int edge, double value)
{
      double x1 = Suru[0], y1 = Suru[1], x2 = Antim[0], y2 = Antim[1];
      if (edge == 0 || edge == 1) {       double y = y1 + (y2 - y1) * (value - x1) / (x2 - x1);
         return new double[] { value, y };
      }
      double x = x1 + (x2 - x1) * (value - y1) / (y2 - y1);
      return new double[] { x, value };
   }
   private double[][] BGClipEdge(double[][] points, int edge, double value) {
      ArrayList<double[]> butwal = new ArrayList<>();
      if (points.length == 0)     return new double[0][2];
      for (int Bikram = 0; Bikram < points.length; Bikram++) {
         double[] Suru = points[Bikram];
         double[] Antim = points[(Bikram + 1) % points.length];
         boolean SuruInside = BGInside(Suru, edge, value);
         boolean AntimInside = BGInside(Antim, edge, value);
         if (SuruInside && AntimInside) {      butwal.add(Antim);
         } else if (SuruInside && !AntimInside) {
```

```
butwal.add(BGIntersection(Suru, Antim, edge, value));
            } else if (!SuruInside && AntimInside) {
                butwal.add(BGIntersection(Suru, Antim, edge, value));
                butwal.add(Antim);
            }         }
        double[][] gyawaliB = new double[butwal.size()][2];
        for (int Bikram = 0; Bikram < butwal.size(); Bikram++)
            gyawaliB[Bikram] = butwal.get(Bikram);
        return gyawaliB;
    }
    public double[][] BGSutherlandHodgman(double[][] points, double xmin, double ymin,
double xmax, double ymax) {
        double[][] bikramG = points;
bikramG = BGClipEdge(bikramG, 0, xmin); bikramG = BGClipEdge(bikramG, 1, xmax);
bikramG = BGClipEdge(bikramG, 2, ymin); bikramG = BGClipEdge(bikramG, 3, ymax);
return bikramG;          }
    public void BGHehe() {
        Scanner sa = new Scanner(System.in);
        BGhelperPackageClipping bg = new BGhelperPackageClipping();
        bg.inputWindow(sa);
        double[][] Suru = bg.inputPolygon(sa);
 double[][] Antim = BGSutherlandHodgman(Suru, bg.xmin, bg.ymin, bg.xmax,
bg.ymax);
        System.out.println("\nOriginal Polygon Points:");
        for (int Bikram = 0; Bikram < Suru.length; Bikram++)
            System.out.println("P" + (Bikram + 1) + " = (" + Suru[Bikram][0] + "," +
Suru[Bikram][1] + ")");
        System.out.println("\nClipped Polygon Points:");
        for (int Bikram = 0; Bikram < Antim.length; Bikram++)
            System.out.println("P" + (Bikram + 1) + "' = (" + Antim[Bikram][0] + "," +
Antim[Bikram][1] + ")");
```

```
        BGhelperPackageClipping.PolygonPanel AshliPanel = new
BGhelperPackageClipping.PolygonPanel(bg.xmin, bg.ymin,
                bg.xmax, bg.ymax, Suru, Antim);
        BGhelperPackageClipping.createFrame("Sutherland-Hodgman Polygon Clipping -
Custom Function", AshliPanel);
    }
    public static void main(String[] args) {
        new SutherlandHodgman_BgCustom().BGHehe();
    }     }
```

**Output:**



## 7. Sutherland Hodgeman Polygon Clipping Algorithm

**Code:**

```
package PolygonClipping;
import packageClipping.BGhelperPackageClipping;
import java.awt.geom.Area;
import java.awt.geom.Path2D;
import java.awt.geom.PathIterator;
import java.awt.geom.Rectangle2D;
import java.util.ArrayList;
import java.util.Scanner;

public class SutherlandHodgman_BgLibrary {
   public double[][] BGClipPolygon(double[][] points, double xmin, double ymin, double xmax, double ymax) {

       if (points.length == 0)
           return new double[0][2];
       Path2D.Double Suru = new Path2D.Double();
       Suru.moveTo(points[0][0], points[0][1]);
       for (int Bikram = 1; Bikram < points.length; Bikram++)
          Suru.lineTo(points[Bikram][0], points[Bikram][1]);
       Suru.closePath();
       Area gyawaliB = new Area(Suru);
       Area butwal = new Area(new Rectangle2D.Double(xmin, ymin, xmax - xmin, ymax - ymin));
       gyawaliB.intersect(butwal);
       PathIterator plapa = gyawaliB.getPathIterator(null);
       double[] data = new double[6];
       ArrayList<double[]> result = new ArrayList<>();
```

```
        while (!plapa.isDone()) {
            int Bikram = plapa.currentSegment(data);
            if (Bikram == PathIterator.SEG_MOVETO || Bikram ==
PathIterator.SEG_LINETO)
                result.add(new double[] { data[0], data[1] });
            plapa.next();
        }

        double[][] Antim = new double[result.size()][2];
        for (int Bikram = 0; Bikram < result.size(); Bikram++)
            Antim[Bikram] = result.get(Bikram);
        return Antim;
    }

    public void BGHehe() {
        Scanner sa = new Scanner(System.in);
        BGhelperPackageClipping bg = new BGhelperPackageClipping();
        bg.inputWindow(sa);
        double[][] Suru = bg.inputPolygon(sa);
        double[][] Antim = BGClipPolygon(Suru, bg.xmin, bg.ymin, bg.xmax, bg.ymax);
        System.out.println("\nLibrary Polygon Clipping Result:");
        for (int Bikram = 0; Bikram < Antim.length; Bikram++)
            System.out.println("P" + (Bikram + 1) + "' = (" + Antim[Bikram][0] + "," +
Antim[Bikram][1] + ")");
        BGhelperPackageClipping.PolygonPanel AshliPanel = new
BGhelperPackageClipping.PolygonPanel(bg.xmin, bg.ymin,
                bg.xmax, bg.ymax, Suru, Antim);
        BGhelperPackageClipping.createFrame("Sutherland-Hodgman Polygon Clipping -
Library Function", AshliPanel);
    }

    public static void main(String[] args) {
```

```
        new SutherlandHodgman_BgLibrary().BGHehe();
    }
}
```

**Output:**

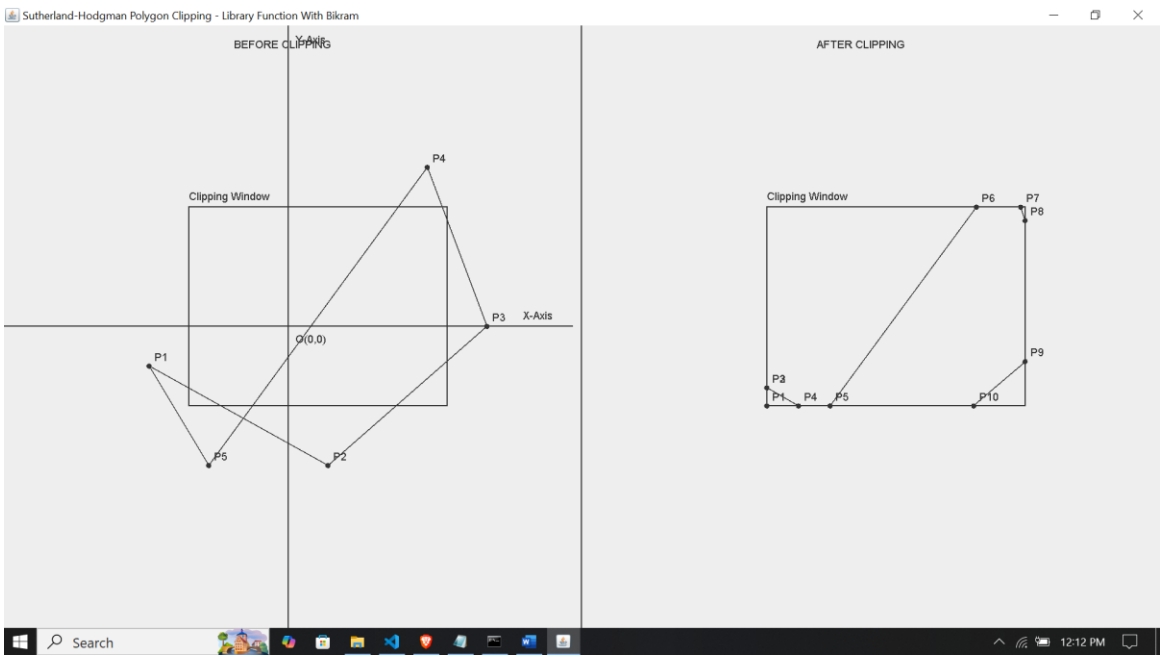
```
C:\Windows\System32\cmd.exe - java  PolygonClipping.SutherlandHodgman_BgLibrary
E:\Bca-All-Notes\5th Semester\Computer Graphics\LabProgram\Clipping>javac PolygonClipping\SutherlandHodgman_BgLibrary.java

E:\Bca-All-Notes\5th Semester\Computer Graphics\LabProgram\Clipping>java PolygonClipping.SutherlandHodgman_BgLibrary

Enter Clipping Window Coordinates:
xmin enter garnu hos: -5
ymin enter garnu hos: -4
xmax enter garnu hos: 8
ymax enter garnu hos: 6

Number of Polygon Points enter garnu hos: 5
Point 1 X enter garnu hos: -7
Point 1 Y enter garnu hos: -2
Point 2 X enter garnu hos: 2
Point 2 Y enter garnu hos: -7
Point 3 X enter garnu hos: 10
Point 3 Y enter garnu hos: 0
Point 4 X enter garnu hos: 7
Point 4 Y enter garnu hos: 8
Point 5 X enter garnu hos: -4
Point 5 Y enter garnu hos: -7

Library Polygon Clipping Result:
P1' = (-5.0,-4.0)
P2' = (-5.0,-3.1111111111111116)
P3' = (-4.999999999999999,-3.1111111111111116)
P4' = (-3.4,-4.0)
P5' = (-1.8000000000000003,-4.0)
P6' = (5.533333333333333,6.0)
P7' = (7.75,6.0)
P8' = (8.0,5.333333333333333)
P9' = (8.0,-1.75)
P10' = (5.428571428571429,-4.0)
```



# VII. Area Filling Algorithm

## 1. Program to create a Package for input and swing

**Code:**

```
package packageAreaFilling;
import java.awt.*;
import java.awt.image.BufferedImage;
import javax.swing.*;

public class BGAreaFilling {
   public static BufferedImage createImage(int bikramG, int gyawaliB) {
      BufferedImage butwal = new BufferedImage(bikramG, gyawaliB,
BufferedImage.TYPE_INT_RGB);
      Graphics2D plapa = butwal.createGraphics();
      plapa.setColor(Color.WHITE);
      plapa.fillRect(0, 0, bikramG, gyawaliB);
      plapa.dispose();
      return butwal;
   }
   public static void drawBoundary(BufferedImage butwal, int suruX, int suruY, int
antimX, int antimY) {
      Graphics2D bikram = butwal.createGraphics();
      bikram.setColor(Color.BLACK);
      bikram.drawRect(suruX, suruY, antimX - suruX, antimY - suruY);
      bikram.dispose();
   }
   public static int getPixelColor(BufferedImage butwal, int bikramG, int gyawaliB) {
      if (bikramG < 0 || bikramG >= butwal.getWidth() || gyawaliB < 0 || gyawaliB >=
butwal.getHeight()) {
         return Color.BLACK.getRGB();
```

```
        }
        return butwal.getRGB(bikramG, gyawaliB);
    }
    public static void setPixelColor(BufferedImage butwal, int bikramG, int gyawaliB,
Color plapa) {
        if (bikramG >= 0 && bikramG < butwal.getWidth() && gyawaliB >= 0 &&
gyawaliB < butwal.getHeight()) {
            butwal.setRGB(bikramG, gyawaliB, plapa.getRGB());
        }
    }
    public static void showImage(BufferedImage butwal, String bikram) {
        JFrame suru = new JFrame(bikram);
        suru.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        suru.add(new JPanel() {
            protected void paintComponent(Graphics gyawaliB) {
                super.paintComponent(gyawaliB);
                gyawaliB.drawImage(butwal, 0, 0, null);
            }
            public Dimension getPreferredSize() {
                return new Dimension(butwal.getWidth(), butwal.getHeight());
            }
        });
        suru.pack();
        suru.setLocationRelativeTo(null);
        suru.setVisible(true);
    }
}
```

## 2. Boundary-Fill Algorithm using Custom Function

**Code:**

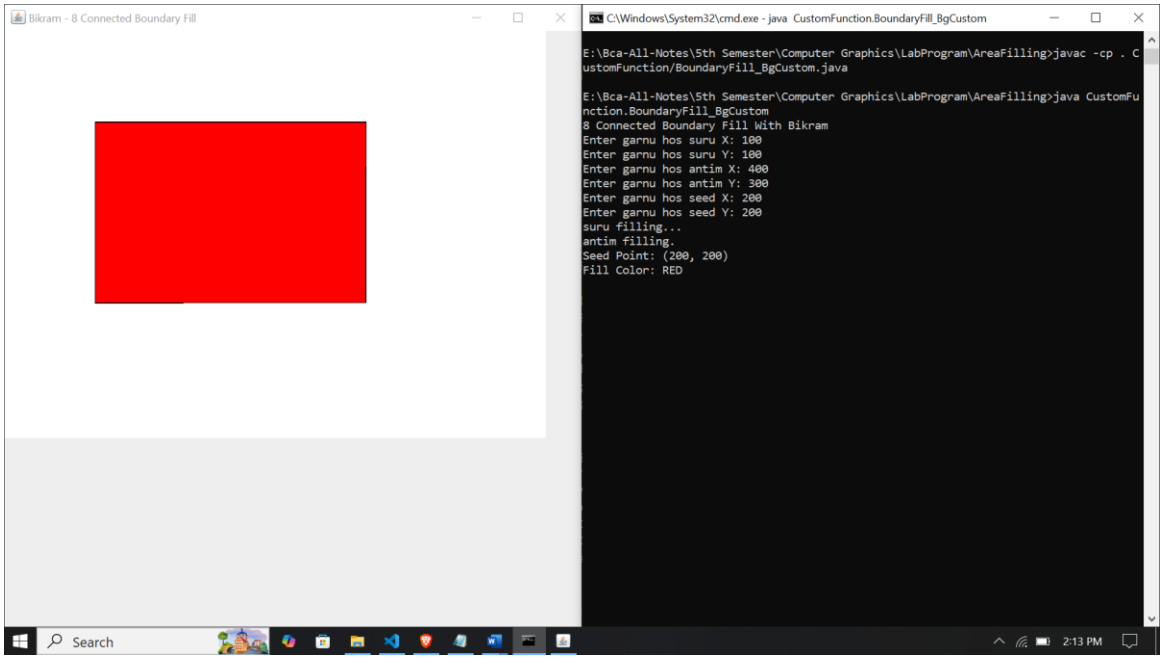
```
package CustomFunction;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.util.Stack;
import java.util.Scanner;
import packageAreaFilling.BGAreaFilling;

public class BoundaryFill_BgCustom {
   static void boundaryFill8(BufferedImage butwal, int bikramG, int gyawaliB, Color boundaryColor, Color fillColor) {
      Stack<Point> plapa = new Stack<>();
      plapa.push(new Point(bikramG, gyawaliB));
      int[] suruX = { -1, 0, 1, -1, 1, -1, 0, 1 };
      int[] suruY = { -1, -1, -1, 0, 0, 1, 1, 1 };
      while (!plapa.isEmpty()) {
         Point bikram = plapa.pop();
         int x = bikram.x;
         int y = bikram.y;
         int currentColor = BGAreaFilling.getPixelColor(butwal, x, y);
         if (currentColor != boundaryColor.getRGB() && currentColor != fillColor.getRGB()) {
            BGAreaFilling.setPixelColor(butwal, x, y, fillColor);
            for (int bikramgg = 0; bikramgg < 8; bikramgg++) {
               int nextX = x + suruX[bikramgg];
               int nextY = y + suruY[bikramgg];
               int nextColor = BGAreaFilling.getPixelColor(butwal, nextX, nextY);
               if (nextColor != boundaryColor.getRGB() && nextColor != fillColor.getRGB()) {
```

```
                plapa.push(new Point(nextX, nextY));
            }
        }
    }
}
public static void main(String[] args) {
    Scanner bikram = new Scanner(System.in);
    int canvasWidth = 600;
    int canvasHeight = 450;
    System.out.println("8 Connected Boundary Fill With Bikram");
    System.out.print("Enter garnu hos suru X: ");
    int suruX = bikram.nextInt();
    System.out.print("Enter garnu hos suru Y: ");
    int suruY = bikram.nextInt();
    System.out.print("Enter garnu hos antim X: ");
    int antimX = bikram.nextInt();
    System.out.print("Enter garnu hos antim Y: ");
    int antimY = bikram.nextInt();
    System.out.print("Enter garnu hos seed X: ");
    int seedX = bikram.nextInt();
    System.out.print("Enter garnu hos seed Y: ");
    int seedY = bikram.nextInt();
    BufferedImage butwal = BGAreaFilling.createImage(canvasWidth, canvasHeight);
    BGAreaFilling.drawBoundary(butwal, suruX, suruY, antimX, antimY);
    System.out.println("suru filling...");
    boundaryFill8(butwal, seedX, seedY, Color.BLACK, Color.RED);
    System.out.println("antim filling.");
    System.out.println("Seed Point: (" + seedX + ", " + seedY + ")");
    System.out.println("Fill Color: RED");
```

```
        BGAreaFilling.showImage(butwal, "Bikram - 8 Connected Boundary Fill");
        bikram.close();
    }
}
```

**Output:**



## 3. Boundary-Fill Algorithm using Library Function.

**Code:**

```
package LibraryFunction;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.util.Scanner;
import packageAreaFilling.BGAreaFilling;

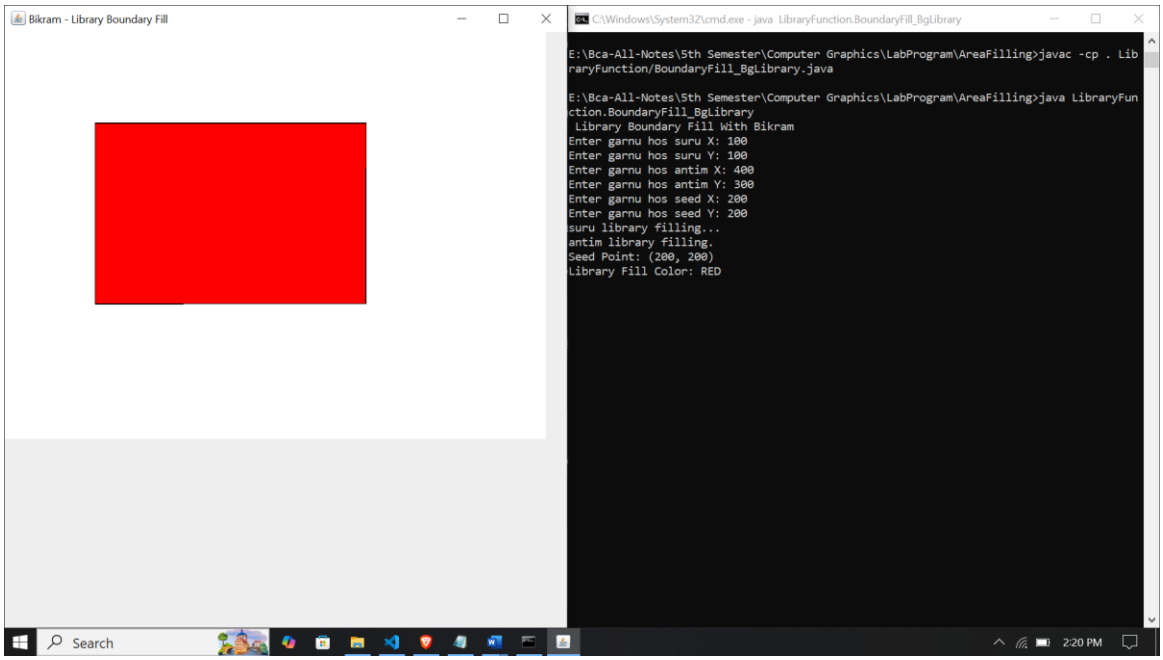
public class BoundaryFill_BgLibrary {
    public static void main(String[] args) {
        Scanner bikram = new Scanner(System.in);
        int canvasWidth = 600;
        int canvasHeight = 450;
        System.out.println(" Library Boundary Fill With Bikram");
        System.out.print("Enter garnu hos suru X: ");
        int suruX = bikram.nextInt();
        System.out.print("Enter garnu hos suru Y: ");
        int suruY = bikram.nextInt();
        System.out.print("Enter garnu hos antim X: ");
        int antimX = bikram.nextInt();
        System.out.print("Enter garnu hos antim Y: ");
        int antimY = bikram.nextInt();
        System.out.print("Enter garnu hos seed X: ");
        int seedX = bikram.nextInt();
        System.out.print("Enter garnu hos seed Y: ");
        int seedY = bikram.nextInt();

        BufferedImage butwal = BGAreaFilling.createImage(canvasWidth, canvasHeight);
        BGAreaFilling.drawBoundary(butwal, suruX, suruY, antimX, antimY);
```

```
        Graphics2D gyawaliB = butwal.createGraphics();
        gyawaliB.setColor(Color.RED);
        gyawaliB.fillRect(suruX + 1, suruY + 1, antimX - suruX - 1, antimY - suruY - 1);
        gyawaliB.setColor(Color.BLACK);
        gyawaliB.drawRect(suruX, suruY, antimX - suruX, antimY - suruY);
        gyawaliB.dispose();

        System.out.println("suru library filling...");
        System.out.println("antim library filling.");
        System.out.println("Seed Point: (" + seedX + ", " + seedY + ")");
        System.out.println("Library Fill Color: RED");
        BGAreaFilling.showImage(butwal, "Bikram - Library Boundary Fill");
        bikram.close();
    }
}
```

**Output:**



## 4. Flood-Fill Algorithm using Custom Function

**Code:**

```
package CustomFunction;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.util.Stack;
import java.util.Scanner;
import packageAreaFilling.BGAreaFilling;

public class FloodFill_BgCustom {
   static void floodFill8(BufferedImage butwal, int bikramG, int gyawaliB, Color
oldColor, Color newColor) {
      if (oldColor.getRGB() == newColor.getRGB()) {
         return;
      }

      Stack<Point> plapa = new Stack<>();
      plapa.push(new Point(bikramG, gyawaliB));
      int[] suruX = { -1, 0, 1, -1, 1, -1, 0, 1 };
      int[] suruY = { -1, -1, -1, 0, 0, 1, 1, 1 };

      while (!plapa.isEmpty()) {
         Point bikram = plapa.pop();
         int x = bikram.x;
         int y = bikram.y;

         if (BGAreaFilling.getPixelColor(butwal, x, y) == oldColor.getRGB()) {
            BGAreaFilling.setPixelColor(butwal, x, y, newColor);
```

```
            for (int gyawaliBi = 0; gyawaliBi < 8; gyawaliBi++) {
                int nextX = x + suruX[gyawaliBi];
                int nextY = y + suruY[gyawaliBi];

                if (BGAreaFilling.getPixelColor(butwal, nextX, nextY) ==
oldColor.getRGB()) {
                    plapa.push(new Point(nextX, nextY));
                }
            }
        }
    }
  }

  public static void main(String[] args) {
    Scanner bikram = new Scanner(System.in);
    int canvasWidth = 600;
    int canvasHeight = 450;

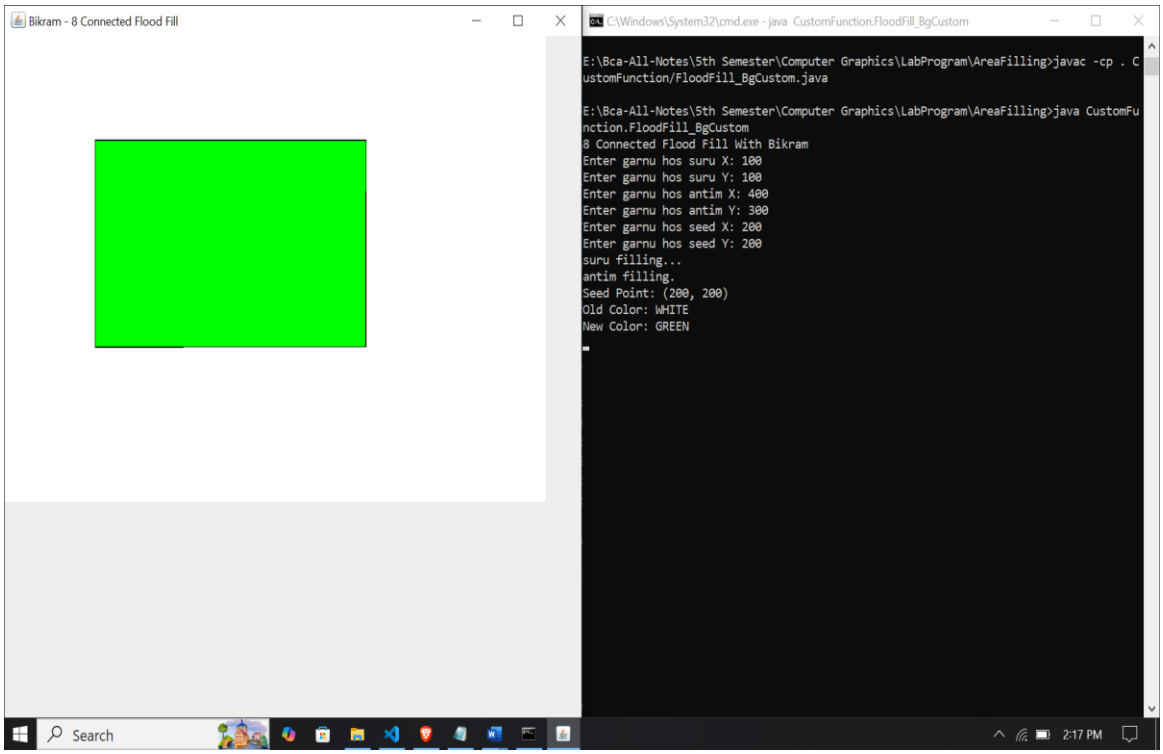
    System.out.println("8 Connected Flood Fill With Bikram");
    System.out.print("Enter garnu hos suru X: ");
    int suruX = bikram.nextInt();
    System.out.print("Enter garnu hos suru Y: ");
    int suruY = bikram.nextInt();
    System.out.print("Enter garnu hos antim X: ");
    int antimX = bikram.nextInt();
    System.out.print("Enter garnu hos antim Y: ");
    int antimY = bikram.nextInt();
    System.out.print("Enter garnu hos seed X: ");
    int seedX = bikram.nextInt();
    System.out.print("Enter garnu hos seed Y: ");
    int seedY = bikram.nextInt();
```

```
        BufferedImage butwal = BGAreaFilling.createImage(canvasWidth, canvasHeight);
        BGAreaFilling.drawBoundary(butwal, suruX, suruY, antimX, antimY);

        System.out.println("suru filling...");
        floodFill8(butwal, seedX, seedY, Color.WHITE, Color.GREEN);
        System.out.println("antim filling.");
        System.out.println("Seed Point: (" + seedX + ", " + seedY + ")");
        System.out.println("Old Color: WHITE");
        System.out.println("New Color: GREEN");

        BGAreaFilling.showImage(butwal, "Bikram - 8 Connected Flood Fill");
        bikram.close();
    }
}
```

**Output:**



## 5. Flood-Fill Algorithm using Library Function

**Code:**

```
package LibraryFunction;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.util.Scanner;
import packageAreaFilling.BGAreaFilling;

public class FloodFill_BgLibrary {
    public static void main(String[] args) {
        Scanner bikram = new Scanner(System.in);
        int canvasWidth = 600;
        int canvasHeight = 450;
        System.out.println(" Library Flood Fill With Bikram");
        System.out.print("Enter garnu hos suru X: ");
        int suruX = bikram.nextInt();
        System.out.print("Enter garnu hos suru Y: ");
        int suruY = bikram.nextInt();
        System.out.print("Enter garnu hos antim X: ");
        int antimX = bikram.nextInt();
        System.out.print("Enter garnu hos antim Y: ");
        int antimY = bikram.nextInt();
        System.out.print("Enter garnu hos seed X: ");
        int seedX = bikram.nextInt();
        System.out.print("Enter garnu hos seed Y: ");
        int seedY = bikram.nextInt();
        BufferedImage butwal = BGAreaFilling.createImage(canvasWidth, canvasHeight);
        BGAreaFilling.drawBoundary(butwal, suruX, suruY, antimX, antimY);
        Graphics2D gyawaliB = butwal.createGraphics();
        gyawaliB.setColor(Color.GREEN);
```

```
        gyawaliB.fillRect(suruX + 1, suruY + 1, antimX - suruX - 1, antimY - suruY - 1);
        gyawaliB.setColor(Color.BLACK);
        gyawaliB.drawRect(suruX, suruY, antimX - suruX, antimY - suruY);
        gyawaliB.dispose();
        System.out.println("suru library filling...");
        System.out.println("antim library filling.");
        System.out.println("Seed Point: (" + seedX + ", " + seedY + ")");
        System.out.println("Library Fill Color: GREEN");
        BGAreaFilling.showImage(butwal, "Bikram - Library Flood Fill");
        bikram.close();
    }
}
```

**Output:**

